Checks:

- Shape is exactly (12, 12)

- All values are finite (no NaN, inf, -inf)

- Matrix is symmetric (rtol=0, atol=1e-12)

- Diagonal is exactly zero (rtol=0, atol=1e-12)

- No negative weights (all values >= -1e-12)

Raises RuntimeError if any check fails.

"""

matrix = np.asarray(matrix, dtype=float)

if matrix.shape != (12, 12):

raise RuntimeError(

f"{key} shape {matrix.shape} != (12, 12)"

)

if not np.all(np.isfinite(matrix)):

```
raise RuntimeError(

f"{key} contains non-finite values"

)



if not np.allclose(matrix, matrix.T, rtol=0.0, atol=1e-12):

raise RuntimeError(f"{key} is not symmetric")



if not np.allclose(np.diag(matrix), 0.0, rtol=0.0, atol=1e-12):

raise RuntimeError(

f"{key} diagonal is not zero"

)



if np.any(matrix &lt; -1e-12):

raise RuntimeError(

f"{key} contains negative weights"

)
```

return matrix

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: str | Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics for both horizon systems.

For each seed, rho, branch, and horizon system:

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets:

- Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

- Similarity target: 0.9 * S_late_continue (for appropriate horizon system)

4. Calculate:

- first_crossing_time: first time metric > target

- sustained_time: earliest time after which metric stays above target for rest of trajectory

- held_time: earliest time after which metric stays above target for at least T_window duration

- censoring indicators for each event type

- restricted_time: horizon for censored events, event_time for observed events

- auc_deficit: one-sided integral of (continue_trajectory - branch_trajectory) using trapezoidal rule

- t_rec / T_window: normalized recovery times

5. Output separate rows for fixed_physical and normalized_EMA horizon systems

Returns DataFrame with complete recovery metrics.

"""

Convert to Path for consistency

npz_path = Path(npz_path)

Get unique seeds, rho values, and branches from the data

seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

```
branches = sorted(df["branch"].unique())



# Build complete list of required keys for all conditions

# We need: sample_times, density_trajectory, similarity_trajectory for all seed/rho/branch

all_keys = set()

for seed in seeds:

for rho in rho_values:

for branch in branches:

prefix = f"{branch}_seed_{seed}_rho_{rho}"

all_keys.update([

f"{prefix}_sample_times",

f"{prefix}_density_trajectory",

f"{prefix}_similarity_trajectory",

])



# Load all required data in one operation

print(f" Loading {len(all_keys)} trajectory arrays from NPZ...")
```

```
npz_data = load_npz_selective(npz_path, keys=list(all_keys))



# Check for missing keys - this should be fatal for production

missing_keys = [k for k in all_keys if k not in npz_data]

if missing_keys:

raise RuntimeError(

f"Missing trajectory keys in NPZ: {missing_keys[:10]}... "

f"({len(missing_keys)} total missing). "

f"This indicates schema mismatch or incomplete simulation."

)



recovery_rows = []



for seed in seeds:

for rho in rho_values:

# Get T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho)]["T_window"].iloc[0])
```

Get continue branch reference values for this seed/rho

cont_row = df[(df["seed"] == seed) & (df["rho"] == rho) & (df["branch"] == "continue")].iloc[0]

Get reference targets for both horizon systems

Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

density_target_continue = 0.9 * float(cont_row["D_B_final"])

Similarity targets for both horizon systems

similarity_target_fixed = 0.9 * float(cont_row["Weighted_similarity_late_fixed_physical"])

similarity_target_normalized = 0.9 * float(cont_row["Weighted_similarity_late_normalized_EMA"])

for branch in branches:

INCLUDE continue branch for DID calculation

For continue: compare trajectory with itself, AUC deficit = 0

This makes recovery DID well-defined: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

prefix = f"{branch}_seed_{seed}_rho_{rho}"

cont_prefix = f"continue_seed_{seed}_rho_{rho}"

```
# Load trajectory data

try:

sample_times = np.asarray(npz_data[f"{prefix}_sample_times"], dtype=float)

density_trajectory = np.asarray(npz_data[f"{prefix}_density_trajectory"], dtype=float)

similarity_trajectory = np.asarray(npz_data[f"{prefix}_similarity_trajectory"], dtype=float)


# Load continue (reference) trajectory data

cont_sample_times = np.asarray(npz_data[f"{cont_prefix}_sample_times"], dtype=float)

cont_density_trajectory = np.asarray(npz_data[f"{cont_prefix}_density_trajectory"], dtype=float)

cont_similarity_trajectory = np.asarray(npz_data[f"{cont_prefix}_similarity_trajectory"], dtype=float)


except Exception as e:

raise RuntimeError(f"Error loading trajectory data for seed={seed}, rho={rho}, branch={branch}:
{e}")


# P0 #3: Validate trajectory lengths match sample_times

if not (len(sample_times) == len(density_trajectory) == len(similarity_trajectory)):

raise RuntimeError(
```

```
f"Trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"times={len(sample_times)}, density={len(density_trajectory)},
similarity={len(similarity_trajectory)}"

)


# P0 #3: Also validate that trajectory lengths match between branch and continue

if len(sample_times) != len(cont_sample_times):

raise RuntimeError(

f"Sample times length mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(sample_times)} times, continue has {len(cont_sample_times)}"

)

if len(density_trajectory) != len(cont_density_trajectory):

raise RuntimeError(

f"Density trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(density_trajectory)} points, continue has {len(cont_density_trajectory)}"

)

if len(similarity_trajectory) != len(cont_similarity_trajectory):

raise RuntimeError(

f"Similarity trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
```

```
f"branch has {len(similarity_trajectory)} points, continue has {len(cont_similarity_trajectory)}"

)


# P0 #4: Validate exact saved-time schedule

if not np.array_equal(sample_times, EXPECTED_SAMPLE_TIMES):

raise RuntimeError(

f"Unexpected sample-time schedule for seed={seed}, rho={rho}, branch={branch}. "

f"Expected first={EXPECTED_SAMPLE_TIMES[0]}, last={EXPECTED_SAMPLE_TIMES[-1]},
n={EXPECTED_N_SAMPLES}"

)

if not np.array_equal(cont_sample_times, EXPECTED_SAMPLE_TIMES):

raise RuntimeError(

f"Unexpected continue sample-time schedule for seed={seed}, rho={rho}. "

f"Expected first={EXPECTED_SAMPLE_TIMES[0]}, last={EXPECTED_SAMPLE_TIMES[-1]},
n={EXPECTED_N_SAMPLES}"

)


# P0 #5: Validate trajectory ranges

# Density must be finite and in [0, 1]
```

```
if not np.all(np.isfinite(density_trajectory)):

raise RuntimeError(

f"Density trajectory contains non-finite values for seed={seed}, rho={rho}, branch={branch}"

)

if np.any((density_trajectory &lt; 0.0) | (density_trajectory &gt; 1.0)):

raise RuntimeError(

f"Density trajectory contains values outside [0,1] for seed={seed}, rho={rho}, branch={branch}"

)


# Similarity NaN is permitted for a zero-vector cosine.

# Infinity is never permitted.

if np.any(np.isinf(similarity_trajectory)):

raise RuntimeError(

f"Similarity trajectory contains infinity for "

f"seed={seed}, rho={rho}, branch={branch}"

)

finite_similarity = similarity_trajectory[np.isfinite(similarity_trajectory)]

if len(finite_similarity) &gt; 0 and np.any((finite_similarity &lt; -1.0 - 1e-12) | (finite_similarity &gt;
1.0 + 1e-12)):
```

```
raise RuntimeError(
f"Similarity trajectory contains values outside [-1,1] for seed={seed}, rho={rho}, branch={branch}"
)

# Validate continue trajectories as well
if not np.all(np.isfinite(cont_density_trajectory)):
raise RuntimeError(
f"Continue density trajectory contains non-finite values for seed={seed}, rho={rho}"
)
if np.any((cont_density_trajectory &lt; 0.0) | (cont_density_trajectory &gt; 1.0)):
raise RuntimeError(
f"Continue density trajectory contains values outside [0,1] for seed={seed}, rho={rho}"
)
# Similarity NaN is permitted for a zero-vector cosine.
# Infinity is never permitted.
if np.any(np.isinf(cont_similarity_trajectory)):
raise RuntimeError(
```

f"Continue similarity trajectory contains infinity "

f"for seed={seed}, rho={rho}"

)

finite_cont_similarity = cont_similarity_trajectory[np.isfinite(cont_similarity_trajectory)]

if len(finite_cont_similarity) > 0 and np.any((finite_cont_similarity < -1.0 - 1e-12) | (finite_cont_similarity > 1.0 + 1e-12)):

raise RuntimeError(

f"Continue similarity trajectory contains values outside [-1,1] for seed={seed}, rho={rho}"

)

Process each horizon system

for horizon_system, similarity_target in [

("fixed_physical", similarity_target_fixed),

("normalized_EMA", similarity_target_normalized),

]:

Process density recovery

density_values = density_trajectory

density_target = density_target_continue

```
# Calculate first crossing for density
density_first_crossing, density_first_observed = first_crossing_time(
sample_times, density_values, density_target
)

# Calculate sustained recovery for density (stays above target for rest of trajectory)
density_sustained, density_sustained_observed = first_time_sustained_for_rest(
sample_times, density_values, density_target
)

# Calculate held recovery for density (stays above target for T_window duration)
density_held, density_held_observed = first_time_held_for_duration(
sample_times, density_values, density_target, t_window
)

# Calculate AUC deficit for density (one-sided: continue - branch)
density_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_density_trajectory, density_values
)
```

```
# Process similarity recovery

similarity_values = similarity_trajectory


# Calculate first crossing for similarity

sim_first_crossing, sim_first_observed = first_crossing_time(

sample_times, similarity_values, similarity_target

)


# Calculate sustained recovery for similarity

sim_sustained, sim_sustained_observed = first_time_sustained_for_rest(

sample_times, similarity_values, similarity_target

)


# Calculate held recovery for similarity

sim_held, sim_held_observed = first_time_held_for_duration(

sample_times, similarity_values, similarity_target, t_window

)
```

```
# Calculate AUC deficit for similarity

sim_auc_deficit = trapezoidal_auc_deficit(

sample_times, cont_similarity_trajectory, similarity_values

)


# P1 #10: Calculate discrete held-duration parameters

# sample_step is the diagnostic sample spacing (0.5 from EXPECTED_SAMPLE_TIMES)

sample_step = float(sample_times[1] - sample_times[0]) if len(sample_times) &gt; 1 else float('nan')

required_samples = max(1, int(np.ceil(t_window / sample_step))) if np.isfinite(sample_step) and
sample_step &gt; 0 else 0


# Calculate restricted times (horizon for censored, event_time for observed)

# Use the last sample time as the horizon (not the elapsed difference)

horizon_time = float(sample_times[-1]) if len(sample_times) &gt; 0 else float('nan')


# P1 #9: Count NaN samples for cosine zero-imputation documentation

# density_nan_count is always zero (density is required to be finite) - removed

similarity_nan_count = int(np.sum(~np.isfinite(similarity_trajectory)))

cont_similarity_nan_count = int(np.sum(~np.isfinite(cont_similarity_trajectory)))
```

```
# For density metrics

density_restricted_first = density_first_crossing if density_first_observed else horizon_time

density_restricted_sustained = density_sustained if density_sustained_observed else horizon_time

density_restricted_held = density_held if density_held_observed else horizon_time


# For similarity metrics

sim_restricted_first = sim_first_crossing if sim_first_observed else horizon_time

sim_restricted_sustained = sim_sustained if sim_sustained_observed else horizon_time

sim_restricted_held = sim_held if sim_held_observed else horizon_time


# Calculate normalized times (t_rec / T_window)

# Raw observed times (conditional on recovery)

density_first_over_Twindow = density_first_crossing / t_window if np.isfinite(density_first_crossing) and t_window &gt; 0 else float('nan')

density_sustained_over_Twindow = density_sustained / t_window if np.isfinite(density_sustained) and t_window &gt; 0 else float('nan')

density_held_over_Twindow = density_held / t_window if np.isfinite(density_held) and t_window &gt; 0 else float('nan')


sim_first_over_Twindow = sim_first_crossing / t_window if np.isfinite(sim_first_crossing) and t_window &gt; 0 else float('nan')
```

sim_sustained_over_Twindow = sim_sustained / t_window if np.isfinite(sim_sustained) and t_window > 0 else float('nan')

sim_held_over_Twindow = sim_held / t_window if np.isfinite(sim_held) and t_window > 0 else float('nan')

Censoring-aware restricted normalized times (cohort-level estimates)

These use restricted times (horizon for censored, event_time for observed) divided by T_window

density_restricted_first_over_Twindow = density_restricted_first / t_window if np.isfinite(density_restricted_first) and t_window > 0 else float('nan')

density_restricted_sustained_over_Twindow = density_restricted_sustained / t_window if np.isfinite(density_restricted_sustained) and t_window > 0 else float('nan')

density_restricted_held_over_Twindow = density_restricted_held / t_window if np.isfinite(density_restricted_held) and t_window > 0 else float('nan')

sim_restricted_first_over_Twindow = sim_restricted_first / t_window if np.isfinite(sim_restricted_first) and t_window > 0 else float('nan')

sim_restricted_sustained_over_Twindow = sim_restricted_sustained / t_window if np.isfinite(sim_restricted_sustained) and t_window > 0 else float('nan')

sim_restricted_held_over_Twindow = sim_restricted_held / t_window if np.isfinite(sim_restricted_held) and t_window > 0 else float('nan')

Add rows for density metrics

Note: For continue branch, AUC deficit = 0 (comparing trajectory with itself)

```
# P1 #11: Density targets are horizon-independent (same 0.9 * D_B_final_continue for both systems)

# P1 #10: Held duration definition is discrete: ceil(T_window / diagnostic_sample_spacing)
consecutive saved samples

# P1 #5: density_horizon_duplication_note moved to density rows

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "density",

"target_value": density_target,

"target_source": "0.9 * D_B_final_continue",

"target_horizon_dependence": "horizon_independent",

"held_duration_definition": "ceil(T_window / diagnostic_sample_spacing) consecutive saved samples",

"held_required_samples": required_samples,

"diagnostic_sample_spacing": sample_step,

"auc_horizon": "full_future",
```

"density_horizon_duplication_note": "Density recovery rows are duplicated across horizon_system by design. They are horizon-independent and must not be pooled across the two labels.",

"first_crossing_time": density_first_crossing,

"first_crossing_observed": density_first_observed,

"sustained_time": density_sustained,

"sustained_observed": density_sustained_observed,

"held_time": density_held,

"held_observed": density_held_observed,

"restricted_first_crossing_time": density_restricted_first,

"restricted_sustained_time": density_restricted_sustained,

"restricted_held_time": density_restricted_held,

"auc_deficit_vs_continue": density_auc_deficit,

"first_crossing_over_Twindow": density_first_over_Twindow,

"sustained_time_over_Twindow": density_sustained_over_Twindow,

"held_time_over_Twindow": density_held_over_Twindow,

"restricted_first_over_Twindow": density_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": density_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": density_restricted_held_over_Twindow,

```
})


# Add rows for similarity metrics
# Similarity targets are horizon-dependent (different for fixed_physical vs normalized_EMA)
# P1 #9: Record NaN imputation counts for cosine zero-vector convention
# P1 #5: density_horizon_duplication_note moved to density rows
recovery_rows.append({
"seed": seed,
"rho": rho,
"T_window": t_window,
"branch": branch,
"horizon_system": horizon_system,
"metric": "similarity",
"target_value": similarity_target,
"target_source": f"0.9 * Weighted_similarity_late_{horizon_system}",
"target_horizon_dependence": "horizon_dependent",
"held_duration_definition": "ceil(T_window / diagnostic_sample_spacing) consecutive saved samples",
"held_required_samples": required_samples,
```

"diagnostic_sample_spacing": sample_step,

"auc_horizon": "full_future",

"target_nan_samples_imputed_zero": similarity_nan_count,

"reference_nan_samples_imputed_zero": cont_similarity_nan_count,

"similarity_nan_convention": "undefined zero-vector cosine imputed as zero for recovery and AUC only",

"first_crossing_time": sim_first_crossing,

"first_crossing_observed": sim_first_observed,

"sustained_time": sim_sustained,

"sustained_observed": sim_sustained_observed,

"held_time": sim_held,

"held_observed": sim_held_observed,

"restricted_first_crossing_time": sim_restricted_first,

"restricted_sustained_time": sim_restricted_sustained,

"restricted_held_time": sim_restricted_held,

"auc_deficit_vs_continue": sim_auc_deficit,

"first_crossing_over_Twindow": sim_first_over_Twindow,

"sustained_time_over_Twindow": sim_sustained_over_Twindow,

"held_time_over_Twindow": sim_held_over_Twindow,

"restricted_first_over_Twindow": sim_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": sim_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": sim_restricted_held_over_Twindow,

})

if not recovery_rows:

raise RuntimeError("No recovery scaling data computed - check NPZ contents and data structure")

Validate expected count: 40 seeds × 5 rho × 4 branches × 2 metrics × 2 horizon systems = 3200

Now includes continue branch for DID calculation

expected_count = len(seeds) *len(rho_values)* len(branches) *2* 2

if len(recovery_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "

f"This may indicate a logic error in recovery calculation."

)

```
recovery_df = pd.DataFrame(recovery_rows)



print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")

print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "

f"Branches: {recovery_df['branch'].nunique()}, Metrics: {recovery_df['metric'].nunique()}, "

f"Systems: {recovery_df['horizon_system'].nunique()}")



return recovery_df

================================================================================
Stage 1: Validation
================================================================================

def validate_run(

text

csv_path: str | Path,

npz_path: str | Path,

manifest_path: str | Path,
```

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

```
"warnings": [],

}



# Convert to Path objects for consistency

csv_path = Path(csv_path)

npz_path = Path(npz_path)

manifest_path = Path(manifest_path)

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results



# P1 #8: Validate manifest study values against prespecified constants

try:
```

```
# Handle both manifest formats:

# Format 1 (new): manifest["study"]["rho_values"]

# Format 2 (frozen production): manifest["model"] has T_window, protocol has branches


# Get rho_values

if "study" in manifest and "rho_values" in manifest["study"]:

manifest_rho_values = tuple(float(r) for r in manifest["study"]["rho_values"])

elif "protocol" in manifest and "rho_values" in manifest["protocol"]:

manifest_rho_values = tuple(float(r) for r in manifest["protocol"]["rho_values"])

else:

# Cannot validate rho_values - skip this check

manifest_rho_values = None


# Get branches

if "protocol" in manifest and "branches" in manifest["protocol"]:

manifest_branches = tuple(manifest["protocol"]["branches"])

else:

manifest_branches = None
```

```
# Get seed count

if "randomization" in manifest and "paired_seeds" in manifest["randomization"]:

manifest_n_seeds = len(manifest["randomization"]["paired_seeds"])

else:

manifest_n_seeds = None


# Only validate if we could extract the values

if manifest_rho_values is not None and manifest_branches is not None and manifest_n_seeds is not None:

if manifest_rho_values != EXPECTED_RHO_VALUES:

results["status"] = "failed"

results["errors"].append(

f"Manifest rho_values {manifest_rho_values} != expected {EXPECTED_RHO_VALUES}"

)

return results


if manifest_branches != EXPECTED_BRANCHES:

results["status"] = "failed"
```

```
results["errors"].append(
f"Manifest branches {manifest_branches} != expected {EXPECTED_BRANCHES}"
)
return results

if manifest_n_seeds != EXPECTED_N_SEEDS:
results["status"] = "failed"
results["errors"].append(
f"Manifest has {manifest_n_seeds} seeds, expected {EXPECTED_N_SEEDS}"
)
return results
except Exception as e:
results["status"] = "failed"
results["errors"].append(f"Could not validate manifest study values: {e}")
return results


# Validate condition status - MANDATORY for production
condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"
```

```
if not condition_status_path.exists():

results["status"] = "failed"

results["errors"].append("Condition status file not found - REQUIRED for production analysis")

return results



try:

condition_df = [pd.read](http://pd.read)_csv(condition_status_path)


# Check for duplicate condition-status rows

if condition_df.duplicated(subset=["seed", "rho"]).any():

results["status"] = "failed"

results["errors"].append("Condition status file has duplicate (seed, rho) rows")

return results


# Validate status values

unique_statuses = set(condition_df["status"].unique())

valid_statuses = {"success", "failed"}

invalid_statuses = unique_statuses - valid_statuses
```

```
if invalid_statuses:
results["status"] = "failed"
results["errors"].append(
f"Condition status file has invalid status values: {invalid_statuses}"
)
return results

successful_conditions = (condition_df["status"] == "success").sum()
failed_conditions = (condition_df["status"] == "failed").sum()
total_conditions = len(condition_df)

expected_conditions = EXPECTED_N_CONDITIONS
if total_conditions != expected_conditions:
results["status"] = "failed"
results["errors"].append(
f"Condition status: expected {expected_conditions} conditions, "
f"got {total_conditions}"
)
```

return results

if failed_conditions > 0:

results["status"] = "failed"

results["errors"].append(

f"Condition status: {failed_conditions} conditions failed, "

f"{successful_conditions} succeeded"

)

return results

Validate exact condition coverage matches manifest

condition_df["seed"] = condition_df["seed"].astype(str)

condition_df["rho"] = pd.to_numeric(condition_df["rho"], errors="coerce")

observed_conditions = set(

zip(condition_df["seed"], condition_df["rho"])

)

Get expected conditions from manifest

```
try:
manifest_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
# Handle both manifest formats for rho_values
if "study" in manifest and "rho_values" in manifest["study"]:
manifest_rho_values = [float(r) for r in manifest["study"]["rho_values"]]
elif "protocol" in manifest and "rho_values" in manifest["protocol"]:
manifest_rho_values = [float(r) for r in manifest["protocol"]["rho_values"]]
else:
manifest_rho_values = []

expected_conditions_from_manifest = {
(str(seed), float(rho))
for seed in manifest_seeds
for rho in manifest_rho_values
}

if observed_conditions != expected_conditions_from_manifest:
missing = expected_conditions_from_manifest - observed_conditions
```

```
extra = observed_conditions - expected_conditions_from_manifest

if missing:

results["status"] = "failed"

results["errors"].append(

f"Condition status missing {len(missing)} conditions: {list(missing)[:5]}"

)

return results

if extra:

results["status"] = "failed"

results["errors"].append(

f"Condition status has {len(extra)} extra conditions: {list(extra)[:5]}"

)

return results

except Exception as e:

results["status"] = "failed"

results["errors"].append(

f"Could not validate exact condition cohort against manifest: {e}"
```

```
)

return results


# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash


except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate condition status: {e}")

return results



# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"
```

```
)

return results



# Check freezeable status - FAIL if not freezeable

# Check both top-level and completion.freezeable, default to False

completion = manifest.get("completion", {})

freezeable = manifest.get("freezeable", completion.get("freezeable", False))



# Also check completion status

completion_status = completion.get("status", "")

n_successful = completion.get("n_conditions_successful", 0)

n_failed = completion.get("n_conditions_failed", 0)

n_expected = EXPECTED_N_CONDITIONS



# Must have all conditions successful and explicit freezeable=true

if not freezeable:

results["status"] = "failed"
```

```
results["errors"].append("Run is marked as non-freezeable")

return results



if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")

return results



if n_successful != n_expected or n_failed != 0:

results["status"] = "failed"

results["errors"].append(

f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results



# Load CSV

try:
```

```
df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)

if actual_rows != expected_rows:

results["status"] = "failed"

results["errors"].append(

f"Row count mismatch: expected {expected_rows}, got {actual_rows}"

)



# Validate unique seeds

unique_seeds = df["seed"].unique()
```

```
if len(unique_seeds) != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"

)



# Validate rho values

unique_rho = sorted([float(r) for r in df["rho"].unique()])

expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])

if unique_rho != expected_rho:

results["status"] = "failed"

results["errors"].append(

f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"

)



# Validate branches

unique_branches = sorted(df["branch"].unique())
```

```
expected_branches = sorted(EXPECTED_BRANCHES)

if unique_branches != expected_branches:

results["status"] = "failed"

results["errors"].append(

f"Branches mismatch: expected {expected_branches}, got {unique_branches}"

)



# Validate completeness: each seed/rho/branch combination should exist

missing_combinations = []

for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):

missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")
```

```
if missing_combinations:

results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate exact cohort coverage in CSV matches manifest

try:

manifest_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
```

```
# Handle both manifest formats for rho_values
if "study" in manifest and "rho_values" in manifest["study"]:
manifest_rho_values = [float(r) for r in manifest["study"]["rho_values"]]
elif "protocol" in manifest and "rho_values" in manifest["protocol"]:
manifest_rho_values = [float(r) for r in manifest["protocol"]["rho_values"]]
else:
manifest_rho_values = []
manifest_branches = manifest["protocol"]["branches"]

expected_csv_conditions = {
(str(seed), float(rho), branch)
for seed in manifest_seeds
for rho in manifest_rho_values
for branch in manifest_branches
}

observed_csv_conditions = set(
zip(df["seed"], df["rho"], df["branch"])
```

```
)

if observed_csv_conditions != expected_csv_conditions:
missing = expected_csv_conditions - observed_csv_conditions
extra = observed_csv_conditions - expected_csv_conditions
if missing:
results["status"] = "failed"
results["errors"].append(
f"CSV missing {len(missing)} conditions: {list(missing)[:5]}"
)
return results
if extra:
results["status"] = "failed"
results["errors"].append(
f"CSV has {len(extra)} extra conditions: {list(extra)[:5]}"
)
return results
except Exception as e:
```

```
results["status"] = "failed"

results["errors"].append(

f"Could not validate exact CSV cohort against manifest: {e}"

)

return results



# Validate primary and secondary outcomes are present and finite

all_outcomes = PRIMARY_OUTCOMES + SECONDARY_NORMALIZED_OUTCOMES

for outcome in all_outcomes:

if outcome not in df.columns:

results["status"] = "failed"

results["errors"].append(f"Outcome '{outcome}' missing from CSV")

else:

# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():

results["status"] = "failed"
```

```
results["errors"].append(
f"Outcome '{outcome}' has {non_numeric.sum()} non-numeric values"
)

# Check that values are finite (not NaN, inf, or -inf)
non_finite = ~np.isfinite(df[outcome])
if non_finite.any():
results["status"] = "failed"
results["errors"].append(
f"Outcome '{outcome}' has {non_finite.sum()} non-finite values"
)


# P0 #2: Validate T_window exists and is consistent
# T_window is study metadata, critical for recovery normalization
if "T_window" not in df.columns:
results["status"] = "failed"
results["errors"].append("T_window missing from scalar CSV")
return results
```

else:

tw = pd.to_numeric(df["T_window"], errors="coerce")

if (~np.isfinite(tw) | (tw <= 0)).any():

results["status"] = "failed"

results["errors"].append("T_window contains non-finite or non-positive values")

return results

Validate T_window is consistent across branches for each (seed, rho)

Handle both manifest formats for rho_values and T_window_values

try:

if "study" in manifest and "rho_values" in manifest["study"] and "T_window_values" in manifest["study"]:

rho_values_list = [float(r) for r in manifest["study"]["rho_values"]]

twindow_values_list = [float(x) for x in manifest["study"]["T_window_values"]]

elif "protocol" in manifest and "rho_values" in manifest["protocol"]:

Alternative format: get rho_values from protocol

rho_values_list = [float(r) for r in manifest["protocol"]["rho_values"]]

T_window might be a single value in model

```
if "model" in manifest and "T_window" in manifest["model"]:

# If T_window is a single value, use it for all rho

twindow_values_list = [float(manifest["model"]["T_window"])] * len(rho_values_list)

elif "study" in manifest and "T_window_values" in manifest["study"]:

twindow_values_list = [float(x) for x in manifest["study"]["T_window_values"]]

else:

# Cannot validate - skip

rho_values_list = []

twindow_values_list = []

else:

# Cannot validate - skip

rho_values_list = []

twindow_values_list = []


if rho_values_list and twindow_values_list and len(rho_values_list) == len(twindow_values_list):

for rho, expected_twindow in zip(rho_values_list, twindow_values_list):

rho_mask = df["rho"] == rho

values = df.loc[rho_mask, "T_window"].unique()
```

```
if len(values) != 1:

results["status"] = "failed"

results["errors"].append(

f"rho={rho} has {len(values)} distinct T_window values"

)

return results

elif float(values[0]) != expected_twindow:

results["status"] = "failed"

results["errors"].append(

f"rho={rho}: T_window={values[0]} != expected {expected_twindow}"

)

return results

except Exception as e:

# If we can't validate T_window consistency, log a warning but don't fail

results["warnings"].append(f"Could not validate T_window consistency: {e}")



# P0 #3: Validate D_B_final exists and is valid
```

D_B_final is used for density targets in recovery analysis

if "D_B_final" not in df.columns:

results["status"] = "failed"

results["errors"].append("D_B_final missing from scalar CSV")

return results

else:

density = pd.to_numeric(df["D_B_final"], errors="coerce")

invalid = (~np.isfinite(density) | (density < 0.0) | (density > 1.0))

if invalid.any():

results["status"] = "failed"

results["errors"].append(

f"D_B_final has {int(invalid.sum())} invalid values (must be in [0,1])"

)

return results

Get baseline equivalence certificate hash - MANDATORY for production

baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"

```
if not baseline_cert_path.exists():

results["status"] = "failed"

results["errors"].append("Baseline equivalence certificate not found - REQUIRED for production analysis")

return results

baseline_cert_hash = file_sha256(baseline_cert_path)



# Validate baseline certificate contents

try:

with open(baseline_cert_path, "r") as f:

certificate = json.load(f)

cert_status = certificate.get("status", "unknown")

cert_n_seeds = certificate.get("n_seeds", 0)

cert_seed_results = certificate.get("seed_results", {})


if cert_status != "passed":

results["status"] = "failed"

results["errors"].append(
```

```
f"Baseline certificate status is '{cert_status}', expected 'passed'"

)

return results


if cert_n_seeds != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate has {cert_n_seeds} seeds, expected {EXPECTED_N_SEEDS}"

)

return results


if len(cert_seed_results) != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate has {len(cert_seed_results)} seed results, expected {EXPECTED_N_SEEDS}"

)

return results


# P1 #7: Verify baseline certificate cohort matches manifest exactly
```

```
try:
expected_cert_seeds = {
str(seed)
for seed in manifest["randomization"]["paired_seeds"]
}
observed_cert_seeds = set(cert_seed_results.keys())

if observed_cert_seeds != expected_cert_seeds:
missing_seeds = expected_cert_seeds - observed_cert_seeds
extra_seeds = observed_cert_seeds - expected_cert_seeds
if missing_seeds:
results["status"] = "failed"
results["errors"].append(
f"Baseline certificate missing {len(missing_seeds)} seeds: {list(missing_seeds)[:5]}"
)
return results
if extra_seeds:
```

```
results["status"] = "failed"
results["errors"].append(
f"Baseline certificate has {len(extra_seeds)} extra seeds: {list(extra_seeds)[:5]}"
)
return results
except Exception as e:
results["status"] = "failed"
results["errors"].append(
f"Could not validate baseline certificate seed cohort: {e}"
)
return results

# P1 #4: Validate baseline certificate branch set
try:
cert_branches = tuple(
certificate.get("protocol", {}).get("branches", [])
)
if cert_branches != EXPECTED_FROZEN_BASELINE_BRANCHES:
```

```
results["status"] = "failed"
results["errors"].append(
f"Baseline certificate branch set/order {cert_branches} != expected "
f"{EXPECTED_FROZEN_BASELINE_BRANCHES}"
)
return results
except Exception as e:
results["status"] = "failed"
results["errors"].append(
f"Could not validate baseline certificate branch set: {e}"
)
return results


for seed_id, seed_result in cert_seed_results.items():
if seed_result.get("status") != "passed":
results["status"] = "failed"
results["errors"].append(
f"Baseline certificate seed {seed_id} status is '{seed_result.get('status')}', expected 'passed'"
```

```
)

return results

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate baseline certificate contents: {e}")

return results



# P0 #4: Verify simulation output hashes against manifest

# Validate that actual input files match the recorded hashes in the simulation manifest

recorded_hashes = manifest.get("output_hashes", {})



files_to_verify = {

"scalar_csv": csv_path,

"branch_npz": npz_path,

"condition_status_csv": condition_status_path,

"baseline_certificate": baseline_cert_path,

}
```

```
for label, path in files_to_verify.items():

expected_hash = recorded_hashes.get(label)

observed_hash = file_sha256(path)


if expected_hash is None:

results["status"] = "failed"

results["errors"].append(

f"Simulation manifest missing output hash for {label}"

)

return results

elif observed_hash != expected_hash:

results["status"] = "failed"

results["errors"].append(

f"Hash mismatch for {label}: expected={expected_hash}, observed={observed_hash}"

)

return results
```

```
# Record validation info
results["validation"] = {
"n_rows": actual_rows,
"n_seeds": len(unique_seeds),
"n_rho": len(unique_rho),
"n_branches": len(unique_branches),
"n_duplicates": int(duplicate_rows.sum()),
"n_missing_combinations": len(missing_combinations),
"baseline_equivalence": baseline_eq.get("status"),
"freezeable": manifest.get("freezeable", False),
"baseline_certificate_hash": baseline_cert_hash,
}

# Also expose at top level for manifest generation
results["baseline_certificate_hash"] = baseline_cert_hash

if results["status"] == "passed":
```

```
print("✅ Stage 1: Validation PASSED")

print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")



return results

================================================================================
Stage 2: Regime Contrasts
================================================================================

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,
```

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Uses direct merge instead of pivot to avoid MultiIndex column issues.

For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

Filter to continue branch only

cont = df.loc[

df["branch"] == "continue",

["seed", "rho", *outcomes],

].copy()

Get baseline (rho=1.0) values

baseline = (

```
cont.loc[cont["rho"] == 1.0]

.drop(columns="rho")

.rename(

columns={

outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values

merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)
```

```
# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]



# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")



rows.append({

"seed": row["seed"],

"rho": row["rho"],
```

```
"outcome": outcome,

"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)

if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)
```

```
print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")




return contrasts_df

================================================================================
Stage 3: Branch Difference-in-Differences
================================================================================

def compute_branch_did(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]
```

```
For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.



Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []



for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]



for rho in sorted([float(r) for r in df["rho"].unique()]):

# Skip baseline rho=1.0

if rho == 1.0:

continue



rho_data = seed_data[seed_data["rho"] == rho]



# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]
```

```
for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

# Get Y_{s,ρ,b}

y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}

y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]

if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:
```

```
raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")

y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,
```

"Y_rho_branch": y_rho_b,

"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})

Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)

did_df = pd.DataFrame(did_rows)

print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches: {did_df['branch'].nunique()}")

return did_df

==========
Stage 4: Cross-T Topology and Recovery
==========

def build_continue_b_final_key(seed: str, rho: float) -> str:

text

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: str | Path,

df: pd.DataFrame,

```
seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.



For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})



Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

# Convert to Path for consistency

npz_path = Path(npz_path)
```

```
# Build list of exact keys to load

keys_to_load = []

for seed in seeds:

for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))



# Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)



topology_rows = []

missing_keys = []



for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline
```

```
# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)


if rho_key not in npz_data:

missing_keys.append(rho_key)

continue

if baseline_key not in npz_data:

missing_keys.append(baseline_key)

continue


B_rho = validate_topology_matrix(npz_data[rho_key], rho_key)

B_baseline = validate_topology_matrix(npz_data[baseline_key], baseline_key)


# Compute metrics

cos_sim = cosine_similarity(B_rho, B_baseline)

jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)

norm_dist = normalized_distance(B_rho, B_baseline)
```

```
# Validate metric outputs are finite
if not np.isfinite(cos_sim):
raise RuntimeError(f"cosine_similarity({rho_key}, {baseline_key}) returned non-finite value")
if not np.isfinite(jaccard):
raise RuntimeError(f"jaccard_similarity({rho_key}, {baseline_key}) returned non-finite value")
if not np.isfinite(norm_dist):
raise RuntimeError(f"normalized_distance({rho_key}, {baseline_key}) returned non-finite value")

topology_rows.append({
"seed": seed,
"rho": rho,
"cross_T_cosine": cos_sim,
"cross_T_Jaccard": jaccard,
"cross_T_normalized_distance": norm_dist,
})

# Validate we have all expected comparisons
```

```
expected_count = EXPECTED_N_TOPOLOGY

if len(topology_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "

f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



# P0 #2: Validate topology metrics are all finite

topology_columns = ("cross_T_cosine", "cross_T_Jaccard", "cross_T_normalized_distance")

for column in topology_columns:

if not np.isfinite(topology_df[column]).all():

raise RuntimeError(

f"Topology metric {column} contains non-finite values"

)
```

```
# Validate topology group sizes

topology_sizes = topology_df.groupby("rho").size()

if len(topology_sizes) != 4:

raise RuntimeError(

f"Expected 4 topology rho groups, got {len(topology_sizes)}"

)

if not (topology_sizes == EXPECTED_N_SEEDS).all():

raise RuntimeError(

f"Topology rho groups do not all contain {EXPECTED_N_SEEDS} seeds"

)



print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")



return topology_df

def compute_recovery_did(

text
```

recovery: pd.DataFrame,

) -> pd.DataFrame:

text

"""Compute recovery interaction DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

For each seed, non-baseline rho, and NON-CONTINUE branch only, compute the

difference-in-differences for recovery outcomes relative to continue branch.

Uses restricted times (censoring-aware) and normalized t/T_window as primary outcomes.

Skips rho=1.0 (baseline) as DID should be zero by construction.

Skips continue branch as it generates trivial zero-DID rows.

Expected count: 40 seeds *4 non-baseline rho* 3 non-continue branches *2 systems* 2 metrics * 13 outcomes = 24960

"""

if recovery.empty:

return pd.DataFrame()

```
did_rows = []


seeds = sorted(recovery["seed"].unique())
rho_values = sorted([float(r) for r in recovery["rho"].unique()])
horizon_systems = sorted(recovery["horizon_system"].unique())
metrics = sorted(recovery["metric"].unique())


# Recovery outcomes to compute DID for
# Prioritize censoring-aware restricted outcomes as primary
recovery_outcomes = [
"restricted_first_crossing_time",
"restricted_sustained_time",
"restricted_held_time",
"restricted_first_over_Twindow",
"restricted_sustained_over_Twindow",
"restricted_held_over_Twindow",
```

```
"auc_deficit_vs_continue",

"first_crossing_over_Twindow",

"sustained_time_over_Twindow",

"held_time_over_Twindow",

]



# Also compute DID for recovery indicators

indicator_outcomes = [

"first_crossing_observed",

"sustained_observed",

"held_observed",

]



for seed in seeds:

seed_data = recovery[recovery["seed"] == seed]



for rho in rho_values:
```

```
# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in NON_CONTINUE_BRANCHES:

# EXCLUDE continue branch - it generates trivial zero-DID rows

# DID is only defined for non-continue branches: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

for horizon_system in horizon_systems:

for metric in metrics:

# Compute DID for each recovery outcome

for outcome in recovery_outcomes:

# Get values: R_{s,ρ,b}, R_{s,ρ,c}, R_{s,1,b}, R_{s,1,c}


# R_{s,ρ,b}
```

```
r_rho_b_match = rho_data[
(rho_data["branch"] == branch) &amp;amp;
(rho_data["horizon_system"] == horizon_system) &amp;amp;
(rho_data["metric"] == metric)
][outcome]

# R_{s,ρ,c} (continue branch at same rho)
r_rho_c_match = rho_data[
(rho_data["branch"] == "continue") &amp;amp;
(rho_data["horizon_system"] == horizon_system) &amp;amp;
(rho_data["metric"] == metric)
][outcome]

# R_{s,1,b} (baseline rho=1.0, same branch)
r_base_b_match = baseline_data[
(baseline_data["branch"] == branch) &amp;amp;
(baseline_data["horizon_system"] == horizon_system) &amp;amp;
(baseline_data["metric"] == metric)
```

```
][outcome]


# R_{s,1,c} (baseline rho=1.0, continue)

r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# Validate exactly one match for each cell - missing cells indicate schema error

matches = {

"rho_branch": r_rho_b_match,

"rho_continue": r_rho_c_match,

"baseline_branch": r_base_b_match,

"baseline_continue": r_base_c_match,

}

for label, match in matches.items():

if len(match) != 1:
```

```
raise RuntimeError(
f"Recovery DID requires exactly one {label} row; "
f"seed={seed}, rho={rho}, branch={branch}, "
f"horizon={horizon_system}, metric={metric}, "
f"outcome={outcome}, found={len(match)}"
)

r_rho_b = float(r_rho_b_match.iloc[0])
r_rho_c = float(r_rho_c_match.iloc[0])
r_base_b = float(r_base_b_match.iloc[0])
r_base_c = float(r_base_c_match.iloc[0])

# Compute DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]
branch_effect_rho = r_rho_b - r_rho_c
branch_effect_baseline = r_base_b - r_base_c
did = branch_effect_rho - branch_effect_baseline

did_rows.append({
"seed": seed,
```

"rho": rho,

"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})

Compute DID for recovery indicators (binary outcomes)

for outcome in indicator_outcomes:

Get values

```
r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


r_base_c_match = baseline_data[
```

```
(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# Validate exactly one match for each cell - missing cells indicate schema error

matches = {

"rho_branch": r_rho_b_match,

"rho_continue": r_rho_c_match,

"baseline_branch": r_base_b_match,

"baseline_continue": r_base_c_match,

}

for label, match in matches.items():

if len(match) != 1:

raise RuntimeError(

f"Recovery DID requires exactly one {label} row; "

f"seed={seed}, rho={rho}, branch={branch}, "
```

```
f"horizon={horizon_system}, metric={metric}, "
f"outcome={outcome}, found={len(match)}"
)

r_rho_b = float(r_rho_b_match.iloc[0])
r_rho_c = float(r_rho_c_match.iloc[0])
r_base_b = float(r_base_b_match.iloc[0])
r_base_c = float(r_base_c_match.iloc[0])

# For binary outcomes, use difference in proportions
branch_effect_rho = r_rho_b - r_rho_c
branch_effect_baseline = r_base_b - r_base_c
did = branch_effect_rho - branch_effect_baseline

did_rows.append({
"seed": seed,
"rho": rho,
"branch": branch,
"horizon_system": horizon_system,
```

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})

if not did_rows:

return pd.DataFrame()

Validate exact count: 40 seeds *4 non-baseline rho* 3 non-continue branches *2 systems* 2 metrics * 13 outcomes = 24960

if len(did_rows) != EXPECTED_N_RECOVERY_DID:

```
raise RuntimeError(
f"Expected exactly {EXPECTED_N_RECOVERY_DID} recovery DID rows, got {len(did_rows)}. "
f"This indicates a logic error - continue branch should be excluded."
)


did_df = pd.DataFrame(did_rows)


print(f"✅ Stage 4c: Recovery DID - {len(did_rows)} recovery DID values computed")
print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, "
f"Branches: {did_df['branch'].nunique()}, Systems: {did_df['horizon_system'].nunique()}")


return did_df
def compute_seed_level_summaries(
text
regime_contrasts: pd.DataFrame,
branch_did: pd.DataFrame,
topology: pd.DataFrame,
```

```
recovery: pd.DataFrame | None = None,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.



For each outcome and rho, compute:

- Mean contrast

- Sample standard deviation

- 95% bootstrap CI



Uses seeded bootstrap for reproducibility.

"""

summaries = {}



# Create seeded RNG for bootstrap
```

```
if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)



# Regime contrast summaries

if not regime_contrasts.empty:

# Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

n=("paired_regime_contrast", "count"),

)



# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):
```

values = group["paired_regime_contrast"].dropna().values

if len(values) > 0:

Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))

idx = regime_summary[(regime_summary["rho"] == rho) & (regime_summary["outcome"] == outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper

summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")

Branch DID summaries

if not branch_did.empty:

```
did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),

std=("difference_in_differences", "std"),

n=("difference_in_differences", "count"),

)


# Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))
```

idx = did_summary[(did_summary["rho"] == rho) & (did_summary["branch"] == branch) & (did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper

summaries["branch_did"] = did_summary

print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")

Topology summaries

if not topology.empty:

Flatten the aggregation to avoid MultiIndex columns

topology_summary = topology.groupby("rho", as_index=False).agg(

cosine_mean=("cross_T_cosine", "mean"),

cosine_std=("cross_T_cosine", "std"),

cosine_n=("cross_T_cosine", "count"),

jaccard_mean=("cross_T_Jaccard", "mean"),

jaccard_std=("cross_T_Jaccard", "std"),

```
jaccard_n=("cross_T_Jaccard", "count"),

distance_mean=("cross_T_normalized_distance", "mean"),

distance_std=("cross_T_normalized_distance", "std"),

distance_n=("cross_T_normalized_distance", "count"),

)


# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)

for rho, group in topology.groupby("rho"):

# Bootstrap cosine similarity

cosine_values = group["cross_T_cosine"].dropna().values

if len(cosine_values) &gt; 0:

indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))

bootstrap_means = np.mean(cosine_values[indices], axis=1)

cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))

cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower
```

```
topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))

bootstrap_means = np.mean(jaccard_values[indices], axis=1)

jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower

topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper


# Bootstrap normalized distance

distance_values = group["cross_T_normalized_distance"].dropna().values

if len(distance_values) &gt; 0:

indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))

bootstrap_means = np.mean(distance_values[indices], axis=1)
```

```
distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))

distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower

topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper


summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")



# Recovery scaling summaries with bootstrap CI

if recovery is not None and not recovery.empty:

# Group by metric, horizon_system, branch, rho and compute summaries

# Include both raw event times AND censoring-aware restricted times

recovery_summary = recovery.groupby(

["rho", "branch", "horizon_system", "metric"],

as_index=False,

).agg(
```

```
# Raw event times (conditional on observed recovery)
mean_first_crossing=("first_crossing_time", "mean"),
std_first_crossing=("first_crossing_time", "std"),
n_first_crossing=("first_crossing_time", "count"),
mean_sustained=("sustained_time", "mean"),
std_sustained=("sustained_time", "std"),
n_sustained=("sustained_time", "count"),
mean_held=("held_time", "mean"),
std_held=("held_time", "std"),
n_held=("held_time", "count"),
mean_auc_deficit=("auc_deficit_vs_continue", "mean"),
std_auc_deficit=("auc_deficit_vs_continue", "std"),
n_auc_deficit=("auc_deficit_vs_continue", "count"),

# Censoring-aware restricted times (cohort-level estimates)
mean_restricted_first=("restricted_first_crossing_time", "mean"),
std_restricted_first=("restricted_first_crossing_time", "std"),
```

```
n_restricted_first=("restricted_first_crossing_time", "count"),

mean_restricted_sustained=("restricted_sustained_time", "mean"),

std_restricted_sustained=("restricted_sustained_time", "std"),

n_restricted_sustained=("restricted_sustained_time", "count"),

mean_restricted_held=("restricted_held_time", "mean"),

std_restricted_held=("restricted_held_time", "std"),

n_restricted_held=("restricted_held_time", "count"),


# Normalized times (t/T_window) - raw observed (conditional on recovery)

mean_first_over_Twindow=("first_crossing_over_Twindow", "mean"),

std_first_over_Twindow=("first_crossing_over_Twindow", "std"),

n_first_over_Twindow=("first_crossing_over_Twindow", "count"),

mean_sustained_over_Twindow=("sustained_time_over_Twindow", "mean"),

std_sustained_over_Twindow=("sustained_time_over_Twindow", "std"),

n_sustained_over_Twindow=("sustained_time_over_Twindow", "count"),

mean_held_over_Twindow=("held_time_over_Twindow", "mean"),

std_held_over_Twindow=("held_time_over_Twindow", "std"),

n_held_over_Twindow=("held_time_over_Twindow", "count"),
```

```
# Censoring-aware restricted normalized times (cohort-level estimates)
mean_restricted_first_over_Twindow=("restricted_first_over_Twindow", "mean"),
std_restricted_first_over_Twindow=("restricted_first_over_Twindow", "std"),
n_restricted_first_over_Twindow=("restricted_first_over_Twindow", "count"),
mean_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "mean"),
std_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "std"),
n_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "count"),
mean_restricted_held_over_Twindow=("restricted_held_over_Twindow", "mean"),
std_restricted_held_over_Twindow=("restricted_held_over_Twindow", "std"),
n_restricted_held_over_Twindow=("restricted_held_over_Twindow", "count"),

# Recovery probabilities
observed_rate_first=("first_crossing_observed", "mean"),
observed_rate_sustained=("sustained_observed", "mean"),
observed_rate_held=("held_observed", "mean"),
)

# Add bootstrap CI for recovery metrics
```

```
for (rho, branch, horizon_system, metric), group in recovery.groupby(["rho", "branch",
"horizon_system", "metric"]):

# Bootstrap first crossing time (conditional on observed)

first_crossing_values = group["first_crossing_time"].dropna().values

if len(first_crossing_values) &gt; 0:

indices = rng.integers(0, len(first_crossing_values), size=(n_bootstrap, len(first_crossing_values)))

bootstrap_means = np.nanmean(first_crossing_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_crossing"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_crossing"] = ci_upper
```

```
# Bootstrap restricted first crossing time (cohort-level, includes censored)

restricted_first_values = group["restricted_first_crossing_time"].dropna().values

if len(restricted_first_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_values), size=(n_bootstrap, len(restricted_first_values)))

bootstrap_means = np.nanmean(restricted_first_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first"] = ci_upper


# Bootstrap restricted sustained time
```

```
restricted_sustained_values = group["restricted_sustained_time"].dropna().values

if len(restricted_sustained_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_values), size=(n_bootstrap,
len(restricted_sustained_values)))

bootstrap_means = np.nanmean(restricted_sustained_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained"] = ci_upper


# Bootstrap restricted held time
```

```
restricted_held_values = group["restricted_held_time"].dropna().values

if len(restricted_held_values) &gt; 0:

indices = rng.integers(0, len(restricted_held_values), size=(n_bootstrap, len(restricted_held_values)))

bootstrap_means = np.nanmean(restricted_held_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held"] = ci_upper


# Bootstrap normalized first crossing time over T_window

first_over_Twindow_values = group["first_crossing_over_Twindow"].dropna().values
```

```
if len(first_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(first_over_Twindow_values), size=(n_bootstrap,
len(first_over_Twindow_values)))

bootstrap_means = np.nanmean(first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_over_Twindow"] = ci_upper


# Bootstrap normalized sustained time over T_window

sustained_over_Twindow_values = group["sustained_time_over_Twindow"].dropna().values

if len(sustained_over_Twindow_values) &gt; 0:
```

```
indices = rng.integers(0, len(sustained_over_Twindow_values), size=(n_bootstrap,
len(sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_sustained_over_Twindow"] = ci_upper


# Bootstrap normalized held time over T_window

held_over_Twindow_values = group["held_time_over_Twindow"].dropna().values

if len(held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(held_over_Twindow_values), size=(n_bootstrap,
len(held_over_Twindow_values)))
```

```
bootstrap_means = np.nanmean(held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_held_over_Twindow"] = ci_upper


# Bootstrap AUC deficit

auc_values = group["auc_deficit_vs_continue"].dropna().values

if len(auc_values) &amp;gt; 0:

indices = rng.integers(0, len(auc_values), size=(n_bootstrap, len(auc_values)))

bootstrap_means = np.nanmean(auc_values[indices], axis=1)
```

```
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_auc_deficit"] = ci_lower

recovery_summary.loc[idx, "ci_upper_auc_deficit"] = ci_upper


# Bootstrap restricted normalized first crossing time over T_window

restricted_first_over_Twindow_values = group["restricted_first_over_Twindow"].dropna().values

if len(restricted_first_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_over_Twindow_values), size=(n_bootstrap,
len(restricted_first_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_first_over_Twindow_values[indices], axis=1)
```

```
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first_over_Twindow"] = ci_upper


# Bootstrap restricted normalized sustained time over T_window

restricted_sustained_over_Twindow_values =
group["restricted_sustained_over_Twindow"].dropna().values

if len(restricted_sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_over_Twindow_values), size=(n_bootstrap,
len(restricted_sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
```

```
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained_over_Twindow"] = ci_upper


# Bootstrap restricted normalized held time over T_window

restricted_held_over_Twindow_values = group["restricted_held_over_Twindow"].dropna().values

if len(restricted_held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_held_over_Twindow_values), size=(n_bootstrap,
len(restricted_held_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
```

```
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held_over_Twindow"] = ci_upper


# Bootstrap recovery probabilities (observed rates)

# First crossing observed rate

first_observed_values = group["first_crossing_observed"].dropna().values

if len(first_observed_values) &gt; 0:

indices = rng.integers(0, len(first_observed_values), size=(n_bootstrap, len(first_observed_values)))

bootstrap_means = np.nanmean(first_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
```

```
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_first"] = ci_upper


# Sustained observed rate

sustained_observed_values = group["sustained_observed"].dropna().values

if len(sustained_observed_values) &amp;gt; 0:

indices = rng.integers(0, len(sustained_observed_values), size=(n_bootstrap,
len(sustained_observed_values)))

bootstrap_means = np.nanmean(sustained_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))
```

```
idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_sustained"] = ci_upper


# Held observed rate

held_observed_values = group["held_observed"].dropna().values

if len(held_observed_values) &amp;gt; 0:

indices = rng.integers(0, len(held_observed_values), size=(n_bootstrap, len(held_observed_values)))

bootstrap_means = np.nanmean(held_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[
```

(recovery_summary["rho"] == rho) &

(recovery_summary["branch"] == branch) &

(recovery_summary["horizon_system"] == horizon_system) &

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_held"] = ci_upper

summaries["recovery"] = recovery_summary

print(f"✅ Stage 5: Recovery Summaries - {len(recovery_summary)} groups")

return summaries

==========
Report Generation
==========

def generate_text_report(

text

validation: dict[str, Any],

```
regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,

recovery_summaries: pd.DataFrame | None = None,

regime_summaries_secondary: pd.DataFrame | None = None,

did_summaries_secondary: pd.DataFrame | None = None,

recovery_did_summary: pd.DataFrame | None = None,

) -> str:

text

"""Generate human-readable text report."""

lines = []

lines.append("=" * 80)

lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")

lines.append("=" * 80)
```

```
lines.append("")



# Validation summary

lines.append("VALIDATION SUMMARY")

lines.append("-" * 40)

val = validation.get("validation", {})

lines.append(f" Status: {validation.get('status', 'unknown')}")

lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")

lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")

lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")
```

```
# Regime contrasts (Primary - fixed_physical horizons)

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not regime_summaries.empty:

for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"
```

```
lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Secondary regime contrasts (normalized_EMA horizons)

lines.append("SECONDARY REGIME CONTRASTS (Normalized EMA horizons)")

lines.append("-" * 40)

if regime_summaries_secondary is not None and not regime_summaries_secondary.empty:

for rho in sorted(regime_summaries_secondary["rho"].unique()):

rho_data = regime_summaries_secondary[regime_summaries_secondary["rho"] == rho]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])
```

```
ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No secondary regime contrast data")

lines.append("")



# Branch DID (Primary - fixed_physical horizons)

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]
```

```
for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")
```

```
# Secondary Branch DID (normalized_EMA horizons)

lines.append("SECONDARY BRANCH DIFFERENCE-IN-DIFFERENCES (Normalized EMA)")

lines.append("-" * 40)

if did_summaries_secondary is not None and not did_summaries_secondary.empty:

for rho in sorted(did_summaries_secondary["rho"].unique()):

rho_data = did_summaries_secondary[did_summaries_secondary["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])
```

```
ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No secondary branch DID data")

lines.append("")



# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])

lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f} (n={int(row['cosine_n'])})")

if "cosine_ci_lower" in row and not np.isnan(float(row.get("cosine_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['cosine_ci_lower']):.4f}, {float(row['cosine_ci_upper']):.4f}]")
```

```
lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

if "jaccard_ci_lower" in row and not np.isnan(float(row.get("jaccard_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['jaccard_ci_lower']):.4f}, {float(row['jaccard_ci_upper']):.4f}]")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

if "distance_ci_lower" in row and not np.isnan(float(row.get("distance_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['distance_ci_lower']):.4f}, {float(row['distance_ci_upper']):.4f}]")

else:

lines.append(" No cross-T topology data")

lines.append("")



# Recovery scaling metrics

lines.append("RECOVERY SCALING METRICS")

lines.append("-" * 40)

lines.append("Cohort-level estimates (censoring-aware) and conditional-on-recovery estimates")

lines.append("")

if recovery_summaries is not None and not recovery_summaries.empty:

for *, row in recovery*summaries.iterrows():
```

```
rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}:")



# Cohort-level estimates (includes censored) - restricted times

lines.append(f" Cohort-level estimates (all seeds, censoring-aware):")

lines.append(f" Mean restricted first crossing: {float(row['mean_restricted_first']):.4f} ±
{float(row['std_restricted_first']):.4f} (n={int(row['n_restricted_first'])})")

if "ci_lower_restricted_first" in row and not np.isnan(float(row.get("ci_lower_restricted_first",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first']):.4f},
{float(row['ci_upper_restricted_first']):.4f}]")

lines.append(f" Mean restricted sustained: {float(row['mean_restricted_sustained']):.4f} ±
{float(row['std_restricted_sustained']):.4f} (n={int(row['n_restricted_sustained'])})")

if "ci_lower_restricted_sustained" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained']):.4f},
{float(row['ci_upper_restricted_sustained']):.4f}]")

lines.append(f" Mean restricted held: {float(row['mean_restricted_held']):.4f} ±
{float(row['std_restricted_held']):.4f} (n={int(row['n_restricted_held'])})")
```

```
if "ci_lower_restricted_held" in row and not np.isnan(float(row.get("ci_lower_restricted_held",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held']):.4f},
{float(row['ci_upper_restricted_held']):.4f}]")

lines.append(f" AUC deficit: {float(row['mean_auc_deficit']):.4f} ± {float(row['std_auc_deficit']):.4f}
(n={int(row['n_auc_deficit'])})")

if "ci_lower_auc_deficit" in row and not np.isnan(float(row.get("ci_lower_auc_deficit", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_auc_deficit']):.4f},
{float(row['ci_upper_auc_deficit']):.4f}]")



# Cohort-level normalized times (restricted t/T_window)

lines.append(f" Cohort-level normalized times (restricted, all seeds):")

lines.append(f" Mean restricted first/t_window: {float(row.get('mean_restricted_first_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_first_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_first_over_Twindow', 0))})")

if "ci_lower_restricted_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first_over_Twindow']):.4f},
{float(row['ci_upper_restricted_first_over_Twindow']):.4f}]")

lines.append(f" Mean restricted sustained/t_window:
{float(row.get('mean_restricted_sustained_over_Twindow', float('nan'))):.4f} ±
{float(row.get('std_restricted_sustained_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_sustained_over_Twindow', 0))})")

if "ci_lower_restricted_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained_over_Twindow", float('nan')))):
```

```
lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained_over_Twindow']):.4f},
{float(row['ci_upper_restricted_sustained_over_Twindow']):.4f}]")

lines.append(f" Mean restricted held/t_window: {float(row.get('mean_restricted_held_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_held_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_held_over_Twindow', 0))})")

if "ci_lower_restricted_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held_over_Twindow']):.4f},
{float(row['ci_upper_restricted_held_over_Twindow']):.4f}]")



# Recovery probabilities with CI

lines.append(f" Recovery probabilities (with 95% bootstrap CI):")

lines.append(f" First crossing observed: {float(row['observed_rate_first']):.4f}")

if "ci_lower_observed_rate_first" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_first", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_first']):.4f},
{float(row['ci_upper_observed_rate_first']):.4f}]")

lines.append(f" Sustained recovery observed: {float(row['observed_rate_sustained']):.4f}")

if "ci_lower_observed_rate_sustained" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_sustained']):.4f},
{float(row['ci_upper_observed_rate_sustained']):.4f}]")

lines.append(f" Held recovery observed: {float(row['observed_rate_held']):.4f}")
```

```
if "ci_lower_observed_rate_held" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_held", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_held']):.4f},
{float(row['ci_upper_observed_rate_held']):.4f}]")



# Conditional-on-recovery estimates (observed seeds only)

lines.append(f" Conditional-on-recovery estimates (observed seeds only):")

lines.append(f" First crossing: {float(row['mean_first_crossing']):.4f} ±
{float(row['std_first_crossing']):.4f} (n={int(row['n_first_crossing'])})")

if "ci_lower_first_crossing" in row and not np.isnan(float(row.get("ci_lower_first_crossing",
float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_crossing']):.4f},
{float(row['ci_upper_first_crossing']):.4f}]")

lines.append(f" Sustained: {float(row['mean_sustained']):.4f} ± {float(row['std_sustained']):.4f}
(n={int(row['n_sustained'])})")

if "ci_lower_sustained" in row and not np.isnan(float(row.get("ci_lower_sustained", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained']):.4f},
{float(row['ci_upper_sustained']):.4f}]")

lines.append(f" Held: {float(row['mean_held']):.4f} ± {float(row['std_held']):.4f}
(n={int(row['n_held'])})")

if "ci_lower_held" in row and not np.isnan(float(row.get("ci_lower_held", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held']):.4f},
{float(row['ci_upper_held']):.4f}]")
```

```
# Conditional-on-recovery normalized times (observed seeds only)

lines.append(f" Conditional-on-recovery normalized times (observed seeds only):")

lines.append(f" First crossing/t_window: {float(row['mean_first_over_Twindow']):.4f} ±
{float(row['std_first_over_Twindow']):.4f} (n={int(row['n_first_over_Twindow'])})")

if "ci_lower_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_over_Twindow']):.4f},
{float(row['ci_upper_first_over_Twindow']):.4f}]")

lines.append(f" Sustained/t_window: {float(row['mean_sustained_over_Twindow']):.4f} ±
{float(row['std_sustained_over_Twindow']):.4f} (n={int(row['n_sustained_over_Twindow'])})")

if "ci_lower_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained_over_Twindow']):.4f},
{float(row['ci_upper_sustained_over_Twindow']):.4f}]")

lines.append(f" Held/t_window: {float(row['mean_held_over_Twindow']):.4f} ±
{float(row['std_held_over_Twindow']):.4f} (n={int(row['n_held_over_Twindow'])})")

if "ci_lower_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held_over_Twindow']):.4f},
{float(row['ci_upper_held_over_Twindow']):.4f}]")

else:

lines.append(" No recovery scaling data")
```

```
lines.append("")



# Recovery interaction DID

lines.append("RECOVERY INTERACTION DID")

lines.append("-" * 40)

lines.append("I_{s,ρ,b}^recovery = [R_{s,ρ,b} - R_{s,ρ,continue}] - [R_{s,1,b} - R_{s,1,continue}]")

lines.append("")

if recovery_did_summary is not None and not recovery_did_summary.empty:

for *, row in recovery*did_summary.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

outcome = str(row["outcome"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}, {outcome}:")

mean_val = float(row["mean"])

std_val = float(row["std"])
```

```
n_val = int(row["n"])

ci_lower = float(row.get("ci_lower", float('nan')))

ci_upper = float(row.get("ci_upper", float('nan')))


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" DID: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No recovery DID data")

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(

text
```

```
csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,

bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)
```

```
output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,


"source": {

"path": str(source_path),

"sha256": source_hash,

},


"command_line": command_line,


"bootstrap": {

"seed": bootstrap_seed,
```

"n_iterations": n_bootstrap,

},

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

P1 #17: Add recovery expected counts

"expected_n_recovery_rows": 3200,

"expected_n_recovery_groups": EXPECTED_N_RECOVERY_GROUPS,

"expected_n_recovery_did_rows": EXPECTED_N_RECOVERY_DID,

"expected_n_recovery_did_groups": EXPECTED_N_RECOVERY_DID_GROUPS,

P1 #9, #10, #11: Document conventions

"similarity_nan_recovery_convention": "undefined zero-vector cosine imputed as zero for recovery and AUC only",

"density_horizon_duplication": "Density recovery rows are duplicated across horizon_system by design. They are horizon-independent and must not be pooled across the two labels.",

"discrete_held_duration_convention": "ceil(T_window / diagnostic_sample_spacing) consecutive saved samples",

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}

return manifest

==
Main Execution
==

def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"run_dir",

type=str,

help="run directory containing study outputs",

)

parser.add_argument(

"--output",

type=str,

default=None,

help="output directory for analysis (default: <run_dir>/analysis)",

)

parser.add_argument(

"--n-bootstrap",

type=int,

default=1000,

help="number of bootstrap iterations (default: 1000)",

)

parser.add_argument(

"--bootstrap-seed",

```
type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

parser.add_argument(

"--overwrite",

action="store_true",

default=False,

help="overwrite existing output files (default: fail if outputs exist)",

)

return parser.parse_args()

def validate_bootstrap_args(args):

text

"""Validate bootstrap arguments."""

if args.n_bootstrap &lt;= 0:

raise ValueError("--n-bootstrap must be positive")

if args.bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required")
```

return True

def main() -> None:

text

args = parse_arguments()

Validate bootstrap arguments

validate_bootstrap_args(args)

Parse run directory

run_dir = Path(args.run_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))

if not csv_files:

print(f"No CSV file found in {run_dir}")

sys.exit(1)

```
if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



# Require exactly one input CSV and NPZ

if len(csv_files) != 1:

print(f"Expected exactly one scalar CSV, found {len(csv_files)}: {[[p.name](http://p.name) for p in csv_files]}")

sys.exit(1)

if len(npz_files) != 1:

print(f"Expected exactly one branch NPZ, found {len(npz_files)}: {[[p.name](http://p.name) for p in npz_files]}")

sys.exit(1)

if len(manifest_files) != 1:

print(f"Expected exactly one manifest, found {len(manifest_files)}")
```

```
sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# P1 #12: Validate inputs BEFORE deleting stale outputs

# This prevents malformed runs from causing deletion of valid previous analysis

print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print()



# Stage 1: Validation - MUST HAPPEN BEFORE any file operations

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)
```

```
if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Set up output directory with stale-file protection

if args.output is None:

output_dir = run_dir / "analysis"

else:

output_dir = Path(args.output)



# Save command line for manifest

command_line = " ".join(sys.argv)
```

```
# Now that validation passed, check for and remove stale files
expected_outputs = [
"twindow_paired_regime_contrasts.csv",
"twindow_paired_regime_contrasts_fixed_physical.csv",
"twindow_paired_regime_contrasts_normalized_EMA.csv",
"twindow_branch_interactions.csv",
"twindow_branch_interactions_fixed_physical.csv",
"twindow_branch_interactions_normalized_EMA.csv",
"twindow_cross_condition_topology.csv",
"twindow_recovery_by_seed.csv",
"twindow_recovery_summary.csv",
"twindow_regime_summary.csv",
"twindow_branch_DID_summary.csv",
"twindow_topology_summary.csv",
"twindow_regime_secondary_summary.csv",
"twindow_branch_DID_secondary_summary.csv",
```

```
"twindow_recovery_interactions.csv",

"twindow_recovery_interactions_summary.csv",

"twindow_sensitivity_report.txt",

"analysis_manifest.json",

]



stale_files = []

if output_dir.exists():

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

stale_files.append(expected_file)


if stale_files and not args.overwrite:

print(f"❌ Output directory contains stale files: {stale_files[:5]}...")

print(f" Use --overwrite to force overwrite, or specify a different --output directory")

print(f" This prevents mixing outputs from different analysis versions.")

sys.exit(1)
```

```
# If --overwrite is passed, remove ALL expected files before starting

if stale_files and args.overwrite:

print(f" Removing {len(stale_files)} stale files from output directory...")

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

try:

file_path.unlink()

except Exception as e:

print(f" WARNING: Could not remove stale file {expected_file}: {e}")



output_dir.mkdir(parents=True, exist_ok=True)



print(f" Output: {output_dir}")

print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()
```

```
if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")

df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)
```

```
seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

# Handle both manifest formats for rho_values

if "study" in manifest and "rho_values" in manifest["study"]:

rho_values = [float(r) for r in manifest["study"]["rho_values"]]

elif "protocol" in manifest and "rho_values" in manifest["protocol"]:

rho_values = [float(r) for r in manifest["protocol"]["rho_values"]]

else:

# If we can't find rho_values, use the unique values from the data

rho_values = sorted([float(r) for r in df["rho"].unique()])

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")

print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts (Primary outcomes - fixed_physical)
```

```
print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary regime contrasts

regime_contrasts["horizon_system"] = "fixed_physical"



# Stage 2b: Regime Contrasts (Secondary normalized-EMA outcomes)

print("\nStage 2b: Computing secondary regime contrasts...")

regime_contrasts_secondary = compute_regime_contrasts(df,
SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary regime contrasts

regime_contrasts_secondary["horizon_system"] = "normalized_EMA"



# Combine primary and secondary regime contrasts into long format

regime_contrasts_combined = pd.concat([regime_contrasts, regime_contrasts_secondary],
ignore_index=True)
```

```
# Stage 3: Branch DID (Primary outcomes - fixed_physical)

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary branch DID

branch_did["horizon_system"] = "fixed_physical"



# Stage 3b: Branch DID (Secondary normalized-EMA outcomes)

print("\nStage 3b: Computing secondary branch difference-in-differences...")

branch_did_secondary = compute_branch_did(df, SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary branch DID

branch_did_secondary["horizon_system"] = "normalized_EMA"



# Combine primary and secondary branch DID into long format

branch_did_combined = pd.concat([branch_did, branch_did_secondary], ignore_index=True)
```

```
# Stage 4: Cross-T Topology

print("\nStage 4: Computing cross-T topology metrics...")

topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)



# Stage 4b: Recovery Scaling (REQUIRED for production)

print("\nStage 4b: Computing recovery scaling metrics...")

recovery = compute_recovery_scaling(df, npz_path, b_c)



# Validate recovery groups have exactly 40 seeds each

print("\n Validating recovery group sizes...")

recovery_group_sizes = recovery.groupby(

["rho", "branch", "horizon_system", "metric"]

).size()



# P0 #5: Validate expected number of recovery groups

if len(recovery_group_sizes) != EXPECTED_N_RECOVERY_GROUPS:

raise RuntimeError(
```

```
f"Expected {EXPECTED_N_RECOVERY_GROUPS} recovery groups, "

f"got {len(recovery_group_sizes)}"

)



bad_groups = recovery_group_sizes[recovery_group_sizes != EXPECTED_N_SEEDS]

if not bad_groups.empty:

raise RuntimeError(

f"Recovery groups do not all contain {EXPECTED_N_SEEDS} seed-level rows. "

f"Bad groups: {bad_groups.head().to_dict()}"

)



# Validate restricted recovery fields have finite values

print(" Validating restricted recovery fields...")

for field in RESTRICTED_RECOVERY_FIELDS:

non_finite = ~np.isfinite([pd.to](http://pd.to)_numeric(recovery[field], errors="coerce"))

if non_finite.any():

raise RuntimeError(
```

```
f"Recovery field '{field}' has {int(non_finite.sum())} non-finite values"

)



print(" ✅ Recovery validation passed")



# Stage 4c: Recovery Interaction DID

print("\nStage 4c: Computing recovery interaction DID...")

recovery_did = compute_recovery_did(recovery)



# Stage 5: Seed-Level Summaries

print("\nStage 5: Computing seed-level summaries...")

summaries = compute_seed_level_summaries(

regime_contrasts, branch_did, topology, recovery,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)
```

```
# Stage 5b: Secondary normalized-EMA summaries

print("\nStage 5b: Computing secondary seed-level summaries...")

secondary_summaries = compute_seed_level_summaries(

regime_contrasts_secondary, branch_did_secondary, pd.DataFrame(), None,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Validate recovery DID groups have exactly 40 seeds each

print("\n Validating recovery DID group sizes...")

recovery_did_group_sizes = recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

).size()



# P0 #5: Validate expected number of recovery DID groups

if len(recovery_did_group_sizes) != EXPECTED_N_RECOVERY_DID_GROUPS:

raise RuntimeError(
```

```
f"Expected {EXPECTED_N_RECOVERY_DID_GROUPS} recovery DID groups, "

f"got {len(recovery_did_group_sizes)}"

)



if not (recovery_did_group_sizes == EXPECTED_N_SEEDS).all():

bad_groups = recovery_did_group_sizes[recovery_did_group_sizes != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Recovery DID groups do not all contain {EXPECTED_N_SEEDS} seed-level rows. "

f"Bad groups: {bad_groups.head().to_dict()}"

)

print(" ✅ Recovery DID validation passed")



# Stage 5c: Recovery DID summaries

print("\nStage 5c: Computing recovery DID summaries...")

recovery_did_summaries = {}

if not recovery_did.empty:

recovery_did_summary = recovery_did.groupby(
```

```
["rho", "branch", "horizon_system", "metric", "outcome"],

as_index=False,

).agg(

mean=("recovery_difference_in_differences", "mean"),

std=("recovery_difference_in_differences", "std"),

n=("recovery_difference_in_differences", "count"),

)


# Add bootstrap CI

rng_did = np.random.default_rng(args.bootstrap_seed)

for (rho, branch, horizon_system, metric, outcome), group in recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

):

values = group["recovery_difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng_did.integers(0, len(values), size=(args.n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)
```

```
ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = recovery_did_summary[

(recovery_did_summary["rho"] == rho) &amp;amp;

(recovery_did_summary["branch"] == branch) &amp;amp;

(recovery_did_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_did_summary["metric"] == metric) &amp;amp;

(recovery_did_summary["outcome"] == outcome)

].index

recovery_did_summary.loc[idx, "ci_lower"] = ci_lower

recovery_did_summary.loc[idx, "ci_upper"] = ci_upper


recovery_did_summaries["recovery_did"] = recovery_did_summary

print(f"✅ Stage 5c: Recovery DID Summaries - {len(recovery_did_summary)} groups")



regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())
```

```
topology_summaries = summaries.get("topology", pd.DataFrame())

recovery_summaries = summaries.get("recovery", pd.DataFrame())

regime_summaries_secondary = secondary_summaries.get("regime_contrasts", pd.DataFrame())

did_summaries_secondary = secondary_summaries.get("branch_did", pd.DataFrame())

recovery_did_summary = recovery_did_summaries.get("recovery_did", pd.DataFrame())



# P1 #6: Validate summary group counts

print("\n Validating summary group counts...")



# Regime summaries: 4 non-baseline rho * 3 outcomes = 12 groups, each with n=40

if not regime_summaries.empty:

if len(regime_summaries) != 12:

raise RuntimeError(

f"Expected 12 regime summary groups, got {len(regime_summaries)}"

)

if not (regime_summaries["n"] == EXPECTED_N_SEEDS).all():

bad_groups = regime_summaries[regime_summaries["n"] != EXPECTED_N_SEEDS]
```

```
raise RuntimeError(

f"Regime summary groups do not all have n={EXPECTED_N_SEEDS}:
{bad_[groups.to](http://groups.to)_dict()}"

)



# Branch DID summaries: 4 non-baseline rho *3 branches* 3 outcomes = 36 groups, each with n=40

if not did_summaries.empty:

if len(did_summaries) != 36:

raise RuntimeError(

f"Expected 36 branch DID summary groups, got {len(did_summaries)}"

)

if not (did_summaries["n"] == EXPECTED_N_SEEDS).all():

bad_groups = did_summaries[did_summaries["n"] != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Branch DID summary groups do not all have n={EXPECTED_N_SEEDS}:
{bad_groups.head().to_dict()}"

)
```

```
# P1 #3: Validate secondary regime and DID summary counts

# Secondary regime summaries: 4 non-baseline rho * 3 outcomes = 12 groups, each with n=40

if not regime_summaries_secondary.empty:

if len(regime_summaries_secondary) != 12:

raise RuntimeError(

f"Expected 12 secondary regime summary groups, got {len(regime_summaries_secondary)}"

)

if not (regime_summaries_secondary["n"] == EXPECTED_N_SEEDS).all():

bad_groups = regime_summaries_secondary[regime_summaries_secondary["n"] !=
EXPECTED_N_SEEDS]

raise RuntimeError(

f"Secondary regime summary groups do not all have n={EXPECTED_N_SEEDS}: {bad_[groups.to]
(http://groups.to)_dict()}"

)



# Secondary branch DID summaries: 4 non-baseline rho *3 branches* 3 outcomes = 36 groups, each
with n=40

if not did_summaries_secondary.empty:

if len(did_summaries_secondary) != 36:
```

```
raise RuntimeError(

f"Expected 36 secondary branch DID summary groups, got {len(did_summaries_secondary)}"

)

if not (did_summaries_secondary["n"] == EXPECTED_N_SEEDS).all():

bad_groups = did_summaries_secondary[did_summaries_secondary["n"] != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Secondary branch DID summary groups do not all have n={EXPECTED_N_SEEDS}:
{bad_groups.head().to_dict()}"

)



# Topology summary: 4 rho rows, each metric n=40

if not topology_summaries.empty:

if len(topology_summaries) != 4:

raise RuntimeError(

f"Expected 4 topology summary rows (one per non-baseline rho), got {len(topology_summaries)}"

)

for col in ["cosine_n", "jaccard_n", "distance_n"]:

if not (topology_summaries[col] == EXPECTED_N_SEEDS).all():
```

```
raise RuntimeError(
f"Topology summary {col} does not all equal {EXPECTED_N_SEEDS}"
)



# Recovery summaries: 80 groups (5 rho *4 branches* 2 systems * 2 metrics)
if not recovery_summaries.empty:
if len(recovery_summaries) != EXPECTED_N_RECOVERY_GROUPS:
raise RuntimeError(
f"Expected {EXPECTED_N_RECOVERY_GROUPS} recovery summary groups, got
{len(recovery_summaries)}"
)
# Check restricted fields have n=40
for field in ["n_restricted_first", "n_restricted_sustained", "n_restricted_held",
"n_restricted_first_over_Twindow", "n_restricted_sustained_over_Twindow",
"n_restricted_held_over_Twindow", "n_auc_deficit"]:
if field in recovery_summaries.columns:
if not (recovery_summaries[field] == EXPECTED_N_SEEDS).all():
bad_groups = recovery_summaries[recovery_summaries[field] != EXPECTED_N_SEEDS]
```

```
raise RuntimeError(

f"Recovery summary {field} does not all equal {EXPECTED_N_SEEDS}:
{bad_groups.head().to_dict()}"

)



# Recovery DID summary: 624 groups

# P0 Issue 2: Distinguish full-cohort vs conditional outcomes for n validation

if not recovery_did_summary.empty:

if len(recovery_did_summary) != EXPECTED_N_RECOVERY_DID_GROUPS:

raise RuntimeError(

f"Expected {EXPECTED_N_RECOVERY_DID_GROUPS} recovery DID summary groups, got
{len(recovery_did_summary)}"

)



# Full-cohort outcomes must have n=40 for all groups

full_cohort = recovery_did_summary[

recovery_did_summary["outcome"].isin(FULL_COHORT_RECOVERY_DID_OUTCOMES)

]

if not (full_cohort["n"] == EXPECTED_N_SEEDS).all():
```

```
bad_groups = full_cohort[full_cohort["n"] != EXPECTED_N_SEEDS]
raise RuntimeError(
f"Full-cohort recovery DID summaries do not all have n={EXPECTED_N_SEEDS}: "
f"{bad_groups.head().to_dict()}"
)

# Conditional outcomes may have n in [0, 40] due to censoring (NaN excluded from count)
conditional = recovery_did_summary[
recovery_did_summary["outcome"].isin(CONDITIONAL_RECOVERY_DID_OUTCOMES)
]
if len(conditional) &gt; 0:
if np.any((conditional["n"] &lt; 0) | (conditional["n"] &gt; EXPECTED_N_SEEDS)):
bad_groups = conditional[
(conditional["n"] &lt; 0) | (conditional["n"] &gt; EXPECTED_N_SEEDS)
]
raise RuntimeError(
f"Conditional recovery DID summary n lies outside [0, 40]: "
f"{bad_groups.head().to_dict()}"
```

```
)


print(" ✅ All summary group counts validated")


# Save outputs - construct output_files incrementally

print("\nSaving outputs...")

output_files: dict[str, Path] = {}


# Paired regime contrasts (combined primary + secondary)

regime_contrast_combined_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_combined_path, regime_contrasts_combined)

output_files["regime_contrasts_combined"] = regime_contrast_combined_path

print(f" Saved: {regime_contrast_combined_[path.name](http://path.name)}")


# Paired regime contrasts (primary only - fixed_physical)

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts_fixed_physical.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)
```

```
output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Paired regime contrasts (secondary only - normalized_EMA)

regime_contrast_secondary_path = output_dir /
"twindow_paired_regime_contrasts_normalized_EMA.csv"

save_csv_atomic(regime_contrast_secondary_path, regime_contrasts_secondary)

output_files["regime_contrasts_secondary"] = regime_contrast_secondary_path

print(f" Saved: {regime_contrast_secondary_[path.name](http://path.name)}")



# Branch DID (combined primary + secondary)

branch_did_combined_path = output_dir / "twindow_branch_interactions.csv"

save_csv_atomic(branch_did_combined_path, branch_did_combined)

output_files["branch_did_combined"] = branch_did_combined_path

print(f" Saved: {branch_did_combined_[path.name](http://path.name)}")



# Branch DID (primary only - fixed_physical)

branch_did_path = output_dir / "twindow_branch_interactions_fixed_physical.csv"
```

```
save_csv_atomic(branch_did_path, branch_did)

output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")



# Branch DID (secondary only - normalized_EMA)

branch_did_secondary_path = output_dir / "twindow_branch_interactions_normalized_EMA.csv"

save_csv_atomic(branch_did_secondary_path, branch_did_secondary)

output_files["branch_did_secondary"] = branch_did_secondary_path

print(f" Saved: {branch_did_secondary_[path.name](http://path.name)}")



# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")
```

```
# Recovery scaling raw output

if not recovery.empty:

recovery_raw_path = output_dir / "twindow_recovery_by_seed.csv"

save_csv_atomic(recovery_raw_path, recovery)

output_files["recovery_by_seed"] = recovery_raw_path

print(f" Saved: {recovery_raw_[path.name](http://path.name)}")



# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)

output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)
```

```
output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Recovery summaries (censoring-aware)

if not recovery_summaries.empty:

recovery_summary_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_summary_path, recovery_summaries)

output_files["recovery_summary"] = recovery_summary_path

print(f" Saved: {recovery_summary_[path.name](http://path.name)}")



# Recovery DID (raw)
```

```
if not recovery_did.empty:

recovery_did_path = output_dir / "twindow_recovery_interactions.csv"

save_csv_atomic(recovery_did_path, recovery_did)

output_files["recovery_did"] = recovery_did_path

print(f" Saved: {recovery_did_[path.name](http://path.name)}")



# Recovery DID summaries

if not recovery_did_summary.empty:

recovery_did_summary_path = output_dir / "twindow_recovery_interactions_summary.csv"

save_csv_atomic(recovery_did_summary_path, recovery_did_summary)

output_files["recovery_did_summary"] = recovery_did_summary_path

print(f" Saved: {recovery_did_summary_[path.name](http://path.name)}")



# Secondary summaries

if not regime_summaries_secondary.empty:

regime_summary_secondary_path = output_dir / "twindow_regime_secondary_summary.csv"

save_csv_atomic(regime_summary_secondary_path, regime_summaries_secondary)
```

```
output_files["regime_summary_secondary"] = regime_summary_secondary_path

print(f" Saved: {regime_summary_secondary_[path.name](http://path.name)}")



if not did_summaries_secondary.empty:

did_summary_secondary_path = output_dir / "twindow_branch_DID_secondary_summary.csv"

save_csv_atomic(did_summary_secondary_path, did_summaries_secondary)

output_files["did_summary_secondary"] = did_summary_secondary_path

print(f" Saved: {did_summary_secondary_[path.name](http://path.name)}")



# Text report

report = generate_text_report(

validation, regime_contrasts, branch_did, topology,

regime_summaries, did_summaries, topology_summaries,

recovery_summaries, regime_summaries_secondary, did_summaries_secondary,

recovery_did_summary

)

report_path = output_dir / "twindow_sensitivity_report.txt"
```

```
save_text_atomic(report_path, report)

output_files["report"] = report_path

print(f" Saved: {report_[path.name](http://path.name)}")



# Analysis manifest

analysis_manifest = generate_analysis_manifest(

csv_path, npz_path, manifest_path, validation, output_files,

source_path=Path(__file__),

bootstrap_seed=args.bootstrap_seed,

n_bootstrap=args.n_bootstrap,

command_line=command_line,

)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out

print(f" Saved: {manifest_path_[out.name](http://out.name)}")
```

```
# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text

main()
```

Ignoring copy/paste corruption, the two previous P0 problems are now fixed:

similarity NaNs are permitted while infinities are rejected;
recovery-DID summary n distinguishes full-cohort from conditional outcomes.

Verdict

Primary/secondary estimands: correct
Topology validation: strong
Recovery logic: correct

Recovery-DID logic: correct
Censoring handling: correctly separated
Input provenance: strong
Decision: GO for staged testing

I do not find a remaining deterministic runtime or scientific-estimand blocker in this version.
One recommended gate-hardening correction
Require the study manifest format instead of silently supporting alternatives

This analysis is specifically versioned for:

text

AIEF_1_4_Twindow_sensitivity

The simulation driver always produces:

Python

manifest["study"]["rho_values"]
manifest["study"]["T_window_values"]
manifest["protocol"]["branches"]
manifest["randomization"]["paired_seeds"]

The new fallback logic can silently skip validation if those fields are missing:

Python

manifest_rho_values = None
...
if all values are not None:
validate

and later:

Python

rho_values_list = []
twindow_values_list = []

If T_window_values is absent or has the wrong length, validation can be skipped rather than failed. That weakens the proof-oriented input gate.
Preferred strict validation

At the start of validate_run() require:

Python

required_manifest_paths = (
("study", "rho_values"),
("study", "T_window_values"),
("protocol", "branches"),
("randomization", "paired_seeds"),
("model", "B_c"),
)

for section, key in required_manifest_paths:
if (
section not in manifest
or key not in manifest[section]
):
results["status"] = "failed"
results["errors"].append(
f"Simulation manifest missing "
f"{section}.{key}"
)
return results

Then use only the expected study format:

Python

manifest_rho_values = tuple(
float(x)
for x in manifest["study"]["rho_values"]
)

```
manifest_twindow_values = tuple(
float(x)
for x in manifest["study"]["T_window_values"]
)
manifest_branches = tuple(
manifest["protocol"]["branches"]
)
manifest_seeds = tuple(
str(x)
for x in manifest["randomization"]["paired_seeds"]
)

if manifest_rho_values != EXPECTED_RHO_VALUES:
...

if manifest_branches != EXPECTED_BRANCHES:
...

if len(manifest_seeds) != EXPECTED_N_SEEDS:
...

if len(manifest_twindow_values) != len(
EXPECTED_RHO_VALUES
):
results["status"] = "failed"
results["errors"].append(
"Manifest T_window_values length mismatch"
)
return results
```

Do not downgrade inability to validate TwindowTwindow to a warning:

Python

```
except Exception as exc:
results["status"] = "failed"
results["errors"].append(
f"Could not validate T_window metadata: {exc}"
)
```

return results

There is no reason for this study-analysis driver to accept the historical frozen-production manifest as its simulation input.

This is certificate hardening rather than an expected failure with the correctly generated study manifest.
Small consistency improvements
1. Use np.isnan() for the documented NaN count

After rejecting infinities, this:

Python

np.sum(~np.isfinite(similarity_trajectory))

is numerically equivalent to counting NaNs. For semantic clarity use:

Python

similarity_nan_count = int(
np.sum(np.isnan(similarity_trajectory))
)
cont_similarity_nan_count = int(
np.sum(np.isnan(cont_similarity_trajectory))
)

2. Record that AUC is horizon-independent

Similarity targets differ by horizon system, but:

Python

auc_deficit_vs_continue

does not depend on the target. It is duplicated across horizon labels, just as density metrics are duplicated.

Add:

Python

"auc_horizon_dependence": "horizon_independent",

and note that repeated AUC rows must not be pooled across horizon labels.
3. Assert recovery-DID outcome classification is complete

At module import:

Python

```
ALL_RECOVERY_DID_OUTCOMES = (
FULL_COHORT_RECOVERY_DID_OUTCOMES
| CONDITIONAL_RECOVERY_DID_OUTCOMES
)

if len(ALL_RECOVERY_DID_OUTCOMES) != 13:
raise RuntimeError(
"Recovery DID outcome classification "
"must contain exactly 13 outcomes"
)

if (
FULL_COHORT_RECOVERY_DID_OUTCOMES
& CONDITIONAL_RECOVERY_DID_OUTCOMES
):
raise RuntimeError(
"Recovery DID full-cohort and conditional "
"outcome sets overlap"
)
```

This prevents future schema drift.
Nonblocking cleanup

MATRIX_KEYS is unused.
df is unused in compute_cross_t_topology().

b_c is unused in compute_recovery_scaling().
top-level csv and hashlib imports appear unused.
validation status is checked twice in main().
conditional raw sustained/held CIs remain optional and absent.
the report header lists only a subset of generated files.
analysis-manifest output hashes intentionally exclude the manifest itself; that is appropriate because self-hashing is recursive.

What is now correct

This version now has:

exact 40-seed study cohort validation;
exact rho and branch validation;
exact condition-status and scalar-table coverage;
exact input-file hash verification;
exact baseline certificate seed cohort;
exact seven-branch baseline certificate protocol;
valid TwindowTwindow and density targets;
finite primary and secondary outcomes;
validated 12×1212×12 final matrices;
finite topology metrics;
exactly 160 topology comparisons;
exact 640-sample schedules;
density range checks;
allowed NaN cosine with explicit recovery-only zero imputation;
exact 3,200 recovery rows;
exact 80 recovery groups;
exact 24,960 recovery-DID rows;
exact 624 recovery-DID groups;
40 physical seed rows per group;
correct full-cohort versus conditional summary counts;
primary and secondary summary group validation;
input validation before stale-output deletion;
atomic output writing and complete provenance.

Final assessment

Analysis-driver static review: PASS for staged testing.

The remaining strict-manifest recommendation is worth applying before freezing the analysis source, but the correctly generated study manifest should already satisfy it. No analysis result exists until the regenerated grid is complete and this script runs successfully.

Okay so re articulate the workflow for testing the scripts and doing base line checks, all the way to actually using the scripts.

Below is the full gated workflow from source validation through frozen-baseline certification, regenerated grid execution, analysis, and freezing.

Rule: stop at the first failure. Do not use --overwrite during the initial pass, and do not launch the grid unless the baseline-only certificate passes exactly.
Phase 0 — Finalize and identify the sources

Run from the directory containing the scripts:

Bash

```
set -euo pipefail

export ENGINE="aief_experiment_engine_v1.py"
export DRIVER="AIEF_1_4_Twindow_sensitivity.py"
export ANALYSIS="AIEF_1_4_Twindow_sensitivity_analysis.py"

export FROZEN_DIR="runs/5a9f093a2fe0"
export FROZEN_MANIFEST="$FROZEN_DIR/manifest.json"
export FROZEN_NPZ="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz"
export FROZEN_SCALAR="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv"

test -f "$ENGINE"
test -f "$DRIVER"
test -f "$ANALYSIS"
test -f "$FROZEN_MANIFEST"
test -f "$FROZEN_NPZ"
test -f "$FROZEN_SCALAR"
```

Record source and frozen-authority hashes:

Bash

```
sha256sum \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS" \
> twindow_pretest_source_sha256.txt

sha256sum \
"$FROZEN_MANIFEST" \
"$FROZEN_NPZ" \
"$FROZEN_SCALAR" \
> twindow_frozen_authority_sha256.txt

cat twindow_pretest_source_sha256.txt
cat twindow_frozen_authority_sha256.txt
```

Do not edit any script after this point. If you do, restart from Phase 0.
Phase 1 — Record the numerical environment

Bash

```
python3 - <<'PY'
import json
import platform
import sys

import numpy as np
import pandas as pd

with open(
"runs/5a9f093a2fe0/manifest.json",
encoding="utf-8",
) as handle:
frozen = json.load(handle)

print("Current:")
print(" Python:", sys.version)
```

```
print(" NumPy: ", np.__version__)
print(" pandas:", pd.__version__)
print(" OS: ", platform.platform())
print()
print("Frozen environment:")
print(json.dumps(frozen.get("environment", {}), indent=2))
PY
```

An environment mismatch does not automatically invalidate the attempt, but it is relevant if exact numerical replay fails.

Phase 2 — Compile, import, and CLI checks

2.1 Compile all three

Bash

```
python3 -m py_compile \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS"
```

2.2 Import all three

This catches module-level errors that py_compile may miss.

Bash

```
python3 - <<'PY'
import aief_experiment_engine_v1
import AIEF_1_4_Twindow_sensitivity
import AIEF_1_4_Twindow_sensitivity_analysis

print("All three modules imported successfully.")
PY
```

2.3 Run the engine’s built-in validation

Bash

```
python3 "$ENGINE"
```

2.4 Test both CLIs

Bash

```
python3 "$DRIVER" --help >/dev/null
python3 "$ANALYSIS" --help >/dev/null

echo "CLI checks passed."
```

Phase 3 — Static schema and count assertions

Bash

```
python3 - <<'PY'
import aief_experiment_engine_v1 as e
import AIEF_1_4_Twindow_sensitivity as d
import AIEF_1_4_Twindow_sensitivity_analysis as a

# Engine configuration.
e.DEFAULT_MODEL_CONFIG.validate()
e.DEFAULT_PROTOCOL_CONFIG.validate(
e.DEFAULT_MODEL_CONFIG
)

# Engine extended schemas.
assert len(e.EXTENDED_BRANCH_MATRIX_KEYS) == len(
set(e.EXTENDED_BRANCH_MATRIX_KEYS)
)
assert set(e.EXTENDED_BRANCH_MATRIX_KEYS) == set(
e.EXTENDED_MATRIX_EQUIVALENCE_MAP
)

assert len(e.EXTENDED_BRANCH_SCALAR_KEYS) == len(
set(e.EXTENDED_BRANCH_SCALAR_KEYS)
)
assert set(e.EXTENDED_BRANCH_SCALAR_KEYS) == set(
```

```
e.EXTENDED_SCALAR_EQUIVALENCE_MAP
)

# Driver/engine constants.
d.validate_engine_contract()
assert tuple(d.FROZEN_BRANCH_ORDER) == tuple(
e.DEFAULT_PROTOCOL_CONFIG.branches
)
assert set(d.STUDY_BRANCHES) <= set(
d.FROZEN_BRANCH_ORDER
)
assert len(d.STUDY_BRANCHES) == len(
set(d.STUDY_BRANCHES)
)
assert len(d.RHO_VALUES) == len(set(d.RHO_VALUES))
assert 1.0 in d.RHO_VALUES

# Analysis prespecified counts.
assert a.EXPECTED_N_CONDITIONS == 200
assert a.EXPECTED_N_ROWS == 800
assert a.EXPECTED_N_REGIME_CONTRASTS == 480
assert a.EXPECTED_N_DID == 1440
assert a.EXPECTED_N_TOPOLOGY == 160
assert a.EXPECTED_N_RECOVERY_GROUPS == 80
assert a.EXPECTED_N_RECOVERY_DID == 24960
assert a.EXPECTED_N_RECOVERY_DID_GROUPS == 624

assert len(
a.FULL_COHORT_RECOVERY_DID_OUTCOMES
| a.CONDITIONAL_RECOVERY_DID_OUTCOMES
) == 13
assert not (
a.FULL_COHORT_RECOVERY_DID_OUTCOMES
& a.CONDITIONAL_RECOVERY_DID_OUTCOMES
)

print("Static schema and count assertions passed.")
PY
```

Phase 4 — Cheap analysis helper smoke tests

Bash

```
python3 - <<'PY'
import numpy as np
import AIEF_1_4_Twindow_sensitivity_analysis as a

times = np.array([0.5, 1.0, 1.5, 2.0])
values = np.array([0.0, 0.8, 0.9, 1.0])

t, observed = a.first_crossing_time(
times,
values,
0.9,
)
assert observed and t == 1.5

t, observed = a.first_time_sustained_for_rest(
times,
values,
0.8,
)
assert observed and t == 1.0

auc = a.trapezoidal_auc_deficit(
times,
values,
values,
)
assert auc == 0.0

# NaN cosine is below threshold for recovery.
nan_values = np.array([np.nan, 0.8, 0.9, 1.0])
t, observed = a.first_crossing_time(
times,
nan_values,
0.9,
)
assert observed and t == 1.5
```

```
# NaNs are imputed as zero for recovery AUC.
auc = a.trapezoidal_auc_deficit(
times,
np.array([0.5, 0.5, 0.5, 0.5]),
np.array([np.nan, 0.5, 0.5, 0.5]),
)
assert np.isfinite(auc)
assert auc >= 0.0

print("Analysis helper smoke tests passed.")
PY
```

Phase 5 — Driver dry-run

The finalized driver’s dry-run should not require frozen paths.

Bash

```
python3 "$DRIVER" --dry-run
```

It should report:

text

```
5 rho values
4 study branches
40 seeds
configuration validated successfully
```

It must not create a production run directory.
Phase 6 — One-seed exact frozen replay smoke test

This runs one frozen seed through all seven historical branches and checks:

prefix arrays;
historical matrix/trajectory fields;

all 42 historical scalar fields;
all extended ρ=1ρ=1 identities;
horizon metadata.

Bash

```
python3 - <<'PY'
import csv
import json
import math
import os

import numpy as np
import aief_experiment_engine_v1 as e

manifest_path = os.environ["FROZEN_MANIFEST"]
npz_path = os.environ["FROZEN_NPZ"]
csv_path = os.environ["FROZEN_SCALAR"]

with open(manifest_path, encoding="utf-8") as handle:
manifest = json.load(handle)

seed = int(manifest["randomization"]["paired_seeds"][0])
branch_order = tuple(manifest["protocol"]["branches"])

config = e.DEFAULT_MODEL_CONFIG
protocol = e.DEFAULT_PROTOCOL_CONFIG

config.validate()
protocol.validate(config)

split_state, prefix = e.simulate_prefix(
seed,
config,
protocol,
)

branch_rngs = e.make_branch_rngs(
```

```
seed,
branch_order,
)

branch_results = {}

for branch in branch_order:
branch_results[branch] = e.simulate_future(
split_state,
prefix,
branch,
config,
protocol,
branch_rngs[branch],
)

e.finalize_historical_branch_metrics(
branch_results
)

with np.load(npz_path, allow_pickle=False) as frozen:
for key in e.REQUIRED_PREFIX_KEYS:
authority_key = f"prefix_seed_{seed}_{key}"
assert e.exact_array_equal_with_nan(
prefix[key],
frozen[authority_key],
), authority_key

for branch in branch_order:
for key in e.FROZEN_BRANCH_MATRIX_KEYS:
authority_key = (
f"{branch}_seed_{seed}_{key}"
)
assert e.exact_array_equal_with_nan(
branch_results[branch][key],
frozen[authority_key],
), authority_key

with open(
csv_path,
```

```
newline="",
encoding="utf-8",
) as handle:
rows = list(csv.DictReader(handle))


frozen_rows = {
(int(row["seed"]), row["branch"]): row
for row in rows
}


for branch in branch_order:
result = branch_results[branch]
frozen_row = frozen_rows[(seed, branch)]


for key in e.FROZEN_BRANCH_SCALAR_KEYS:
observed = float(result[key])
expected = float(frozen_row[key])


equal = (
math.isnan(observed)
and math.isnan(expected)
) or observed == expected


assert equal, (
branch,
key,
observed,
expected,
)


for key, legacy in (
e.EXTENDED_MATRIX_EQUIVALENCE_MAP.items()
):
assert e.exact_array_equal_with_nan(
result[key],
result[legacy],
), (branch, key, legacy)


for key, legacy in (
```

```
e.EXTENDED_SCALAR_EQUIVALENCE_MAP.items()
):
observed = float(result[key])
expected = float(result[legacy])


equal = (
math.isnan(observed)
and math.isnan(expected)
) or observed == expected


assert equal, (
branch,
key,
legacy,
observed,
expected,
)


for key, expected in (
e.BASELINE_HORIZON_METADATA.items()
):
assert int(result[key]) == int(expected), (
branch,
key,
result[key],
expected,
)


print(
"One-seed, seven-branch exact frozen replay passed:",
seed,
)
PY
```

If this fails, do not run the 40-seed baseline.
Phase 7 — One-condition parameterized driver smoke test

Use a nonbaseline value such as $\rho=0.5$ρ=0.5 to exercise distinct fixed and normalized horizons.

Bash

```
python3 - <<'PY'
import json
import os

import AIEF_1_4_Twindow_sensitivity as d

with open(
os.environ["FROZEN_MANIFEST"],
encoding="utf-8",
) as handle:
frozen = json.load(handle)

seed = int(
frozen["randomization"]["paired_seeds"][0]
)
rho = 0.5

config = d.create_model_config(rho)
protocol = d.create_protocol_config()

config.validate()
protocol.validate(config)

result = d.simulate_seed_rho(
seed=seed,
rho=rho,
config=config,
protocol=protocol,
study_branches=d.STUDY_BRANCHES,
)

d.validate_prefix_outputs(result["prefix"])

assert tuple(result["branches"]) == d.STUDY_BRANCHES

for branch in d.STUDY_BRANCHES:
```

```
branch_result = result["branches"][branch]

d.validate_branch_outputs(
branch=branch,
branch_result=branch_result,
expected_rho=rho,
expected_T_window=config.T_window,
)

row = d.extract_scalar_row(
seed=seed,
seed_index=0,
rho=rho,
branch=branch,
branch_result=branch_result,
)

for key in d.PRIMARY_OUTCOMES:
assert d.is_finite_number(row[key]), (
branch,
key,
)

for key in d.SECONDARY_OUTCOMES:
assert d.is_finite_number(row[key]), (
branch,
key,
)

payload = d.flatten_result(
seed,
rho,
result,
)

assert payload
assert len(payload) == len(set(payload))

print(
```

```
"One-condition parameterized driver smoke passed:",
seed,
rho,
)
PY
```

Phase 8 — Mandatory 40-seed baseline-only run

Use a fresh root. Do not use --overwrite.

Bash

```
export BASELINE_ROOT="twindow_baseline_tests_20260810"

test ! -e "$BASELINE_ROOT"

set -o pipefail
time python3 "$DRIVER" \
--frozen-manifest "$FROZEN_MANIFEST" \
--frozen-npz "$FROZEN_NPZ" \
--frozen-scalar "$FROZEN_SCALAR" \
--baseline-only \
--run-root "$BASELINE_ROOT" \
2>&1 | tee twindow_baseline_only.log
```

Locate the one generated run directory:

Bash

```
mapfile -t BASELINE_DIRS < <(
find "$BASELINE_ROOT" \
-mindepth 1 \
-maxdepth 1 \
-type d \
-print
)

test "${#BASELINE_DIRS[@]}" -eq 1
```

```
export BASELINE_DIR="${BASELINE_DIRS[0]}"
echo "$BASELINE_DIR"
```

Phase 9 — Audit the baseline certificate

Bash

```
python3 - <<'PY'
import json
import os
from pathlib import Path

root = Path(os.environ["BASELINE_DIR"])

with open(
root / "baseline_equivalence_certificate.json",
encoding="utf-8",
) as handle:
certificate = json.load(handle)

with open(
root / "manifest.json",
encoding="utf-8",
) as handle:
manifest = json.load(handle)

expected_branches = [
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
]

assert certificate["status"] == "passed"
```

```
assert certificate["n_seeds"] == 40
assert len(certificate["seed_results"]) == 40
assert certificate["protocol"]["branches"] == expected_branches


for seed, result in certificate["seed_results"].items():
assert result["status"] == "passed", seed


failed_checks = [
check
for check, passed in result["checks"].items()
if not passed
]


assert not failed_checks, (
seed,
failed_checks[:20],
)


assert certificate["frozen_manifest_file_sha256"]
assert certificate["frozen_npz_file_sha256"]
assert certificate["frozen_scalar_csv_file_sha256"]
assert certificate["engine_source_sha256"]


assert manifest["execution_mode"] == "baseline_only"
assert manifest["baseline_certified"] is True
assert manifest["freezeable"] is False
assert (
manifest["completion"]["status"]
== "baseline_only_completed"
)
assert manifest["completion"]["study_grid_started"] is False


# Baseline-only must not have started the grid.
assert not list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*"
"_scalar_summary.csv"
)
)
assert not list(
```

root.glob(
"AIEF_1_4_Twindow_sensitivity_*"
"_branch_diagnostics.npz"
)
)

print("Baseline-only certificate audit passed.")
PY

The required baseline result is:

text

40/40 seeds passed
7/7 branches passed
all frozen prefix arrays exact
all historical matrix/trajectory fields exact
all 42 historical scalar fields exact
all rho=1 extended identities exact
all horizon metadata exact

Anything less is failure.
Phase 10 — Confirm source immutability

Bash

sha256sum -c twindow_pretest_source_sha256.txt

All three scripts must report OK.

If any source changed, the certificate no longer certifies the current stack. Restart from Phase 0.
Phase 11 — Launch the full regenerated grid

Use a different fresh root. The production run will repeat its mandatory baseline and then run all 200 (seed,ρ)(seed,ρ) conditions.

Bash

```
export PRODUCTION_ROOT="twindow_production_runs_20260810"

test ! -e "$PRODUCTION_ROOT"

set -o pipefail
time python3 "$DRIVER" \
--frozen-manifest "$FROZEN_MANIFEST" \
--frozen-npz "$FROZEN_NPZ" \
--frozen-scalar "$FROZEN_SCALAR" \
--run-root "$PRODUCTION_ROOT" \
2>&1 | tee twindow_full_grid.log
```

Locate the production run:

Bash

```
mapfile -t PRODUCTION_DIRS < <(
find "$PRODUCTION_ROOT" \
-mindepth 1 \
-maxdepth 1 \
-type d \
-print
)

test "${#PRODUCTION_DIRS[@]}" -eq 1

export RUN_DIR="${PRODUCTION_DIRS[0]}"
echo "$RUN_DIR"
```

Phase 12 — Audit the full run

Bash

```
python3 - <<'PY'
import hashlib
import json
import os
from pathlib import Path
```

```
import numpy as np
import pandas as pd


root = Path(os.environ["RUN_DIR"])


with open(root / "manifest.json", encoding="utf-8") as f:
manifest = json.load(f)


assert manifest["freezeable"] is True
assert (
manifest["baseline_equivalence"]["status"]
== "passed"
)


completion = manifest["completion"]


assert completion["status"] == "completed"
assert completion["n_conditions_requested"] == 200
assert completion["n_conditions_successful"] == 200
assert completion["n_conditions_failed"] == 0
assert completion["n_scalar_rows"] == 800
assert completion["n_missing_primary_outcomes"] == 0
assert completion["n_missing_secondary_outcomes"] == 0


status_path = root / "twindow_condition_status.csv"
status = pd.read_csv(
status_path,
dtype={"seed": str},
)


assert len(status) == 200
assert not status.duplicated(["seed", "rho"]).any()
assert (status["status"] == "success").all()


csv_files = list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*"
```

```
"_scalar_summary.csv"
)
)
npz_files = list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*"
"_branch_diagnostics.npz"
)
)

assert len(csv_files) == 1
assert len(npz_files) == 1

df = pd.read_csv(
csv_files[0],
dtype={"seed": str},
)

assert len(df) == 800
assert df["seed"].nunique() == 40
assert set(df["rho"].astype(float)) == {
0.5,
0.75,
1.0,
1.5,
2.0,
}
assert set(df["branch"]) == {
"continue",
"reset_ema",
"reset_B",
"future_shuffle",
}
assert not df.duplicated(
["seed", "rho", "branch"]
).any()

for column in [
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
"P_mean_late_fixed_physical",
```

```
"Weighted_similarity_late_normalized_EMA",
"Retention_contrast_late_normalized_EMA",
"P_mean_late_normalized_EMA",
]:
values = pd.to_numeric(
df[column],
errors="coerce",
)
assert np.isfinite(values).all(), column


def sha256(path):
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(
lambda: handle.read(1024 * 1024),
b"",
):
digest.update(block)
return digest.hexdigest()


hash_paths = {
"scalar_csv": csv_files[0],
"branch_npz": npz_files[0],
"condition_status_csv": status_path,
"baseline_certificate":
root / "baseline_equivalence_certificate.json",
"baseline_seed_results":
root / "baseline_equivalence_by_seed.json",
}


for label, path in hash_paths.items():
assert (
sha256(path)
== manifest["output_hashes"][label]
), label


print("Full regenerated grid audit passed.")
PY
```

Phase 13 — Run the analysis

Use the prespecified bootstrap seed and production bootstrap count:

Bash

```
set -o pipefail
time python3 "$ANALYSIS" \
"$RUN_DIR" \
--bootstrap-seed 20260808 \
--n-bootstrap 5000 \
2>&1 | tee twindow_analysis.log
```

Do not use --overwrite on the first analysis.
Phase 14 — Audit analysis output counts

Bash

```
python3 - <<'PY'
import json
import os
from pathlib import Path

import pandas as pd

analysis_dir = Path(os.environ["RUN_DIR"]) / "analysis"

expected_rows = {
"twindow_paired_regime_contrasts.csv": 960,
"twindow_paired_regime_contrasts_fixed_physical.csv": 480,
"twindow_paired_regime_contrasts_normalized_EMA.csv": 480,
"twindow_branch_interactions.csv": 2880,
"twindow_branch_interactions_fixed_physical.csv": 1440,
"twindow_branch_interactions_normalized_EMA.csv": 1440,
"twindow_cross_condition_topology.csv": 160,
"twindow_recovery_by_seed.csv": 3200,
"twindow_recovery_summary.csv": 80,
"twindow_recovery_interactions.csv": 24960,
"twindow_recovery_interactions_summary.csv": 624,
```

```
"twindow_regime_summary.csv": 12,
"twindow_branch_DID_summary.csv": 36,
"twindow_topology_summary.csv": 4,
"twindow_regime_secondary_summary.csv": 12,
"twindow_branch_DID_secondary_summary.csv": 36,
}


for name, expected in expected_rows.items():
path = analysis_dir / name
assert path.is_file(), name


observed = len(pd.read_csv(path))


assert observed == expected, (
name,
observed,
expected,
)


recovery = pd.read_csv(
analysis_dir / "twindow_recovery_by_seed.csv",
dtype={"seed": str},
)


recovery_sizes = recovery.groupby(
["rho", "branch", "horizon_system", "metric"]
).size()


assert len(recovery_sizes) == 80
assert (recovery_sizes == 40).all()


recovery_did = pd.read_csv(
analysis_dir / "twindow_recovery_interactions.csv",
dtype={"seed": str},
)


did_sizes = recovery_did.groupby(
[
"rho",
```

```
"branch",
"horizon_system",
"metric",
"outcome",
]
).size()

assert len(did_sizes) == 624
assert (did_sizes == 40).all()
assert set(recovery_did["branch"]) == {
"reset_ema",
"reset_B",
"future_shuffle",
}

with open(
analysis_dir / "analysis_manifest.json",
encoding="utf-8",
) as f:
manifest = json.load(f)

assert manifest["validation"]["status"] == "passed"
assert manifest["bootstrap"]["seed"] == 20260808
assert manifest["bootstrap"]["n_iterations"] == 5000

print("Analysis-output audit passed.")
PY
```

Phase 15 — Final source and artifact freeze

Confirm source files again:

Bash

```
sha256sum -c twindow_pretest_source_sha256.txt
```

Create external hash inventories so the inventory files do not hash themselves:

Bash

```
find "$RUN_DIR" \
-type f \
-print0 \
| sort -z \
| xargs -0 sha256sum \
> twindow_production_artifact_sha256.txt
```

Then:

freeze the production run directory;
freeze the analysis directory;
preserve all console logs;
preserve source and artifact hash inventories;
update the evidence-lineage manifest;
only then interpret the scientific results.

Inferential status

Keep these categories separate:

Exact baseline identities: finite replay against frozen artifacts.
Regime and branch contrasts: paired seed-level controlled evidence.
Recovery and topology results: regenerated finite-condition evidence.
General/asymptotic claims: remain open unless separately proved.

The 40 seed/graphs are the inferential units. Branches, edges, rho levels, horizons, and recovery metrics within a seed are paired measurements, not independent replicates.

This is the directory listed out for the project we are working on, this is on my system:

```
(deck@steamdeck SSRH)$ ls -R
.:
87dfe5798954 aief_1_4_observable_jacobian
AI_AGENT_HANDOFF_PROMPT.md aief_1_4_observable_jacobian.py
AIEF_1_3_analysis.py AIEF_1_4_OPERATOR_COCYCLE_CLOSURE_THEOREM_NOTE.md
aief_1_3_closure_diagnostics AIEF_1_4_OPERATOR_COCYCLE_THOUGHT_TREE.md
```

AIEF_1_3_closure_diagnostics.py aief_1_4_operator_compression
AIEF_1_3_closure_diagnostics_updated.py aief_1_4_operator_compression.py
AIEF_1_3_falsification.py AIEF_1_4_OPERATOR_THEORY_MANUSCRIPT.md
AIEF_1_3_falsification_updated.py aief_1_4_parameter_free_triangular_closure
AIEF_1_3_paired_statistics.py AIEF_1_4_parameter_free_triangular_closure.py
AIEF_1_3_paired_statistics_updated.py aief_1_4_phase_balance_source_dynamics
aief_1_4_active_boundary_analysis AIEF_1_4_phase_balance_source_dynamics.py
AIEF_1_4_active_boundary_analysis.py aief_1_4_phase_cancellation_audit
aief_1_4_actual_admissibility_branch aief_1_4_phase_source_decomposition
AIEF_1_4_actual_admissibility_branch_audit.py aief_1_4_phase_source_decomposition.py
aief_1_4_augmented_duhamel_feedback aief_1_4_phase_source_decomposition_v2.py
AIEF_1_4_augmented_duhamel_feedback_analysis.py aief_1_4_phase_source_locking
aief_1_4_augmented_spectrum AIEF_1_4_phase_source_locking_analysis.py
aief_1_4_augmented_spectrum.py aief_1_4_propagation_gap_geometric_control
aief_1_4_augmented_tangent_cocycle AIEF_1_4_propagation_gap_geometric_control.py
AIEF_1_4_augmented_tangent_cocycle_analysis.py aief_1_4_proposal_equalization_bridge
aief_1_4_biorthogonal_triangular_closure AIEF_1_4_proposal_equalization_bridge_analysis.py
AIEF_1_4_biorthogonal_triangular_closure.py
AIEF_1_4_PROPOSAL_EQUALIZATION_BRIDGE_INTERPRETATION.md
aief_1_4_channel_invariance aief_1_4_radial_observable_subsystem
AIEF_1_4_channel_invariance_analysis.py AIEF_1_4_radial_observable_subsystem_analysis.py
aief_1_4_cocycle_controls_coupled aief_1_4_resonant_tracking_angular
AIEF_1_4_cocycle_controls_coupled_comparison.py
AIEF_1_4_resonant_tracking_angular_analysis.py
aief_1_4_component_controller_audit aief_1_4_results_analysis
AIEF_1_4_component_controller_audit.py AIEF_1_4_results_analysis.py
AIEF_1_4_CONDITIONAL_PROPOSAL_EQUALIZATION_THEOREM.md
aief_1_4_scalar_spectral_gap_reduction
aief_1_4_controller_boundary_margin AIEF_1_4_scalar_spectral_gap_reduction.py
aief_1_4_controller_boundary_margin.py aief_1_4_slow_bundle_geometry
AIEF_1_4_CONTROLLER_FIBER_THEOREM_PACKAGE.md aief_1_4_slow_bundle_geometry.py
aief_1_4_coupling_lock_audit aief_1_4_slow_eigenvector_analysis
aief_1_4_coupling_lock_audit.py aief_1_4_slow_eigenvector_analysis.py
aief_1_4_defect_causal_ema_trace aief_1_4_slow_subspace_projection
AIEF_1_4_defect_causal_ema_trace.py aief_1_4_slow_subspace_projection.py
aief_1_4_defect_recurrence_analytic_mode aief_1_4_stationarity_perturbation
AIEF_1_4_defect_recurrence_analytic_mode.py
AIEF_1_4_stationarity_perturbation_analysis_updated.py
aief_1_4_duhamel_memory_alignment AIEF_1_4_superseded_claims.csv
AIEF_1_4_duhamel_memory_alignment_analysis.py aief_1_4_survival_outlier_spectral
aief_1_4_entropy_geometry AIEF_1_4_survival_outlier_spectral.py
AIEF_1_4_entropy_geometry_diagnostics.py aief_1_4_tangent_spectrum_switch

aief_1_4_exceptional_K3_transition AIEF_1_4_tangent_spectrum_switch_analysis.py
AIEF_1_4_exceptional_K3_transition_audit.py aief_1_4_trajectory_recovery
aief_1_4_exceptional_tangent AIEF_1_4_trajectory_recovery_equivalence.py
AIEF_1_4_exceptional_tangent_analysis.py aief_1_4_transverse_phase_jacobian
aief_1_4_existing_clique_invariant_subspaces aief_1_4_transverse_phase_jacobian.py
AIEF_1_4_existing_clique_invariant_subspaces.py AIEF_1_4_Twindow_sensitivity_analysis_v3.py
aief_1_4_face_spectrum AIEF_1_4_Twindow_sensitivity.py
AIEF_1_4_face_spectrum_analysis.py aief_1_4_two_channel_alignment_v4
aief_1_4_feedback_rank_memory_reduction aief_1_4_two_channel_alignment_v4.py
'aief_1_4_feedback_rank_memory_reduction 01' aief_1_4_two_channel_alignment_v5
AIEF_1_4_feedback_rank_memory_reduction.py aief_1_4_two_channel_alignment_v5.py
AIEF_1_4_FORWARD_PLAN.md AIEF_CODING_AGENT_PROMPT.md
AIEF_1_4_freeze_controller_fiber_theorem.py aief_experiment_engine_v1.py
aief_1_4_frozen_driver_cocycle AIEF_HISTORIC_UNFOLDING.md
AIEF_1_4_frozen_driver_cocycle_analysis.py check_001.py
aief_1_4_intertwining_controller check_01.py
'aief_1_4_intertwining_controller 01' check_02.py
'aief_1_4_intertwining_controller 02' check.py
AIEF_1_4_intertwining_controller_analysis.py diagnose_import.py
aief_1_4_k3_structure_scan gap_phase_scan_v9.py
aief_1_4_k3_structure_scan.py 'Older files'
aief_1_4_local_update_comparison paired_seed_analysis
AIEF_1_4_local_update_comparison.py Papers
aief_1_4_matched_phase_memory_ablation probe_01.py
AIEF_1_4_matched_phase_memory_ablation.py probe_02.py
AIEF_1_4_memory_ablation.py pycache
AIEF_1_4_memory_ablation_updated_07.py Results.pdf
AIEF_1_4_memory_ablation_updated.py runs
aief_1_4_multiseed_geometry runs_aief14_smoke
aief_1_4_multiseed_geometry.py 'Sci- stuff.txt'
aief_1_4_nested_operator_family test_output
AIEF_1_4_nested_operator_family_audit.py

./87dfe5798954:
AIEF_1_3_87dfe5798954_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_87dfe5798954_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz

baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./aief_1_3_closure_diagnostics:
baseline_persistence_by_seed.csv closure_report.txt matrix_scalar_consistency.csv
baseline_persistence_unique_values_by_seed.csv closure_status.json
projection_consistency_by_seed.csv
baseline_persistence_unique_values.csv cohort_seed_coverage.csv
projection_diagnostics_summary.csv
baseline_persistence_unique_values_pooled.csv cohort_seed_overlap.csv

./aief_1_4_active_boundary_analysis:
active_boundary_report.txt analysis_manifest.json coupled_epsilon_dt_summary.csv
active_stratified_comparisons.csv boundary_field_convergence_orders.csv hessian_eigenmodes.csv
active_stratified_orders_by_seed.csv boundary_field_finite_difference.csv
hessian_eigenspace_summary_by_seed.csv
active_stratified_order_summary.csv coupled_epsilon_dt_rows.csv hessian_eigenspace_summary.csv

./aief_1_4_actual_admissibility_branch:
actual_admissibility_branch_report.txt actual_branch_key_matrices.npz analysis_manifest.json
actual_branch_edge_trajectory.csv actual_branch_node_trajectory.csv branch_jacobian_validation.csv
actual_branch_events.csv actual_branch_step_summary.csv directional_derivative_validation.csv

./aief_1_4_augmented_duhamel_feedback:
analysis_manifest.json duhamel_vectors.npz tangent_ablation_summary.csv
augmented_duhamel_feedback_report.txt forcing_contribution_to_final.csv
tangent_ablation_trajectory.csv
duhamel_trajectory.csv forcing_time_bin_attribution.csv

./aief_1_4_augmented_spectrum:
analysis_manifest.json augmented_eigenvalues.csv augmented_spectrum_report.txt

./aief_1_4_augmented_tangent_cocycle:
analysis_manifest.json augmented_tangent_report.txt matched_horizon_amplitude_collapse.csv
augmented_tangent_comparison.csv augmented_tangent_summary.csv

./aief_1_4_biorthogonal_triangular_closure:
analysis_manifest.json biorthogonal_covector_trajectory.csv biorthogonal_rollout_summary.csv
biorthogonal_closure_arrays.npz biorthogonal_one_step_summary.csv
biorthogonal_rollout_trajectory.csv
biorthogonal_coordinate_summary.csv biorthogonal_one_step_trajectory.csv
biorthogonal_triangular_report.txt

./aief_1_4_channel_invariance:
analysis_manifest.json channel_multiplicities_by_seed.csv
geometry_spectrum_consistency_tolerant.csv
channel_invariance_by_seed.csv exceptional_channel_stages.csv modal_success_summary.json
channel_invariance_report.txt exceptional_transition_metadata.json
unique_direction_modal_success.csv

./aief_1_4_cocycle_controls_coupled:
analysis_manifest.json coupled_frozen_centered_comparison.csv stable_cocycle_spectra.csv
cocycle_control_matrices.npz coupled_frozen_centered_summary.csv
cocycle_controls_coupled_report.txt dense_sliding_window_amplification.csv

./aief_1_4_component_controller_audit:
analysis_manifest.json component_mechanism_audit.csv state_mechanism_summary.csv
component_controller_matrices.npz controller_residual_by_node.csv
component_controller_report.txt edge_locking_balance_audit.csv

./aief_1_4_controller_boundary_margin:

analysis_manifest.json controller_margin_intervals.csv controller_margin_report.txt
controller_margin_trajectory.csv

./aief_1_4_coupling_lock_audit:
analysis_manifest.json coupling_lock_intervals.csv coupling_lock_report.txt
coupling_lock_trajectory.csv

./aief_1_4_defect_causal_ema_trace:
analysis_manifest.json defect_causal_identity_checks.csv defect_causal_trace_trajectory.csv
defect_causal_ema_report.txt defect_causal_trace_intervals.csv

./aief_1_4_defect_recurrence_analytic_mode:
analysis_manifest.json analytic_defect_recurrence_arrays.npz defect_coefficient_trajectory.csv
analytic_defect_POD_comparison.csv analytic_defect_recurrence_report.txt
defect_recurrence_summary.csv

./aief_1_4_duhamel_memory_alignment:
analysis_manifest.json exact_crossover_and_angles.csv selected_ablation_states.csv
duhamel_mechanism_summary.csv forcing_alignment_and_final_influence.csv
duhamel_memory_alignment_report.txt recent_forcing_memory_curve.csv

./aief_1_4_entropy_geometry:
analysis_manifest.json interior_geometry_by_seed.csv intrinsic_distances_summary.csv
boundary_occupancy_by_seed.csv interior_geometry_summary.csv raw_edge_weights.csv
entropy_geometry_report.txt intrinsic_distances_by_seed.csv raw_node_margins.csv

./aief_1_4_exceptional_K3_transition:
analysis_manifest.json exceptional_K3_step_summary.csv
boundary_tolerance_switch_sensitivity.csv exceptional_K3_transition_report.txt
exceptional_K3_edge_trajectory.csv exceptional_K3_transition_window_edges.csv
exceptional_K3_key_matrices.npz exceptional_K3_transition_window_nodes.csv
exceptional_K3_node_trajectory.csv exceptional_K3_transition_window_summary.csv

./aief_1_4_exceptional_tangent:
analysis_manifest.json corrected_tangent_trajectories.csv exceptional_seed_switch_matrices.npz
corrected_tangent_by_seed.csv exceptional_seed_summary.json exceptional_seed_timeseries.csv
corrected_tangent_summary.csv exceptional_seed_switches.csv exceptional_tangent_report.txt

./aief_1_4_existing_clique_invariant_subspaces:

analysis_manifest.json clique_invariant_subspace_by_component.csv
clique_invariant_subspace_report.txt
clique_fiber_direction_checks.csv clique_invariant_subspace_matrices.npz

./aief_1_4_face_spectrum:
active_face_fingerprints.csv filtered_anomalous_directions.csv
modal_trajectory_prediction_summary.csv
analysis_manifest.json fingerprint_spectrum_consistency.csv tangent_direction_summary.csv
direction_agreement_summary.json jacobian_direction_diagnostics.csv tangent_jacobian_matrices.npz
exceptional_stage_spectra.csv jacobian_eigenvalues.csv tangent_rate_face_crosstab_rows.csv
exceptional_transition_summary.json jacobian_step_sensitivity.csv
tangent_rate_face_crosstab_summary.csv
face_spectrum_report.txt modal_trajectory_predictions.csv

./aief_1_4_feedback_rank_memory_reduction:
analysis_manifest.json late_scalar_transverse_summary.csv propagated_forcing_rank_by_target.csv
feedback_rank_memory_report.txt memory_stationarity_summary.csv target_time_memory_curves.csv
instantaneous_feedback_rank.csv pod_first_mode_stability_summary.csv target_time_memory_fits.csv
late_channel_scalar_projection.csv pod_rank_by_interval.csv

'./aief_1_4_feedback_rank_memory_reduction 01':
analysis_manifest.json late_channel_scalar_projection.csv target_time_memory_curves.csv
feedback_rank_memory_report.txt pod_rank_by_interval.csv target_time_memory_fits.csv
instantaneous_feedback_rank.csv propagated_forcing_rank_by_target.csv

./aief_1_4_frozen_driver_cocycle:
analysis_manifest.json frozen_cocycle_nonlinear_summary.csv frozen_cocycle_step_diagnostics.csv
frozen_cocycle_matrices.npz frozen_cocycle_propagator.csv frozen_driver_cocycle_report.txt
frozen_cocycle_nonlinear_comparison.csv frozen_cocycle_sliding_windows.csv

./aief_1_4_intertwining_controller:
analysis_manifest.json exceptional_channel_stages.csv modal_success_summary.json
channel_invariance_by_seed.csv exceptional_transition_metadata.json
node_controller_equilibrium_by_seed.csv
channel_invariance_report.txt geometry_spectrum_consistency_tolerant.csv
unique_direction_modal_success.csv
channel_multiplicities_by_seed.csv intertwining_controller_matrices.npz

'./aief_1_4_intertwining_controller 01':
analysis_manifest.json exceptional_channel_stages.csv modal_success_summary.json

channel_invariance_by_seed.csv exceptional_transition_metadata.json
node_controller_equilibrium_by_seed.csv
channel_invariance_report.txt geometry_spectrum_consistency_tolerant.csv
unique_direction_modal_success.csv
channel_multiplicities_by_seed.csv intertwining_controller_matrices.npz

'./aief_1_4_intertwining_controller 02':
analysis_manifest.json exceptional_channel_stages.csv modal_success_summary.json
channel_invariance_by_seed.csv exceptional_transition_metadata.json
node_controller_equilibrium_by_seed.csv
channel_invariance_report.txt geometry_spectrum_consistency_tolerant.csv
unique_direction_modal_success.csv
channel_multiplicities_by_seed.csv intertwining_controller_matrices.npz

./aief_1_4_k3_structure_scan:
analysis_manifest.json k3_prevalence.csv k3_prevalence_report.txt

./aief_1_4_local_update_comparison:
analysis_manifest.json one_step_comparison_summary.csv one_step_methods_by_seed.csv
local_update_comparison_report.txt one_step_convergence_orders_by_seed.csv
one_step_method_summary.csv
one_step_comparisons_by_seed.csv one_step_convergence_order_summary.csv

./aief_1_4_matched_phase_memory_ablation:
analysis_manifest.json frozen_anchor_validation.csv matched_initial_state_audit.csv

./aief_1_4_multiseed_geometry:
k3_structure_scan.csv

./aief_1_4_nested_operator_family:
analysis_manifest.json nested_family_decay_fits.csv nested_family_key_times.csv
nested_family_trajectory.csv
nested_family_arrays.npz nested_family_defect_pod.csv nested_family_parameters.csv
nested_operator_family_report.txt

./aief_1_4_observable_jacobian:
analysis_manifest.json eigenvector_decomposition.csv observable_jacobian.csv
observable_jacobian_report.txt

./aief_1_4_operator_compression:
analysis_manifest.json operator_compression.csv operator_compression_report.txt

./aief_1_4_parameter_free_triangular_closure:
analysis_manifest.json triangular_gap_summary.csv triangular_rollout_summary.csv
parameter_free_triangular_report.txt triangular_one_step_summary.csv
triangular_rollout_trajectory.csv
triangular_closure_arrays.npz triangular_one_step_trajectory.csv

./aief_1_4_phase_balance_source_dynamics:
analysis_manifest.json phase_balance_interval_summary.csv phase_balance_trajectory.csv
phase_balance_identity_checks.csv phase_balance_source_report.txt

./aief_1_4_phase_cancellation_audit:
analysis_manifest.json phase_cancellation_report.txt
phase_cancellation_intervals.csv phase_cancellation_trajectory.csv

./aief_1_4_phase_source_decomposition:
analysis_manifest.json source_decomposition_report.txt
source_decomposition_intervals.csv source_decomposition_trajectory.csv

./aief_1_4_phase_source_locking:
analysis_manifest.json phase_source_locking_intervals.csv phase_source_locking_trajectory.csv
phase_source_locking_identity_checks.csv phase_source_locking_report.txt

./aief_1_4_propagation_gap_geometric_control:
analysis_manifest.json forcing_state_identifiability.csv propagated_contribution_envelopes.csv
direct_propagation_gap_tests.csv geometric_vs_POD_direction_control.csv
propagation_gap_geometric_report.txt

./aief_1_4_proposal_equalization_bridge:
analysis_manifest.json proposal_equalization_report.txt
proposal_equalization_identity_summary.csv proposal_equalization_trajectory.csv
proposal_equalization_interval_summary.csv

./aief_1_4_radial_observable_subsystem:
analysis_manifest.json radial_observable_identity_checks.csv radial_observable_report.txt
observable_subsystem_fits.csv radial_observable_interval_summary.csv
radial_observable_trajectory.csv

./aief_1_4_resonant_tracking_angular:
alignment_component_asymptotics.csv resonant_tracking_angular_report.txt
resonant_tracking_trajectory.csv
analysis_manifest.json resonant_tracking_fit_summary.csv
angular_radius_summary.csv resonant_tracking_identity_checks.csv

./aief_1_4_results_analysis:
AIEF_1_4_analysis_report.txt density_trajectories.png paired_branch_effects.csv
analysis_manifest.json frobenius_distances_by_seed.csv projection_summary.csv
B_EMA_interactions_by_seed.csv frobenius_distance_summary.csv similarity_trajectories.png
B_EMA_interactions.csv paired_branch_effects_by_seed.csv trajectory_summary.csv

./aief_1_4_scalar_spectral_gap_reduction:
analysis_manifest.json scalar_reduction_arrays.npz two_state_model_summary.csv
scalar_projection_summary.csv scalar_spectral_gap_report.txt two_state_rollout_validation.csv
scalar_projection_trajectory.csv target_spectral_gap_tests.csv

./aief_1_4_slow_bundle_geometry:
analysis_manifest.json compressed_operator_entries.csv slow_bundle_geometry_report.txt

./aief_1_4_slow_eigenvector_analysis:
analysis_manifest.json participation_factors.csv slow_eigenvector_report.txt
eigenvector_overlaps.csv singular_values.csv slow_eigenvectors.csv

./aief_1_4_slow_subspace_projection:
analysis_manifest.json observable_projections.csv variance_fractions.csv
eigenvector_decomposition.csv slow_subspace_report.txt

./aief_1_4_stationarity_perturbation:
analysis_manifest.json coupled_perturbation_trajectories.csv mature_stationarity_by_seed.csv
coupled_perturbations_by_seed.csv frozen_fast_perturbations_by_seed.csv
mature_stationarity_timeseries.csv
coupled_perturbation_summary.csv frozen_fast_perturbation_summary.csv
stationarity_perturbation_report.txt

./aief_1_4_survival_outlier_spectral:
analysis_manifest.json paired_recovery_contrasts.csv resetB_residual_svd.csv
feature_recovery_associations.csv recovery_by_seed.csv spectral_metrics_split_late_final.csv

KM_curves.csv recovery_summary.csv split_state_features.csv
KM_density90_recovery.png resetB_residual_first_mode_edges.csv
survival_outlier_spectral_report.txt
KM_similarity90_recovery.png resetB_residual_localization_by_seed.csv

./aief_1_4_tangent_spectrum_switch:
analysis_manifest.json exceptional_switch_node_window.csv tangent_jacobian_matrices.npz
anomalous_tangent_directions.csv exceptional_switch_summary.json tangent_jacobian_summary.csv
direction_agreement_summary.json tangent_direction_summary.csv
tangent_spectrum_switch_report.txt
exceptional_switch_events.csv tangent_jacobian_direction_details.csv tangent_transient_growth.csv

./aief_1_4_trajectory_recovery:
analysis_manifest.json similarity_recovery_envelope.png trajectory_metrics_summary.csv
density_recovery_envelope.png trajectory_envelopes.csv trajectory_recovery_report.txt
equivalence_tests.csv trajectory_metrics_by_seed.csv

./aief_1_4_transverse_phase_jacobian:
analysis_manifest.json transverse_jacobian_report.txt
transverse_jacobian_intervals.csv transverse_jacobian_trajectory.csv

./aief_1_4_two_channel_alignment_v4:
analysis_manifest.json two_channel_identity_checks.csv two_channel_trajectory.csv
two_channel_decomposition_report.txt two_channel_summary.csv

./aief_1_4_two_channel_alignment_v5:
analysis_manifest.json two_channel_identity_checks.csv two_channel_trajectory.csv
two_channel_decomposition_report.txt two_channel_summary.csv

'./Older files':
AIEF_1_4_augmented_duhamel_feedback_analysis_01.py aief_1_4_multiseed_geometry_01.py
AIEF_1_4_augmented_duhamel_feedback_analysis.py aief_1_4_multiseed_geometry_02.py
aief_1_4_augmented_spectrum_02.py aief_1_4_multiseed_geometry_03.py
aief_1_4_augmented_spectrum.py aief_1_4_multiseed_geometry_05.py
AIEF_1_4_augmented_tangent_cocycle_analysis.py aief_1_4_multiseed_geometry_08.py
aief_1_4_coupling_lock_audit.py aief_1_4_multiseed_geometry_10.py
AIEF_1_4_duhamel_memory_alignment_analysis.py aief_1_4_multiseed_geometry_11.py
aief_1_4_face_spectrum aief_1_4_multiseed_geometry_1.py
AIEF_1_4_face_spectrum_analysis.py AIEF_1_4_phase_source_locking_analysis.py
AIEF_1_4_feedback_rank_memory_reduction.py AIEF_1_4_resonant_tracking_angular_analysis.py

AIEF_1_4_freeze_controller_fiber_theorem.py aief_1_4_slow_bundle_geometry_01.py
AIEF_1_4_matched_phase_memory_ablation_01.py aief_1_4_slow_subspace_projection_01.py
AIEF_1_4_matched_phase_memory_ablation_07.py aief_1_4_slow_subspace_projection.py
AIEF_1_4_matched_phase_memory_ablation.py aief_1_4_two_channel_alignment_v4.py
AIEF_1_4_memory_ablation_updated_01.py aief_1_4_two_channel_alignment_v5_01.py
AIEF_1_4_memory_ablation_updated.py aief_1_4_two_channel_alignment_v5.py

'./Older files/aief_1_4_face_spectrum':
active_face_fingerprints.csv face_spectrum_report.txt jacobian_step_sensitivity.csv
analysis_manifest.json filtered_anomalous_directions.csv tangent_direction_summary.csv
direction_agreement_summary.json fingerprint_spectrum_consistency.csv
tangent_jacobian_matrices.npz
exceptional_stage_spectra.csv jacobian_direction_diagnostics.csv tangent_rate_face_crosstab_rows.csv
exceptional_transition_summary.json jacobian_eigenvalues.csv
tangent_rate_face_crosstab_summary.csv

./paired_seed_analysis:
analysis_manifest.json paired_cohort_heterogeneity.csv paired_report.txt
paired_binary_effects.csv paired_effects.csv paired_seed_deltas.csv

./Papers:
2804.05652v1.pdf
'H-Atom_Paper 3 Geometric Radius Scaling Law 19.Jul.2026_v1.0.pdf'
'The Holland Atom_Paper_1_30.May.2026_v1.pdf'
'The Holland Atom_Paper 2 Radius Variation 23.Jun.2026_v1.0.pdf'

./pycache:
AIEF_1_4_actual_admissibility_branch_audit.cpython-313.pyc
AIEF_1_4_augmented_tangent_cocycle_analysis.cpython-313.pyc
AIEF_1_4_component_controller_audit.cpython-313.pyc
AIEF_1_4_exceptional_K3_transition_audit.cpython-313.pyc
AIEF_1_4_exceptional_tangent_analysis.cpython-313.pyc
AIEF_1_4_face_spectrum_analysis.cpython-313.pyc
AIEF_1_4_frozen_driver_cocycle_analysis.cpython-313.pyc
AIEF_1_4_matched_phase_memory_ablation.cpython-313.pyc
AIEF_1_4_memory_ablation_updated.cpython-313.pyc
AIEF_1_4_nested_operator_family_audit.cpython-313.pyc
AIEF_1_4_proposal_equalization_bridge_analysis.cpython-313.pyc
AIEF_1_4_stationarity_perturbation_analysis_updated.cpython-313.pyc
AIEF_1_4_tangent_spectrum_switch_analysis.cpython-313.pyc
AIEF_1_4_trajectory_recovery_equivalence.cpython-313.pyc

./runs:
0ddf42b3e20a 2e2843f5e26c 5a338105785a 73dca37cf93f 92dd4ddbdba1 a3dcc58f4939 c3da1ac707bf
123b2cadbb0d 46fb82c784ef 5a9f093a2fe0 87dfe5798954 9fb83552f0ce a937d5e96efb fccebfc14855

./runs/0ddf42b3e20a:
AIEF_1_3_0ddf42b3e20a_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_0ddf42b3e20a_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./runs/123b2cadbb0d:
AIEF_1_3_123b2cadbb0d_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_123b2cadbb0d_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz

baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./runs/2e2843f5e26c:
AIEF_1_3_2e2843f5e26c_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_2e2843f5e26c_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz

baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./runs/46fb82c784ef:
AIEF_1_3_46fb82c784ef_edge_diagnostics.npz no_delay_seed_11042.npz no_gap_seed_17042.npz
no_mu_seed_42.npz
AIEF_1_3_46fb82c784ef_scalar_summary.csv no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_analysis_results.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz

baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz

./runs/5a338105785a:
AIEF_1_3_5a338105785a_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_5a338105785a_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./runs/5a9f093a2fe0:
AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz seed_16952466944611321358.npz
seed_3952397966055989215.npz
AIEF_1_4_5a9f093a2fe0_scalar_summary.csv seed_172180395821707403.npz
seed_4040285944958847150.npz
manifest.json seed_17279552413280415429.npz seed_4219039243791140697.npz

seed_10431384713483787270.npz seed_17521310920850592782.npz
seed_4619122171158768524.npz
seed_10856823803646276806.npz seed_17715532056777442089.npz
seed_4932511216876650745.npz
seed_11242426753702356865.npz seed_17883847597205629106.npz
seed_5213047409853278064.npz
seed_12123519807951751650.npz seed_18127452951968972828.npz
seed_5415544643585892346.npz
seed_1230287949579537403.npz seed_1888415619847841128.npz seed_6023008189364820510.npz
seed_12851640484293199234.npz seed_1918007468868772336.npz seed_6108867655028980287.npz
seed_14040336450255043598.npz seed_2461295749352584108.npz seed_6942591650975756121.npz
seed_14924267685654216746.npz seed_283404770198602602.npz seed_9036321132460399255.npz
seed_15403081841477312463.npz seed_305986450614148690.npz seed_9082055256280075172.npz
seed_15853699719446171583.npz seed_3258463328937640595.npz seed_9189988769602851417.npz
seed_16329425431442249417.npz seed_3582616062465030267.npz
seed_16627818730624180313.npz seed_3704716633526084363.npz

./runs/73dca37cf93f:
AIEF_1_3_73dca37cf93f_edge_diagnostics.npz no_gap_seed_4042692546438370928.npz
AIEF_1_3_73dca37cf93f_scalar_summary.csv no_gap_seed_4274086224959828596.npz
AIEF_1_3_analysis_results.csv no_gap_seed_4365343787365008077.npz
baseline_seed_11078602408408019102.npz no_gap_seed_4723126346505719299.npz
baseline_seed_11197595738654224886.npz no_gap_seed_4866117689794208263.npz
baseline_seed_1246923751594727930.npz no_gap_seed_5014145038466735176.npz
baseline_seed_13104999914199871252.npz no_gap_seed_5304325715998943684.npz
baseline_seed_13168160597277742379.npz no_gap_seed_536452804929066902.npz
baseline_seed_1316891844312232182.npz no_gap_seed_5727400265110223831.npz
baseline_seed_14101137954994639910.npz no_gap_seed_577339501563627643.npz
baseline_seed_14247790844233709566.npz no_gap_seed_5776146606078111419.npz
baseline_seed_14413360560776613994.npz no_gap_seed_5996580515567889036.npz
baseline_seed_14498610575954930932.npz no_gap_seed_6384490036851063518.npz
baseline_seed_14668531591501948822.npz no_gap_seed_6597351216039696310.npz
baseline_seed_14743978782784222765.npz no_gap_seed_6810304725465717906.npz
baseline_seed_15011858390678511204.npz no_gap_seed_6833618658979281150.npz
baseline_seed_159500863463728126.npz no_gap_seed_7550509291629200394.npz
baseline_seed_16649945166774720395.npz no_gap_seed_7706845139332910089.npz
baseline_seed_1698991527437907300.npz no_gap_seed_7938073305027935059.npz
baseline_seed_17587848749391215576.npz no_gap_seed_8063201614575931673.npz
baseline_seed_17852680212380503812.npz no_gap_seed_8103184518762191864.npz
baseline_seed_18195599253746093149.npz no_gap_seed_8264568941032381199.npz
baseline_seed_18405604921195991417.npz no_gap_seed_8526943454688901591.npz
baseline_seed_2799856556347805964.npz no_gap_seed_9127451059220389752.npz

baseline_seed_2841400781150679534.npz no_gap_seed_9156045106123447864.npz
baseline_seed_3050482280156901559.npz no_gap_seed_9198312127684667017.npz
baseline_seed_4042692546438370928.npz no_gap_seed_956874784835626302.npz
baseline_seed_4274086224959828596.npz no_mu_seed_11078602408408019102.npz
baseline_seed_4365343787365008077.npz no_mu_seed_11197595738654224886.npz
baseline_seed_4723126346505719299.npz no_mu_seed_1246923751594727930.npz
baseline_seed_4866117689794208263.npz no_mu_seed_13104999914199871252.npz
baseline_seed_5014145038466735176.npz no_mu_seed_13168160597277742379.npz
baseline_seed_5304325715998943684.npz no_mu_seed_1316891844312232182.npz
baseline_seed_536452804929066902.npz no_mu_seed_14101137954994639910.npz
baseline_seed_5727400265110223831.npz no_mu_seed_14247790844233709566.npz
baseline_seed_577339501563627643.npz no_mu_seed_14413360560776613994.npz
baseline_seed_5776146606078111419.npz no_mu_seed_14498610575954930932.npz
baseline_seed_5996580515567889036.npz no_mu_seed_14668531591501948822.npz
baseline_seed_6384490036851063518.npz no_mu_seed_14743978782784222765.npz
baseline_seed_6597351216039696310.npz no_mu_seed_15011858390678511204.npz
baseline_seed_6810304725465717906.npz no_mu_seed_159500863463728126.npz
baseline_seed_6833618658979281150.npz no_mu_seed_16649945166774720395.npz
baseline_seed_7550509291629200394.npz no_mu_seed_1698991527437907300.npz
baseline_seed_7706845139332910089.npz no_mu_seed_17587848749391215576.npz
baseline_seed_7938073305027935059.npz no_mu_seed_17852680212380503812.npz
baseline_seed_8063201614575931673.npz no_mu_seed_18195599253746093149.npz
baseline_seed_8103184518762191864.npz no_mu_seed_18405604921195991417.npz
baseline_seed_8264568941032381199.npz no_mu_seed_2799856556347805964.npz
baseline_seed_8526943454688901591.npz no_mu_seed_2841400781150679534.npz
baseline_seed_9127451059220389752.npz no_mu_seed_3050482280156901559.npz
baseline_seed_9156045106123447864.npz no_mu_seed_4042692546438370928.npz
baseline_seed_9198312127684667017.npz no_mu_seed_4274086224959828596.npz
baseline_seed_956874784835626302.npz no_mu_seed_4365343787365008077.npz
manifest.json no_mu_seed_4723126346505719299.npz
no_delay_seed_11078602408408019102.npz no_mu_seed_4866117689794208263.npz
no_delay_seed_11197595738654224886.npz no_mu_seed_5014145038466735176.npz
no_delay_seed_1246923751594727930.npz no_mu_seed_5304325715998943684.npz
no_delay_seed_13104999914199871252.npz no_mu_seed_536452804929066902.npz
no_delay_seed_13168160597277742379.npz no_mu_seed_5727400265110223831.npz
no_delay_seed_1316891844312232182.npz no_mu_seed_577339501563627643.npz
no_delay_seed_14101137954994639910.npz no_mu_seed_5776146606078111419.npz
no_delay_seed_14247790844233709566.npz no_mu_seed_5996580515567889036.npz
no_delay_seed_14413360560776613994.npz no_mu_seed_6384490036851063518.npz
no_delay_seed_14498610575954930932.npz no_mu_seed_6597351216039696310.npz
no_delay_seed_14668531591501948822.npz no_mu_seed_6810304725465717906.npz
no_delay_seed_14743978782784222765.npz no_mu_seed_6833618658979281150.npz

no_delay_seed_15011858390678511204.npz no_mu_seed_7550509291629200394.npz
no_delay_seed_159500863463728126.npz no_mu_seed_7706845139332910089.npz
no_delay_seed_16649945166774720395.npz no_mu_seed_7938073305027935059.npz
no_delay_seed_1698991527437907300.npz no_mu_seed_8063201614575931673.npz
no_delay_seed_17587848749391215576.npz no_mu_seed_8103184518762191864.npz
no_delay_seed_17852680212380503812.npz no_mu_seed_8264568941032381199.npz
no_delay_seed_18195599253746093149.npz no_mu_seed_8526943454688901591.npz
no_delay_seed_18405604921195991417.npz no_mu_seed_9127451059220389752.npz
no_delay_seed_2799856556347805964.npz no_mu_seed_9156045106123447864.npz
no_delay_seed_2841400781150679534.npz no_mu_seed_9198312127684667017.npz
no_delay_seed_3050482280156901559.npz no_mu_seed_956874784835626302.npz
no_delay_seed_4042692546438370928.npz shuffle_R_seed_11078602408408019102.npz
no_delay_seed_4274086224959828596.npz shuffle_R_seed_11197595738654224886.npz
no_delay_seed_4365343787365008077.npz shuffle_R_seed_1246923751594727930.npz
no_delay_seed_4723126346505719299.npz shuffle_R_seed_13104999914199871252.npz
no_delay_seed_4866117689794208263.npz shuffle_R_seed_13168160597277742379.npz
no_delay_seed_5014145038466735176.npz shuffle_R_seed_1316891844312232182.npz
no_delay_seed_5304325715998943684.npz shuffle_R_seed_14101137954994639910.npz
no_delay_seed_536452804929066902.npz shuffle_R_seed_14247790844233709566.npz
no_delay_seed_5727400265110223831.npz shuffle_R_seed_14413360560776613994.npz
no_delay_seed_577339501563627643.npz shuffle_R_seed_14498610575954930932.npz
no_delay_seed_5776146606078111419.npz shuffle_R_seed_14668531591501948822.npz
no_delay_seed_5996580515567889036.npz shuffle_R_seed_14743978782784222765.npz
no_delay_seed_6384490036851063518.npz shuffle_R_seed_15011858390678511204.npz
no_delay_seed_6597351216039696310.npz shuffle_R_seed_159500863463728126.npz
no_delay_seed_6810304725465717906.npz shuffle_R_seed_16649945166774720395.npz
no_delay_seed_6833618658979281150.npz shuffle_R_seed_1698991527437907300.npz
no_delay_seed_7550509291629200394.npz shuffle_R_seed_17587848749391215576.npz
no_delay_seed_7706845139332910089.npz shuffle_R_seed_17852680212380503812.npz
no_delay_seed_7938073305027935059.npz shuffle_R_seed_18195599253746093149.npz
no_delay_seed_8063201614575931673.npz shuffle_R_seed_18405604921195991417.npz
no_delay_seed_8103184518762191864.npz shuffle_R_seed_2799856556347805964.npz
no_delay_seed_8264568941032381199.npz shuffle_R_seed_2841400781150679534.npz
no_delay_seed_8526943454688901591.npz shuffle_R_seed_3050482280156901559.npz
no_delay_seed_9127451059220389752.npz shuffle_R_seed_4042692546438370928.npz
no_delay_seed_9156045106123447864.npz shuffle_R_seed_4274086224959828596.npz
no_delay_seed_9198312127684667017.npz shuffle_R_seed_4365343787365008077.npz
no_delay_seed_956874784835626302.npz shuffle_R_seed_4723126346505719299.npz
no_gap_seed_11078602408408019102.npz shuffle_R_seed_4866117689794208263.npz
no_gap_seed_11197595738654224886.npz shuffle_R_seed_5014145038466735176.npz
no_gap_seed_1246923751594727930.npz shuffle_R_seed_5304325715998943684.npz
no_gap_seed_13104999914199871252.npz shuffle_R_seed_536452804929066902.npz

no_gap_seed_13168160597277742379.npz shuffle_R_seed_5727400265110223831.npz
no_gap_seed_13168918443122321 82.npz shuffle_R_seed_577339501563627643.npz
no_gap_seed_14101137954994639910.npz shuffle_R_seed_5776146606078111419.npz
no_gap_seed_14247790844233709566.npz shuffle_R_seed_5996580515567889036.npz
no_gap_seed_14413360560776613994.npz shuffle_R_seed_6384490036851063518.npz
no_gap_seed_14498610575954930932.npz shuffle_R_seed_6597351216039696310.npz
no_gap_seed_14668531591501948822.npz shuffle_R_seed_6810304725465717906.npz
no_gap_seed_14743978782784222765.npz shuffle_R_seed_6833618658979281150.npz
no_gap_seed_15011858390678511204.npz shuffle_R_seed_7550509291629200394.npz
no_gap_seed_159500863463728126.npz shuffle_R_seed_7706845139332910089.npz
no_gap_seed_16649945166774720395.npz shuffle_R_seed_7938073305027935059.npz
no_gap_seed_16989915274379073 00.npz shuffle_R_seed_8063201614575931673.npz
no_gap_seed_17587848749391215576.npz shuffle_R_seed_8103184518762191864.npz
no_gap_seed_17852680212380503812.npz shuffle_R_seed_8264568941032381199.npz
no_gap_seed_18195599253746093149.npz shuffle_R_seed_8526943454688901591.npz
no_gap_seed_18405604921195991417.npz shuffle_R_seed_9127451059220389752.npz
no_gap_seed_2799856556347805964.npz shuffle_R_seed_9156045106123447864.npz
no_gap_seed_2841400781150679534.npz shuffle_R_seed_9198312127684667017.npz
no_gap_seed_3050482280156901559.npz shuffle_R_seed_956874784835626302.npz

./runs/87dfe5798954:
AIEF_1_3_87dfe5798954_edge_diagnostics.npz no_delay_seed_11042.npz no_gap_seed_17042.npz
no_mu_seed_42.npz
AIEF_1_3_87dfe5798954_scalar_summary.csv no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_analysis_results.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz

baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz

./runs/92dd4ddbdba1:
AIEF_1_3_92dd4ddbdba1_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_92dd4ddbdba1_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./runs/9fb83552f0ce:

AIEF_1_3_9fb83552f0ce_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_9fb83552f0ce_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./runs/a3dcc58f4939:
AIEF_1_3_a3dcc58f4939_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_a3dcc58f4939_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz

baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz
baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./runs/a937d5e96efb:
baseline_seed_12232032881337366345.npz baseline_seed_13443745119513809312.npz
baseline_seed_9723627428708784598.npz
baseline_seed_12681060683294206227.npz baseline_seed_15655786368575661155.npz manifest.json
baseline_seed_13048312801923903802.npz baseline_seed_7157263871600225889.npz

./runs/c3da1ac707bf:
AIEF_1_3_analysis_results.csv no_gap_seed_15696189914383701419.npz
AIEF_1_3_c3da1ac707bf_edge_diagnostics.npz no_gap_seed_16004921505170220512.npz
AIEF_1_3_c3da1ac707bf_scalar_summary.csv no_gap_seed_16165962154861038927.npz
baseline_seed_10138059242446602359.npz no_gap_seed_16301226009792754359.npz
baseline_seed_10162605287015011567.npz no_gap_seed_17260448256843220567.npz
baseline_seed_10280671141729910962.npz no_gap_seed_17619579101733829571.npz
baseline_seed_10414151041756256180.npz no_gap_seed_17629994763630970995.npz
baseline_seed_10460494971182033875.npz no_gap_seed_17813068995467247720.npz
baseline_seed_10500527434474296483.npz no_gap_seed_17942362857188767334.npz
baseline_seed_10619621485132495208.npz no_gap_seed_18120325990998516096.npz
baseline_seed_10714358886600929 17.npz no_gap_seed_3464540282706013453.npz
baseline_seed_11552247507883116273.npz no_gap_seed_3867900559517386202.npz
baseline_seed_11699415664738731472.npz no_gap_seed_4004306315412470776.npz
baseline_seed_11760291472930266701.npz no_gap_seed_4026263611424279724.npz
baseline_seed_12101127255601305672.npz no_gap_seed_4213031461048503485.npz
baseline_seed_12423041489110211363.npz no_gap_seed_4225958022653391162.npz

baseline_seed_12672897826997023262.npz no_gap_seed_4297606350873079899.npz
baseline_seed_12828118411696793387.npz no_gap_seed_4773450812113592330.npz
baseline_seed_12932465548281524219.npz no_gap_seed_4884909319511143462.npz
baseline_seed_12990827071890121233.npz no_gap_seed_4989123116796778357.npz
baseline_seed_13011308780887378248.npz no_gap_seed_6799375687132975133.npz
baseline_seed_13144254950108472985.npz no_gap_seed_6799580967308193508.npz
baseline_seed_13475711373734757509.npz no_gap_seed_6847020926293287670.npz
baseline_seed_13877067520498498525.npz no_gap_seed_7443702551313363331.npz
baseline_seed_14789669151170384611.npz no_gap_seed_7518080831024782954.npz
baseline_seed_15538769139318490183.npz no_gap_seed_7568763936099751332.npz
baseline_seed_15696189914383701419.npz no_gap_seed_9166811211305776210.npz
baseline_seed_16004921505170220512.npz no_mu_seed_10138059242446602359.npz
baseline_seed_16165962154861038927.npz no_mu_seed_10162605287015011567.npz
baseline_seed_16301226009792754359.npz no_mu_seed_10280671141729910962.npz
baseline_seed_17260448256843220567.npz no_mu_seed_10414151041756256180.npz
baseline_seed_17619579101733829571.npz no_mu_seed_10460494971182033875.npz
baseline_seed_17629994763630970995.npz no_mu_seed_10500527434474296483.npz
baseline_seed_17813068995467247720.npz no_mu_seed_10619621485132495208.npz
baseline_seed_17942362857188767334.npz no_mu_seed_1071435888660092917.npz
baseline_seed_18120325990998516096.npz no_mu_seed_11552247507883116273.npz
baseline_seed_3464540282706013453.npz no_mu_seed_11699415664738731472.npz
baseline_seed_3867900559517386202.npz no_mu_seed_11760291472930266701.npz
baseline_seed_4004306315412470776.npz no_mu_seed_12101127255601305672.npz
baseline_seed_4026263611424279724.npz no_mu_seed_12423041489110211363.npz
baseline_seed_4213031461048503485.npz no_mu_seed_12672897826997023262.npz
baseline_seed_4225958022653391162.npz no_mu_seed_12828118411696793387.npz
baseline_seed_4297606350873079899.npz no_mu_seed_12932465548281524219.npz
baseline_seed_4773450812113592330.npz no_mu_seed_12990827071890121233.npz
baseline_seed_4884909319511143462.npz no_mu_seed_13011308780887378248.npz
baseline_seed_4989123116796778357.npz no_mu_seed_13144254950108472985.npz
baseline_seed_6799375687132975133.npz no_mu_seed_13475711373734757509.npz
baseline_seed_6799580967308193508.npz no_mu_seed_13877067520498498525.npz
baseline_seed_6847020926293287670.npz no_mu_seed_14789669151170384611.npz
baseline_seed_7443702551313363331.npz no_mu_seed_15538769139318490183.npz
baseline_seed_7518080831024782954.npz no_mu_seed_15696189914383701419.npz
baseline_seed_7568763936099751332.npz no_mu_seed_16004921505170220512.npz
baseline_seed_9166811211305776210.npz no_mu_seed_16165962154861038927.npz
manifest.json no_mu_seed_16301226009792754359.npz
no_delay_seed_10138059242446602359.npz no_mu_seed_17260448256843220567.npz
no_delay_seed_10162605287015011567.npz no_mu_seed_17619579101733829571.npz
no_delay_seed_10280671141729910962.npz no_mu_seed_17629994763630970995.npz
no_delay_seed_10414151041756256180.npz no_mu_seed_17813068995467247720.npz

no_delay_seed_10460494971182033875.npz no_mu_seed_17942362857188767334.npz
no_delay_seed_10500527434474296483.npz no_mu_seed_18120325990998516096.npz
no_delay_seed_10619621485132495208.npz no_mu_seed_3464540282706013453.npz
no_delay_seed_1071435888660092917.npz no_mu_seed_3867900559517386202.npz
no_delay_seed_11552247507883116273.npz no_mu_seed_4004306315412470776.npz
no_delay_seed_11699415664738731472.npz no_mu_seed_4026263611424279724.npz
no_delay_seed_11760291472930266701.npz no_mu_seed_4213031461048503485.npz
no_delay_seed_12101127255601305672.npz no_mu_seed_4225958022653391162.npz
no_delay_seed_12423041489110211363.npz no_mu_seed_429760635087307899.npz
no_delay_seed_12672897826997023262.npz no_mu_seed_4773450812113592330.npz
no_delay_seed_12828118411696793387.npz no_mu_seed_4884909319511143462.npz
no_delay_seed_12932465548281524219.npz no_mu_seed_4989123116796778357.npz
no_delay_seed_12990827071890121233.npz no_mu_seed_6799375687132975133.npz
no_delay_seed_13011308780887378248.npz no_mu_seed_6799580967308193508.npz
no_delay_seed_13144254950108472985.npz no_mu_seed_6847020926293287670.npz
no_delay_seed_13475711373734757509.npz no_mu_seed_7443702551313363331.npz
no_delay_seed_13877067520498498525.npz no_mu_seed_7518080831024782954.npz
no_delay_seed_14789669151170384611.npz no_mu_seed_7568763936099751332.npz
no_delay_seed_15538769139318490183.npz no_mu_seed_9166811211305776210.npz
no_delay_seed_15696189914383701419.npz shuffle_R_seed_10138059242446602359.npz
no_delay_seed_16004921505170220512.npz shuffle_R_seed_10162605287015011567.npz
no_delay_seed_16165962154861038927.npz shuffle_R_seed_10280671141729910962.npz
no_delay_seed_16301226009792754359.npz shuffle_R_seed_10414151041756256180.npz
no_delay_seed_17260448256843220567.npz shuffle_R_seed_10460494971182033875.npz
no_delay_seed_17619579101733829571.npz shuffle_R_seed_10500527434474296483.npz
no_delay_seed_17629994763630970995.npz shuffle_R_seed_10619621485132495208.npz
no_delay_seed_17813068995467247720.npz shuffle_R_seed_1071435888660092917.npz
no_delay_seed_17942362857188767334.npz shuffle_R_seed_11552247507883116273.npz
no_delay_seed_18120325990998516096.npz shuffle_R_seed_11699415664738731472.npz
no_delay_seed_3464540282706013453.npz shuffle_R_seed_11760291472930266701.npz
no_delay_seed_3867900559517386202.npz shuffle_R_seed_12101127255601305672.npz
no_delay_seed_4004306315412470776.npz shuffle_R_seed_12423041489110211363.npz
no_delay_seed_4026263611424279724.npz shuffle_R_seed_12672897826997023262.npz
no_delay_seed_4213031461048503485.npz shuffle_R_seed_12828118411696793387.npz
no_delay_seed_4225958022653391162.npz shuffle_R_seed_12932465548281524219.npz
no_delay_seed_429760635087307899.npz shuffle_R_seed_12990827071890121233.npz
no_delay_seed_4773450812113592330.npz shuffle_R_seed_13011308780887378248.npz
no_delay_seed_4884909319511143462.npz shuffle_R_seed_13144254950108472985.npz
no_delay_seed_4989123116796778357.npz shuffle_R_seed_13475711373734757509.npz
no_delay_seed_6799375687132975133.npz shuffle_R_seed_13877067520498498525.npz
no_delay_seed_6799580967308193508.npz shuffle_R_seed_14789669151170384611.npz
no_delay_seed_6847020926293287670.npz shuffle_R_seed_15538769139318490183.npz

no_delay_seed_7443702551313363331.npz shuffle_R_seed_15696189914383701419.npz
no_delay_seed_7518080831024782954.npz shuffle_R_seed_16004921505170220512.npz
no_delay_seed_7568763936099751332.npz shuffle_R_seed_16165962154861038927.npz
no_delay_seed_9166811211305776210.npz shuffle_R_seed_16301226009792754359.npz
no_gap_seed_10138059242446602359.npz shuffle_R_seed_17260448256843220567.npz
no_gap_seed_10162605287015011567.npz shuffle_R_seed_17619579101733829571.npz
no_gap_seed_10280671141729910962.npz shuffle_R_seed_17629994763630970995.npz
no_gap_seed_10414151041756256180.npz shuffle_R_seed_17813068995467247720.npz
no_gap_seed_10460494971182033875.npz shuffle_R_seed_17942362857188767334.npz
no_gap_seed_10500527434474296483.npz shuffle_R_seed_18120325990998516096.npz
no_gap_seed_10619621485132495208.npz shuffle_R_seed_3464540282706013453.npz
no_gap_seed_1071435888660092917.npz shuffle_R_seed_3867900559517386202.npz
no_gap_seed_11552247507883116273.npz shuffle_R_seed_4004306315412470776.npz
no_gap_seed_11699415664738731472.npz shuffle_R_seed_4026263611424279724.npz
no_gap_seed_11760291472930266701.npz shuffle_R_seed_4213031461048503485.npz
no_gap_seed_12101127255601305672.npz shuffle_R_seed_4225958022653391162.npz
no_gap_seed_12423041489110211363.npz shuffle_R_seed_4297606350873079899.npz
no_gap_seed_12672897826997023262.npz shuffle_R_seed_4773450812113592330.npz
no_gap_seed_12828118411696793387.npz shuffle_R_seed_4884909319511143462.npz
no_gap_seed_12932465548281524219.npz shuffle_R_seed_4989123116796778357.npz
no_gap_seed_12990827071890121233.npz shuffle_R_seed_6799375687132975133.npz
no_gap_seed_13011308780887378248.npz shuffle_R_seed_6799580967308193508.npz
no_gap_seed_13144254950108472985.npz shuffle_R_seed_6847020926293287670.npz
no_gap_seed_13475711373734757509.npz shuffle_R_seed_7443702551313363331.npz
no_gap_seed_13877067520498498525.npz shuffle_R_seed_7518080831024782954.npz
no_gap_seed_14789669151170384611.npz shuffle_R_seed_7568763936099751332.npz
no_gap_seed_15538769139318490183.npz shuffle_R_seed_9166811211305776210.npz

./runs/fccebfc14855:
AIEF_1_3_fccebfc14855_edge_diagnostics.npz no_delay_seed_12042.npz no_gap_seed_18042.npz
no_mu_seed_5042.npz
AIEF_1_3_fccebfc14855_scalar_summary.csv no_delay_seed_13042.npz no_gap_seed_19042.npz
no_mu_seed_6042.npz
baseline_seed_10042.npz no_delay_seed_14042.npz no_gap_seed_2042.npz no_mu_seed_7042.npz
baseline_seed_1042.npz no_delay_seed_15042.npz no_gap_seed_3042.npz no_mu_seed_8042.npz
baseline_seed_11042.npz no_delay_seed_16042.npz no_gap_seed_4042.npz no_mu_seed_9042.npz
baseline_seed_12042.npz no_delay_seed_17042.npz no_gap_seed_42.npz shuffle_R_seed_10042.npz
baseline_seed_13042.npz no_delay_seed_18042.npz no_gap_seed_5042.npz shuffle_R_seed_1042.npz
baseline_seed_14042.npz no_delay_seed_19042.npz no_gap_seed_6042.npz
shuffle_R_seed_11042.npz
baseline_seed_15042.npz no_delay_seed_2042.npz no_gap_seed_7042.npz shuffle_R_seed_12042.npz
baseline_seed_16042.npz no_delay_seed_3042.npz no_gap_seed_8042.npz shuffle_R_seed_13042.npz

baseline_seed_17042.npz no_delay_seed_4042.npz no_gap_seed_9042.npz shuffle_R_seed_14042.npz
baseline_seed_18042.npz no_delay_seed_42.npz no_mu_seed_10042.npz shuffle_R_seed_15042.npz
baseline_seed_19042.npz no_delay_seed_5042.npz no_mu_seed_1042.npz shuffle_R_seed_16042.npz
baseline_seed_2042.npz no_delay_seed_6042.npz no_mu_seed_11042.npz shuffle_R_seed_17042.npz
baseline_seed_3042.npz no_delay_seed_7042.npz no_mu_seed_12042.npz shuffle_R_seed_18042.npz
baseline_seed_4042.npz no_delay_seed_8042.npz no_mu_seed_13042.npz shuffle_R_seed_19042.npz
baseline_seed_42.npz no_delay_seed_9042.npz no_mu_seed_14042.npz shuffle_R_seed_2042.npz
baseline_seed_5042.npz no_gap_seed_10042.npz no_mu_seed_15042.npz shuffle_R_seed_3042.npz
baseline_seed_6042.npz no_gap_seed_1042.npz no_mu_seed_16042.npz shuffle_R_seed_4042.npz
baseline_seed_7042.npz no_gap_seed_11042.npz no_mu_seed_17042.npz shuffle_R_seed_42.npz
baseline_seed_8042.npz no_gap_seed_12042.npz no_mu_seed_18042.npz shuffle_R_seed_5042.npz
baseline_seed_9042.npz no_gap_seed_13042.npz no_mu_seed_19042.npz shuffle_R_seed_6042.npz
manifest.json no_gap_seed_14042.npz no_mu_seed_2042.npz shuffle_R_seed_7042.npz
no_delay_seed_10042.npz no_gap_seed_15042.npz no_mu_seed_3042.npz shuffle_R_seed_8042.npz
no_delay_seed_1042.npz no_gap_seed_16042.npz no_mu_seed_4042.npz shuffle_R_seed_9042.npz
no_delay_seed_11042.npz no_gap_seed_17042.npz no_mu_seed_42.npz

./runs_aief14_smoke:
ae136b3a165b

./runs_aief14_smoke/ae136b3a165b:
AIEF_1_4_ae136b3a165b_branch_diagnostics.npz manifest.json seed_8824921114796074700.npz
AIEF_1_4_ae136b3a165b_scalar_summary.csv seed_14673009024601869667.npz

./test_output:
analysis_manifest.json matched_phase_paired_contrasts.csv matched_phase_seed_level_contrasts.csv
frozen_anchor_validation.csv matched_phase_recovery_summary.csv matched_phase_seed_status.csv
matched_initial_state_audit.csv matched_phase_report.txt
matched_phase_branch_diagnostics.npz matched_phase_scalar_summary.csv

Your project layout contains all required sources and frozen artifacts. One filename correction is important:

text

Engine: aief_experiment_engine_v1.py
Driver: AIEF_1_4_Twindow_sensitivity.py
Analysis: AIEF_1_4_Twindow_sensitivity_analysis_v3.py

The earlier generic commands used AIEF_1_4_Twindow_sensitivity_analysis.py; on your system, use the _v3.py filename.

The frozen authority is correctly located at:

text

runs/5a9f093a2fe0/manifest.json
runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz
runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv

Do not use the similarly named AIEF-1.3 run directories or runs_aief14_smoke as the frozen authority.
1. Enter and verify the exact project root

You appear to already be in SSRH, but confirm:

Bash

pwd

Then set paths using absolute names:

Bash

set -euo pipefail

export PROJECT_ROOT="$PWD"

export ENGINE="$PROJECT_ROOT/aief_experiment_engine_v1.py"
export DRIVER="$PROJECT_ROOT/AIEF_1_4_Twindow_sensitivity.py"
export ANALYSIS="$PROJECT_ROOT/AIEF_1_4_Twindow_sensitivity_analysis_v3.py"

export FROZEN_DIR="$PROJECT_ROOT/runs/5a9f093a2fe0"
export FROZEN_MANIFEST="$FROZEN_DIR/manifest.json"
export FROZEN_NPZ="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz"
export FROZEN_SCALAR="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv"

```
for path in \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS" \
"$FROZEN_MANIFEST" \
"$FROZEN_NPZ" \
"$FROZEN_SCALAR"
do
test -f "$path" || {
echo "MISSING: $path"
exit 1
}
echo "FOUND: $path"
done
```

If any filename is uncertain because of shell formatting, inspect it literally:

Bash

```
ls -lb -- \
aief_experiment_engine_v1.py \
AIEF_1_4_Twindow_sensitivity.py \
AIEF_1_4_Twindow_sensitivity_analysis_v3.py
```

2. Preserve source and authority hashes

Bash

```
sha256sum \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS" \
> "$PROJECT_ROOT/twindow_source_hashes_before.txt"
```

```
sha256sum \
"$FROZEN_MANIFEST" \
"$FROZEN_NPZ" \
"$FROZEN_SCALAR" \
> "$PROJECT_ROOT/twindow_frozen_authority_hashes.txt"
```

cat "$PROJECT_ROOT/twindow_source_hashes_before.txt"
cat "$PROJECT_ROOT/twindow_frozen_authority_hashes.txt"

Do not edit the three source files after this step. If you edit any source, restart certification.
3. Check environment and disk space

Bash

df -h "$PROJECT_ROOT"

python3 - <<'PY'
import json
import platform
import sys

import numpy as np
import pandas as pd

print("Current environment")
print("Python:", sys.version)
print("NumPy: ", np.__version__)
print("pandas:", pd.__version__)
print("OS: ", platform.platform())

with open(
"runs/5a9f093a2fe0/manifest.json",
encoding="utf-8",
) as handle:
frozen = json.load(handle)

print("\nFrozen environment")
print(json.dumps(frozen.get("environment", {}), indent=2))
PY

Your __pycache__ listing suggests Python 3.13 is being used. Record any difference from the frozen environment; exact replay will decide whether it matters.
4. Compile all three

Bash

```
python3 -m py_compile \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS"
```

5. Import all three

Bash

```
cd "$PROJECT_ROOT"

python3 - <<'PY'
import aief_experiment_engine_v1
import AIEF_1_4_Twindow_sensitivity
import AIEF_1_4_Twindow_sensitivity_analysis_v3

print("All three modules imported successfully.")
PY
```

This is important because module-level errors may not be caught by py_compile.
6. Test basic execution and help

Bash

```
python3 "$ENGINE"

python3 "$DRIVER" --help >/dev/null
python3 "$ANALYSIS" --help >/dev/null

echo "Engine and CLI checks passed."
```

7. Run the driver dry-run

The finalized driver should not require frozen paths for dry-run:

Bash

```
python3 "$DRIVER" --dry-run
```

Expected:

text

```
5 rho values
4 study branches
40 seeds
configuration validated successfully
```

No run directory should be created.
8. Run the one-condition parameterized smoke test

This exercises the actual grid path at ρ=0.5ρ=0.5, including all four study branches and all validation logic:

Bash

```
python3 - <<'PY'
import json
import os

import AIEF_1_4_Twindow_sensitivity as d

with open(
os.environ["FROZEN_MANIFEST"],
encoding="utf-8",
) as handle:
frozen = json.load(handle)

seed = int(
frozen["randomization"]["paired_seeds"][0]
```

```
)
rho = 0.5

config = d.create_model_config(rho)
protocol = d.create_protocol_config()

result = d.simulate_seed_rho(
seed=seed,
rho=rho,
config=config,
protocol=protocol,
study_branches=d.STUDY_BRANCHES,
)

assert tuple(result["branches"]) == d.STUDY_BRANCHES

for branch in d.STUDY_BRANCHES:
branch_result = result["branches"][branch]

d.validate_branch_outputs(
branch=branch,
branch_result=branch_result,
expected_rho=rho,
expected_T_window=config.T_window,
)

for key in d.PRIMARY_OUTCOMES:
assert d.is_finite_number(
branch_result[key]
), (branch, key)

for key in d.SECONDARY_OUTCOMES:
assert d.is_finite_number(
branch_result[key]
), (branch, key)

payload = d.flatten_result(seed, rho, result)
```

```
assert payload
assert len(payload) == len(set(payload))


print(
"Parameterized one-condition smoke passed:",
seed,
rho,
)
PY
```

This may take some time because it runs one complete prefix and four future branches.
9. Run the mandatory baseline-only gate

Use a new root that does not already exist. Your listing shows runs_aief14_smoke, so do not reuse it.

Bash

```
export BASELINE_ROOT="$PROJECT_ROOT/twindow_baseline_gate_20260810_01"


test ! -e "$BASELINE_ROOT"


set -o pipefail
time python3 "$DRIVER" \
--frozen-manifest "$FROZEN_MANIFEST" \
--frozen-npz "$FROZEN_NPZ" \
--frozen-scalar "$FROZEN_SCALAR" \
--baseline-only \
--run-root "$BASELINE_ROOT" \
2>&1 | tee "$PROJECT_ROOT/twindow_baseline_only.log"
```

Locate the generated hash directory:

Bash

```
mapfile -t BASELINE_DIRS < <(
find "$BASELINE_ROOT" \
-mindepth 1 \
-maxdepth 1 \
```

```
-type d \
-print
)

test "${#BASELINE_DIRS[@]}" -eq 1

export BASELINE_DIR="${BASELINE_DIRS[0]}"

echo "Baseline run: $BASELINE_DIR"
```

10. Audit the baseline certificate

Bash

```
python3 - <<'PY'
import json
import os
from pathlib import Path

root = Path(os.environ["BASELINE_DIR"])

with open(
root / "baseline_equivalence_certificate.json",
encoding="utf-8",
) as handle:
certificate = json.load(handle)

with open(
root / "manifest.json",
encoding="utf-8",
) as handle:
manifest = json.load(handle)

expected_branches = [
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
```

```
"B_carrier_only",
"full_reset",
"future_shuffle",
]

assert certificate["status"] == "passed"
assert certificate["n_seeds"] == 40
assert len(certificate["seed_results"]) == 40
assert certificate["protocol"]["branches"] == expected_branches

for seed, result in certificate["seed_results"].items():
assert result["status"] == "passed", seed

failed = [
key
for key, passed in result["checks"].items()
if not passed
]

assert not failed, (seed, failed[:20])

assert certificate["frozen_manifest_file_sha256"]
assert certificate["frozen_npz_file_sha256"]
assert certificate["frozen_scalar_csv_file_sha256"]
assert certificate["engine_source_sha256"]

assert manifest["execution_mode"] == "baseline_only"
assert manifest["baseline_certified"] is True
assert manifest["freezeable"] is False
assert (
manifest["completion"]["status"]
== "baseline_only_completed"
)
assert manifest["completion"]["study_grid_started"] is False

print("Baseline-only certificate audit passed.")
PY
```

The baseline must establish:

text

40/40 seeds passed
7/7 branches passed
all historical arrays exact
all 42 historical scalar fields exact
all rho=1 extensions exact
all baseline horizon metadata exact

If anything fails, stop.
11. Recheck source immutability

Bash

sha256sum -c "$PROJECT_ROOT/twindow_source_hashes_before.txt"

All three must report OK.
12. Launch the production grid

Use another fresh root:

Bash

export PRODUCTION_ROOT="$PROJECT_ROOT/twindow_production_20260810_01"

test ! -e "$PRODUCTION_ROOT"

set -o pipefail
time python3 "$DRIVER" \
--frozen-manifest "$FROZEN_MANIFEST" \
--frozen-npz "$FROZEN_NPZ" \
--frozen-scalar "$FROZEN_SCALAR" \
--run-root "$PRODUCTION_ROOT" \
2>&1 | tee "$PROJECT_ROOT/twindow_full_grid.log"

Locate the run:

Bash

```
mapfile -t PRODUCTION_DIRS < <(
find "$PRODUCTION_ROOT" \
-mindepth 1 \
-maxdepth 1 \
-type d \
-print
)

test "${#PRODUCTION_DIRS[@]}" -eq 1

export RUN_DIR="${PRODUCTION_DIRS[0]}"

echo "Production run: $RUN_DIR"
```

The driver will repeat the mandatory baseline and then run:

text

```
40 seeds × 5 rho values = 200 conditions
200 conditions × 4 branches = 800 scalar rows
```

13. Check production completion before analysis

Bash

```
python3 - <<'PY'
import json
import os
from pathlib import Path

import pandas as pd

root = Path(os.environ["RUN_DIR"])
```

with open(root / "manifest.json", encoding="utf-8") as f:
manifest = json.load(f)

assert manifest["freezeable"] is True
assert (
manifest["baseline_equivalence"]["status"]
== "passed"
)

completion = manifest["completion"]

assert completion["status"] == "completed"
assert completion["n_conditions_requested"] == 200
assert completion["n_conditions_successful"] == 200
assert completion["n_conditions_failed"] == 0
assert completion["n_scalar_rows"] == 800
assert completion["n_missing_primary_outcomes"] == 0
assert completion["n_missing_secondary_outcomes"] == 0

status = pd.read_csv(
root / "twindow_condition_status.csv",
dtype={"seed": str},
)

assert len(status) == 200
assert not status.duplicated(["seed", "rho"]).any()
assert (status["status"] == "success").all()

print("Production completion audit passed.")
PY

14. Run the actual analysis

Use the actual _v3.py filename:

Bash

set -o pipefail

```
time python3 "$ANALYSIS" \
"$RUN_DIR" \
--bootstrap-seed 20260808 \
--n-bootstrap 5000 \
2>&1 | tee "$PROJECT_ROOT/twindow_analysis.log"
```

The analysis output will be created under:

text

$RUN_DIR/analysis/

15. Check analysis output counts

Bash

```
python3 - <<'PY'
import json
import os
from pathlib import Path

import pandas as pd

analysis_dir = Path(os.environ["RUN_DIR"]) / "analysis"

expected = {
"twindow_paired_regime_contrasts.csv": 960,
"twindow_paired_regime_contrasts_fixed_physical.csv": 480,
"twindow_paired_regime_contrasts_normalized_EMA.csv": 480,
"twindow_branch_interactions.csv": 2880,
"twindow_branch_interactions_fixed_physical.csv": 1440,
"twindow_branch_interactions_normalized_EMA.csv": 1440,
"twindow_cross_condition_topology.csv": 160,
"twindow_recovery_by_seed.csv": 3200,
"twindow_recovery_summary.csv": 80,
"twindow_recovery_interactions.csv": 24960,
"twindow_recovery_interactions_summary.csv": 624,
"twindow_regime_summary.csv": 12,
```

```
"twindow_branch_DID_summary.csv": 36,
"twindow_topology_summary.csv": 4,
"twindow_regime_secondary_summary.csv": 12,
"twindow_branch_DID_secondary_summary.csv": 36,
}

for name, expected_rows in expected.items():
path = analysis_dir / name
assert path.is_file(), name

observed_rows = len(pd.read_csv(path))

assert observed_rows == expected_rows, (
name,
observed_rows,
expected_rows,
)

with open(
analysis_dir / "analysis_manifest.json",
encoding="utf-8",
) as f:
manifest = json.load(f)

assert manifest["validation"]["status"] == "passed"
assert manifest["bootstrap"]["seed"] == 20260808
assert manifest["bootstrap"]["n_iterations"] == 5000

print("Analysis-output audit passed.")
PY
```

16. Final freeze

Recheck sources:

Bash

```
sha256sum -c "$PROJECT_ROOT/twindow_source_hashes_before.txt"
```

Create an external artifact inventory:

Bash

```
find "$RUN_DIR" \
-type f \
-print0 \
| sort -z \
| xargs -0 sha256sum \
> "$PROJECT_ROOT/twindow_production_artifact_hashes.txt"
```

Preserve:

text

```
twindow_source_hashes_before.txt
twindow_frozen_authority_hashes.txt
twindow_baseline_only.log
twindow_full_grid.log
twindow_analysis.log
twindow_production_artifact_hashes.txt
baseline-only run directory
production run directory
production analysis directory
```

Important directory note

Your root already contains:

text

```
runs_aief14_smoke/
test_output/
```

Do not reuse or overwrite either for this study. Use the fresh, separately named baseline and production roots above.

The repeated runs use the same seed cohort. They establish reproducibility and parameter sensitivity; they are not independent experiments.

text

```
(deck@steamdeck SSRH)$ source ~/sci-env/bin/activate
(deck@steamdeck SSRH)$ set -euo pipefail
export PROJECT_ROOT="$PWD"
export ENGINE="$PROJECT_ROOT/aief_experiment_engine_v1.py"
export DRIVER="$PROJECT_ROOT/AIEF_1_4_Twindow_sensitivity.py"
export ANALYSIS="$PROJECT_ROOT/AIEF_1_4_Twindow_sensitivity_analysis_v3.py"
export FROZEN_DIR="$PROJECT_ROOT/runs/5a9f093a2fe0"
export FROZEN_MANIFEST="$FROZEN_DIR/manifest.json"
export FROZEN_NPZ="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz"
export FROZEN_SCALAR="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv"
for path in \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS" \
"$FROZEN_MANIFEST" \
"$FROZEN_NPZ" \
"$FROZEN_SCALAR"
do
test -f "$path" || {
echo "MISSING: $path"
exit 1
}
echo "FOUND: $path"
done
FOUND: /home/deck/AI_Programs/SSRH/aief_experiment_engine_v1.py
FOUND: /home/deck/AI_Programs/SSRH/AIEF_1_4_Twindow_sensitivity.py
FOUND: /home/deck/AI_Programs/SSRH/AIEF_1_4_Twindow_sensitivity_analysis_v3.py
FOUND: /home/deck/AI_Programs/SSRH/runs/5a9f093a2fe0/manifest.json
FOUND:
/home/deck/AI_Programs/SSRH/runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz
FOUND:
/home/deck/AI_Programs/SSRH/runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv
(deck@steamdeck SSRH)$ sha256sum \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS" \
> "$PROJECT_ROOT/twindow_source_hashes_before.txt"
```

```
sha256sum \
"$FROZEN_MANIFEST" \
"$FROZEN_NPZ" \
"$FROZEN_SCALAR" \
> "$PROJECT_ROOT/twindow_frozen_authority_hashes.txt"
cat "$PROJECT_ROOT/twindow_source_hashes_before.txt"
cat "$PROJECT_ROOT/twindow_frozen_authority_hashes.txt"
ca4593fda86a1b0a3d450d68b5ce4bddd08583fdb93deaf17db20dd6b1acdede
/home/deck/AI_Programs/SSRH/aief_experiment_engine_v1.py
633d1a21502598021d456bff7ff39b4065b5e5d05218c4411a4f36d53e0956ee
/home/deck/AI_Programs/SSRH/AIEF_1_4_Twindow_sensitivity.py
348093079c37697a1b84a250aca2d0fa74180b39a1b62e20893190a974ffd919
/home/deck/AI_Programs/SSRH/AIEF_1_4_Twindow_sensitivity_analysis_v3.py
bf4d8c4152efafac352c8e1e56759ee7867fdd418cb04a8ee1f2fc87c2315a33
/home/deck/AI_Programs/SSRH/runs/5a9f093a2fe0/manifest.json
8fa9cc059117a7dbef0ed14e52907ec63f7626b50e4b3c8815977547e4590f0f
/home/deck/AI_Programs/SSRH/runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz
f4bcdd1403796f0e2fa4f6fbedaee9d79fd5c970b4c33d49ac995b1f0c3bffda
/home/deck/AI_Programs/SSRH/runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv
(deck@steamdeck SSRH)$ python3 -m py_compile \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS"
(deck@steamdeck SSRH)$ python3 "$ENGINE"
python3 "$DRIVER" --help >/dev/null
python3 "$ANALYSIS" --help >/dev/null
echo "Engine and CLI checks passed."
Engine v1 loaded successfully
N=12, dt=0.01, tau=0.5
T_window=17.502917152858814, alpha=0.999429
mean_omega=1.428333, T_*=17.502917
delay_steps=50
Protocol: T_sim=800.0, burn=160.0, split=480.0, future=320.0
Engine and CLI checks passed.
(deck@steamdeck SSRH)$ python3 "$DRIVER" --dry-run
Dry run: configuration validated successfully.
Validated 5 rho values: (0.5, 0.75, 1.0, 1.5, 2.0)
Validated 4 study branches: ('continue', 'reset_ema', 'reset_B', 'future_shuffle')
Validated 40 seeds from master seed 20260714
Configuration hash: b57562a8ee77
To execute, run without --dry-run
Expected run directory: runs/b57562a8ee77
```

(deck@steamdeck SSRH)$ cd /home/deck/AI_Programs/SSRH
source ~/sci-env/bin/activate

set -euo pipefail
export PROJECT_ROOT="$PWD"
export DRIVER="$PROJECT_ROOT/AIEF_1_4_Twindow_sensitivity.py"
export FROZEN_DIR="$PROJECT_ROOT/runs/5a9f093a2fe0"
export FROZEN_MANIFEST="$FROZEN_DIR/manifest.json"
export FROZEN_NPZ="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz"
export FROZEN_SCALAR="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv"

export OMP_NUM_THREADS=2
export MKL_NUM_THREADS=2
export OPENBLAS_NUM_THREADS=2
export NUMEXPR_NUM_THREADS=2

systemd-run --user --scope -p MemoryMax=10G \
python3 "$DRIVER" \
--master-seed 20260714 \
--n-seeds 40 \
--run-root runs \
--frozen-manifest "$FROZEN_MANIFEST" \
--frozen-npz "$FROZEN_NPZ" \
--frozen-scalar "$FROZEN_SCALAR" \
--baseline-only \
2>&1 | tee /tmp/twindow_baseline_only.log
Running as unit: run-p808844-i808845.scope; invocation ID: bc492923b5b44dd58aa208076f54e693
AIEF_1_4_Twindow_sensitivity
master seed: 20260714
n_seeds: 40
rho values: (0.5, 0.75, 1.0, 1.5, 2.0)
configuration: b57562a8ee77
output: runs/b57562a8ee77

==========
Running MANDATORY baseline equivalence test at rho=1...
==========
Running baseline equivalence test...
Seed 11242426753702356865: passed

Seed 4619122171158768524: passed
Seed 16329425431442249417: passed
Seed 10431384713483787270: passed
Seed 18127452951968972828: passed
Seed 12851640484293199234: passed
Seed 4219039243791140697: passed
Seed 9082055256280075172: passed
Seed 19180074688687723 36: passed
Seed 17883847597205629106: passed
Seed 16627818730624180313: passed
Seed 15403081841477312463: passed
Seed 4040285944958847150: passed
Seed 17521310920850592782: passed
Seed 2461295749352584108: passed
Seed 9036321132460399255: passed
Seed 14040336450255043598: passed
Seed 1888415619847841128: passed
Seed 172180395821707403: passed
Seed 17715532056777442089: passed
Seed 3704716633526084363: passed
Seed 6942591650975756121: passed
Seed 3059864506141486 90: passed
Seed 16952466944611321358: passed
Seed 12123519807951751650: passed
Seed 15853699719446171583: passed
Seed 9189988769602851417: passed
Seed 5213047409853278064: passed
Seed 4932511216876650745: passed
Seed 6108867655028980287: passed
Seed 6023008189364820510: passed
Seed 1230287949579537403: passed
Seed 283404770198602602: passed
Seed 3582616062465030267: passed
Seed 3258463328937640595: passed
Seed 3952397966055989215: passed
Seed 17279552413280415429: passed
Seed 5415544643585892346: passed
Seed 10856823803646276806: passed
Seed 14924267685654216746: passed
Baseline equivalence: PASSED

Baseline equivalence test PASSED

==========
=======
Detailed certificate saved to runs/b57562a8ee77/baseline_equivalence_certificate.json
Per-seed results saved to runs/b57562a8ee77/baseline_equivalence_by_seed.json

Baseline-only run completed successfully.
The T_window grid was not started.
(deck@steamdeck SSRH)$

text

(deck@steamdeck SSRH)$ source ~/sci-env/bin/activateset -euo pipefailexport
PROJECT_ROOT="$PWD"export
DRIVER="$PROJECT_ROOT/AIEF_1_4_Twindow_sensitivity.py"export
FROZEN_DIR="$PROJECT_ROOT/runs/5a9f093a2fe0"export
FROZEN_MANIFEST="$FROZEN_DIR/manifest.json"export
FROZEN_NPZ="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz"export
FROZEN_SCALAR="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv"

export OMP_NUM_THREADS=2export MKL_NUM_THREADS=2export
OPENBLAS_NUM_THREADS=2export NUMEXPR_NUM_THREADS=2

python3 "$DRIVER" --master-seed 20260714 --n-seeds 40 --run-root runs --frozen-manifest
"$FROZEN_MANIFEST" --frozen-npz "$FROZEN_NPZ" --frozen-scalar "$FROZEN_SCALAR"
2>&1 | tee /tmp/twindow_full_grid.logAIEF_1_4_Twindow_sensitivitymaster seed: 20260714n_seeds:
40rho values: (0.5, 0.75, 1.0, 1.5, 2.0)configuration: b57562a8ee77output: runs/b57562a8ee77

Running baseline equivalence test...Seed 11242426753702356865: passedSeed
4619122171158768524: passedSeed 16329425431442249417: passedSeed 10431384713483787270:
passedSeed 18127452951968972828: passedSeed 12851640484293199234: passedSeed
4219039243791140697: passedSeed 9082055256280075172: passedSeed 1918007468868772336:
passedSeed 17883847597205629106: passedSeed 16627818730624180313: passedSeed
15403081841477312463: passedSeed 4040285944958847150: passedSeed 17521310920850592782:
passedSeed 2461295749352584108: passedSeed 9036321132460399255: passedSeed
14040336450255043598: passedSeed 1888415619847841128: passedSeed 172180395821707403:
passedSeed 17715532056777442089: passedSeed 3704716633526084363: passedSeed
6942591650975756121: passedSeed 305986450614148690: passedSeed 16952466944611321358:
passedSeed 12123519807951751650: passedSeed 15853699719446171583: passedSeed
9189988769602851417: passedSeed 5213047409853278064: passedSeed 4932511216876650745:
passedSeed 6108867655028980287: passedSeed 6023008189364820510: passedSeed

1230287949579537403: passedSeed 283404770198602602: passedSeed 3582616062465030267: passedSeed 3258463328937640595: passedSeed 3952397966055989215: passedSeed 17279552413280415429: passedSeed 5415544643585892346: passedSeed 10856823803646276806: passedSeed 14924267685654216746: passedBaseline equivalence: PASSED

Baseline equivalence test PASSED

Detailed certificate saved to runs/b57562a8ee77/baseline_equivalence_certificate.jsonPer-seed results saved to runs/b57562a8ee77/baseline_equivalence_by_seed.json

Running T_window sensitivity study...rho values: (0.5, 0.75, 1.0, 1.5, 2.0)branches: ('continue', 'reset_ema', 'reset_B', 'future_shuffle')seeds: 40[1/200] seed=11242426753702356865, rho=0.50 (22.2s)[2/200] seed=11242426753702356865, rho=0.75 (44.4s)[3/200] seed=11242426753702356865, rho=1.00 (68.1s)[4/200] seed=11242426753702356865, rho=1.50 (92.4s)[5/200] seed=11242426753702356865, rho=2.00 (116.8s)[6/200] seed=4619122171158768524, rho=0.50 (140.1s)[7/200] seed=4619122171158768524, rho=0.75 (163.2s)[8/200] seed=4619122171158768524, rho=1.00 (186.4s)[9/200] seed=4619122171158768524, rho=1.50 (209.2s)[10/200] seed=4619122171158768524, rho=2.00 (232.2s)[11/200] seed=16329425431442249417, rho=0.50 (255.1s)[12/200] seed=16329425431442249417, rho=0.75 (277.5s)[13/200] seed=16329425431442249417, rho=1.00 (299.9s)[14/200] seed=16329425431442249417, rho=1.50 (322.5s)[15/200] seed=16329425431442249417, rho=2.00 (345.2s)[16/200] seed=10431384713483787270, rho=0.50 (368.5s)[17/200] seed=10431384713483787270, rho=0.75 (391.4s)[18/200] seed=10431384713483787270, rho=1.00 (413.8s)[19/200] seed=10431384713483787270, rho=1.50 (436.8s)[20/200] seed=10431384713483787270, rho=2.00 (459.4s)[21/200] seed=18127452951968972828, rho=0.50 (482.6s)[22/200] seed=18127452951968972828, rho=0.75 (505.4s)[23/200] seed=18127452951968972828, rho=1.00 (528.1s)[24/200] seed=18127452951968972828, rho=1.50 (550.3s)[25/200] seed=18127452951968972828, rho=2.00 (572.7s)[26/200] seed=12851640484293199234, rho=0.50 (595.8s)[27/200] seed=12851640484293199234, rho=0.75 (617.9s)[28/200] seed=12851640484293199234, rho=1.00 (639.8s)[29/200] seed=12851640484293199234, rho=1.50 (661.9s)[30/200] seed=12851640484293199234, rho=2.00 (684.6s)[31/200] seed=4219039243791140697, rho=0.50 (707.5s)[32/200] seed=4219039243791140697, rho=0.75 (729.1s)[33/200] seed=4219039243791140697, rho=1.00 (750.9s)[34/200] seed=4219039243791140697, rho=1.50 (772.6s)[35/200] seed=4219039243791140697, rho=2.00 (793.7s)[36/200] seed=9082055256280075172, rho=0.50 (815.4s)[37/200] seed=9082055256280075172, rho=0.75 (836.6s)[38/200] seed=9082055256280075172, rho=1.00 (857.6s)[39/200] seed=9082055256280075172, rho=1.50 (878.5s)[40/200] seed=9082055256280075172, rho=2.00 (899.9s)[41/200] seed=1918007468868772336, rho=0.50 (922.3s)[42/200] seed=1918007468868772336, rho=0.75 (944.2s)[43/200] seed=1918007468868772336, rho=1.00 (965.9s)[44/200] seed=1918007468868772336, rho=1.50 (987.8s)[45/200] seed=1918007468868772336, rho=2.00 (1009.7s)[46/200]

seed=17883847597205629106, rho=0.50 (1032.0s)[47/200] seed=17883847597205629106, rho=0.75 (1054.1s)[48/200] seed=17883847597205629106, rho=1.00 (1076.1s)[49/200] seed=17883847597205629106, rho=1.50 (1098.0s)[50/200] seed=17883847597205629106, rho=2.00 (1120.1s)[51/200] seed=16627818730624180313, rho=0.50 (1141.5s)[52/200] seed=16627818730624180313, rho=0.75 (1162.4s)[53/200] seed=16627818730624180313, rho=1.00 (1183.3s)[54/200] seed=16627818730624180313, rho=1.50 (1204.0s)[55/200] seed=16627818730624180313, rho=2.00 (1224.9s)[56/200] seed=15403081841477312463, rho=0.50 (1246.4s)[57/200] seed=15403081841477312463, rho=0.75 (1267.3s)[58/200] seed=15403081841477312463, rho=1.00 (1288.3s)[59/200] seed=15403081841477312463, rho=1.50 (1309.6s)[60/200] seed=15403081841477312463, rho=2.00 (1330.6s)[61/200] seed=4040285944958847150, rho=0.50 (1351.9s)[62/200] seed=4040285944958847150, rho=0.75 (1372.9s)[63/200] seed=4040285944958847150, rho=1.00 (1393.8s)[64/200] seed=4040285944958847150, rho=1.50 (1414.8s)[65/200] seed=4040285944958847150, rho=2.00 (1435.7s)[66/200] seed=17521310920850592782, rho=0.50 (1457.2s)[67/200] seed=17521310920850592782, rho=0.75 (1478.0s)[68/200] seed=17521310920850592782, rho=1.00 (1498.8s)[69/200] seed=17521310920850592782, rho=1.50 (1519.6s)[70/200] seed=17521310920850592782, rho=2.00 (1540.5s)[71/200] seed=2461295749352584108, rho=0.50 (1561.8s)[72/200] seed=2461295749352584108, rho=0.75 (1582.8s)[73/200] seed=2461295749352584108, rho=1.00 (1603.8s)[74/200] seed=2461295749352584108, rho=1.50 (1624.6s)[75/200] seed=2461295749352584108, rho=2.00 (1645.5s)[76/200] seed=9036321132460399255, rho=0.50 (1666.8s)[77/200] seed=9036321132460399255, rho=0.75 (1687.9s)[78/200] seed=9036321132460399255, rho=1.00 (1708.9s)[79/200] seed=9036321132460399255, rho=1.50 (1729.8s)[80/200] seed=9036321132460399255, rho=2.00 (1750.6s)[81/200] seed=14040336450255043598, rho=0.50 (1771.9s)[82/200] seed=14040336450255043598, rho=0.75 (1792.8s)[83/200] seed=14040336450255043598, rho=1.00 (1814.0s)[84/200] seed=14040336450255043598, rho=1.50 (1834.7s)[85/200] seed=14040336450255043598, rho=2.00 (1855.9s)[86/200] seed=1888415619847841128, rho=0.50 (1877.5s)[87/200] seed=1888415619847841128, rho=0.75 (1898.5s)[88/200] seed=1888415619847841128, rho=1.00 (1919.3s)[89/200] seed=1888415619847841128, rho=1.50 (1940.3s)[90/200] seed=1888415619847841128, rho=2.00 (1961.3s)[91/200] seed=172180395821707403, rho=0.50 (1982.8s)[92/200] seed=172180395821707403, rho=0.75 (2003.9s)[93/200] seed=172180395821707403, rho=1.00 (2024.9s)[94/200] seed=172180395821707403, rho=1.50 (2045.9s)[95/200] seed=172180395821707403, rho=2.00 (2066.7s)[96/200] seed=17715532056777442089, rho=0.50 (2088.2s)[97/200] seed=17715532056777442089, rho=0.75 (2109.1s)[98/200] seed=17715532056777442089, rho=1.00 (2130.2s)[99/200] seed=17715532056777442089, rho=1.50 (2151.2s)[100/200] seed=17715532056777442089, rho=2.00 (2172.0s)[101/200] seed=3704716633526084363, rho=0.50 (2193.4s)[102/200] seed=3704716633526084363, rho=0.75 (2214.4s)[103/200] seed=3704716633526084363, rho=1.00 (2235.5s)[104/200] seed=3704716633526084363, rho=1.50 (2256.5s)[105/200] seed=3704716633526084363, rho=2.00 (2277.6s)[106/200] seed=6942591650975756121, rho=0.50 (2299.2s)[107/200] seed=6942591650975756121, rho=0.75 (2320.4s)[108/200] seed=6942591650975756121, rho=1.00 (2341.6s)[109/200]

seed=6942591650975756121, rho=1.50 (2362.9s)[110/200] seed=6942591650975756121, rho=2.00 (2384.4s)[111/200] seed=305986450614148690, rho=0.50 (2406.5s)[112/200] seed=305986450614148690, rho=0.75 (2427.9s)[113/200] seed=305986450614148690, rho=1.00 (2449.0s)[114/200] seed=305986450614148690, rho=1.50 (2470.2s)[115/200] seed=305986450614148690, rho=2.00 (2491.4s)[116/200] seed=16952466944611321358, rho=0.50 (2513.4s)[117/200] seed=16952466944611321358, rho=0.75 (2535.1s)[118/200] seed=16952466944611321358, rho=1.00 (2557.5s)[119/200] seed=16952466944611321358, rho=1.50 (2580.2s)[120/200] seed=16952466944611321358, rho=2.00 (2602.4s)[121/200] seed=12123519807951751650, rho=0.50 (2625.0s)[122/200] seed=12123519807951751650, rho=0.75 (2647.0s)[123/200] seed=12123519807951751650, rho=1.00 (2669.0s)[124/200] seed=12123519807951751650, rho=1.50 (2690.9s)[125/200] seed=12123519807951751650, rho=2.00 (2712.8s)[126/200] seed=15853699719446171583, rho=0.50 (2735.5s)[127/200] seed=15853699719446171583, rho=0.75 (2757.9s)[128/200] seed=15853699719446171583, rho=1.00 (2779.9s)[129/200] seed=15853699719446171583, rho=1.50 (2801.7s)[130/200] seed=15853699719446171583, rho=2.00 (2823.6s)[131/200] seed=9189988769602851417, rho=0.50 (2846.3s)[132/200] seed=9189988769602851417, rho=0.75 (2868.3s)[133/200] seed=9189988769602851417, rho=1.00 (2890.2s)[134/200] seed=9189988769602851417, rho=1.50 (2913.0s)[135/200] seed=9189988769602851417, rho=2.00 (2935.4s)[136/200] seed=5213047409853278064, rho=0.50 (2958.0s)[137/200] seed=5213047409853278064, rho=0.75 (2980.4s)[138/200] seed=5213047409853278064, rho=1.00 (3002.5s)[139/200] seed=5213047409853278064, rho=1.50 (3024.7s)[140/200] seed=5213047409853278064, rho=2.00 (3046.9s)[141/200] seed=4932511216876650745, rho=0.50 (3069.6s)[142/200] seed=4932511216876650745, rho=0.75 (3091.3s)[143/200] seed=4932511216876650745, rho=1.00 (3112.8s)[144/200] seed=4932511216876650745, rho=1.50 (3135.2s)[145/200] seed=4932511216876650745, rho=2.00 (3157.8s)[146/200] seed=6108867655028980287, rho=0.50 (3180.6s)[147/200] seed=6108867655028980287, rho=0.75 (3202.9s)[148/200] seed=6108867655028980287, rho=1.00 (3225.0s)[149/200] seed=6108867655028980287, rho=1.50 (3247.1s)[150/200] seed=6108867655028980287, rho=2.00 (3269.0s)[151/200] seed=6023008189364820510, rho=0.50 (3291.8s)[152/200] seed=6023008189364820510, rho=0.75 (3314.6s)[153/200] seed=6023008189364820510, rho=1.00 (3337.0s)[154/200] seed=6023008189364820510, rho=1.50 (3359.3s)[155/200] seed=6023008189364820510, rho=2.00 (3382.3s)[156/200] seed=1230287949579537403, rho=0.50 (3405.0s)[157/200] seed=1230287949579537403, rho=0.75 (3427.4s)[158/200] seed=1230287949579537403, rho=1.00 (3449.7s)[159/200] seed=1230287949579537403, rho=1.50 (3471.7s)[160/200] seed=1230287949579537403, rho=2.00 (3494.1s)[161/200] seed=283404770198602602, rho=0.50 (3516.9s)[162/200] seed=283404770198602602, rho=0.75 (3538.5s)[163/200] seed=283404770198602602, rho=1.00 (3560.0s)[164/200] seed=283404770198602602, rho=1.50 (3582.0s)[165/200] seed=283404770198602602, rho=2.00 (3603.5s)[166/200] seed=3582616062465030267, rho=0.50 (3625.3s)[167/200] seed=3582616062465030267, rho=0.75 (3646.7s)[168/200] seed=3582616062465030267, rho=1.00 (3668.0s)[169/200] seed=3582616062465030267, rho=1.50 (3689.6s)[170/200] seed=3582616062465030267, rho=2.00 (3710.6s)[171/200] seed=3258463328937640595, rho=0.50 (3732.5s)[172/200]

seed=3258463328937640595, rho=0.75 (3754.5s)[173/200] seed=3258463328937640595, rho=1.00 (3776.6s)[174/200] seed=3258463328937640595, rho=1.50 (3798.4s)[175/200] seed=3258463328937640595, rho=2.00 (3819.9s)[176/200] seed=3952397966055989215, rho=0.50 (3842.5s)[177/200] seed=3952397966055989215, rho=0.75 (3864.2s)[178/200] seed=3952397966055989215, rho=1.00 (3885.6s)[179/200] seed=3952397966055989215, rho=1.50 (3906.8s)[180/200] seed=3952397966055989215, rho=2.00 (3928.3s)[181/200] seed=17279552413280415429, rho=0.50 (3950.5s)[182/200] seed=17279552413280415429, rho=0.75 (3972.2s)[183/200] seed=17279552413280415429, rho=1.00 (3993.6s)[184/200] seed=17279552413280415429, rho=1.50 (4015.0s)[185/200] seed=17279552413280415429, rho=2.00 (4036.4s)[186/200] seed=5415544643585892346, rho=0.50 (4058.5s)[187/200] seed=5415544643585892346, rho=0.75 (4080.2s)[188/200] seed=5415544643585892346, rho=1.00 (4102.3s)[189/200] seed=5415544643585892346, rho=1.50 (4124.4s)[190/200] seed=5415544643585892346, rho=2.00 (4145.6s)[191/200] seed=10856823803646276806, rho=0.50 (4167.1s)[192/200] seed=10856823803646276806, rho=0.75 (4188.3s)[193/200] seed=10856823803646276806, rho=1.00 (4210.0s)[194/200] seed=10856823803646276806, rho=1.50 (4231.6s)[195/200] seed=10856823803646276806, rho=2.00 (4253.0s)[196/200] seed=14924267685654216746, rho=0.50 (4274.7s)[197/200] seed=14924267685654216746, rho=0.75 (4295.7s)[198/200] seed=14924267685654216746, rho=1.00 (4317.5s)[199/200] seed=14924267685654216746, rho=1.50 (4338.9s)[200/200] seed=14924267685654216746, rho=2.00 (4360.2s)

Condition status saved to runs/b57562a8ee77/twindow_condition_status.csvSuccessful: 200/200Failed: 0/200

Run COMPLETED successfully and is FREEZEABLE

Completed in 4366.2sSaved runs/b57562a8ee77/AIEF_1_4_Twindow_sensitivity_b57562a8ee77_scalar_summary.csvSaved runs/b57562a8ee77/AIEF_1_4_Twindow_sensitivity_b57562a8ee77_branch_diagnostics.npzManifest: runs/b57562a8ee77/manifest.jsonFreezeable: True

rho=0.50, branch=continue | Weighted_similarity_late_fixed_physical=1.0000±0.0000 | Retention_contrast_late_fixed_physical=1.0000±0.0000 | P_mean_late_fixed_physical=0.3205±0.0325rho=0.50, branch=reset_ema | Weighted_similarity_late_fixed_physical=1.0000±0.0000 | Retention_contrast_late_fixed_physical=1.0000±0.0000 | P_mean_late_fixed_physical=0.3205±0.0325rho=0.50, branch=reset_B | Weighted_similarity_late_fixed_physical=1.0000±0.0000 | Retention_contrast_late_fixed_physical=1.0000±0.0000 | P_mean_late_fixed_physical=0.3205±0.0325rho=0.50, branch=future_shuffle | Weighted_similarity_late_fixed_physical=0.5016±0.0349 |

Retention_contrast_late_fixed_physical=0.0022±0.0034 |
P_mean_late_fixed_physical=0.0006±0.0010rho=0.75, branch=continue |
Weighted_similarity_late_fixed_physical=1.0000±0.0000 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.3038±0.0220rho=0.75, branch=reset_ema |
Weighted_similarity_late_fixed_physical=1.0000±0.0000 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.3038±0.0220rho=0.75, branch=reset_B |
Weighted_similarity_late_fixed_physical=1.0000±0.0000 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.3038±0.0220rho=0.75, branch=future_shuffle |
Weighted_similarity_late_fixed_physical=0.5982±0.1189 |
Retention_contrast_late_fixed_physical=0.0001±0.0003 |
P_mean_late_fixed_physical=0.0000±0.0001rho=1.00, branch=continue |
Weighted_similarity_late_fixed_physical=0.9995±0.0033 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.3011±0.0257rho=1.00, branch=reset_ema |
Weighted_similarity_late_fixed_physical=0.9927±0.0418 |
Retention_contrast_late_fixed_physical=0.9900±0.0518 |
P_mean_late_fixed_physical=0.2975±0.0239rho=1.00, branch=reset_B |
Weighted_similarity_late_fixed_physical=0.9995±0.0033 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.3011±0.0257rho=1.00, branch=future_shuffle |
Weighted_similarity_late_fixed_physical=0.4752±0.0484 |
Retention_contrast_late_fixed_physical=0.0000±0.0000 |
P_mean_late_fixed_physical=0.0000±0.0000rho=1.50, branch=continue |
Weighted_similarity_late_fixed_physical=1.0000±0.0000 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.2943±0.0301rho=1.50, branch=reset_ema |
Weighted_similarity_late_fixed_physical=0.9940±0.0162 |
Retention_contrast_late_fixed_physical=0.9875±0.0334 |
P_mean_late_fixed_physical=0.3038±0.0220rho=1.50, branch=reset_B |
Weighted_similarity_late_fixed_physical=1.0000±0.0000 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.2943±0.0301rho=1.50, branch=future_shuffle |
Weighted_similarity_late_fixed_physical=0.4544±0.0246 |
Retention_contrast_late_fixed_physical=0.0000±0.0000 |
P_mean_late_fixed_physical=0.0000±0.0000rho=2.00, branch=continue |
Weighted_similarity_late_fixed_physical=1.0000±0.0000 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.2913±0.0347rho=2.00, branch=reset_ema |
Weighted_similarity_late_fixed_physical=0.9953±0.0144 |

Retention_contrast_late_fixed_physical=0.9907±0.0284 |
P_mean_late_fixed_physical=0.2984±0.0296rho=2.00, branch=reset_B |
Weighted_similarity_late_fixed_physical=1.0000±0.0000 |
Retention_contrast_late_fixed_physical=1.0000±0.0000 |
P_mean_late_fixed_physical=0.2913±0.0347rho=2.00, branch=future_shuffle |
Weighted_similarity_late_fixed_physical=0.4551±0.0271 |
Retention_contrast_late_fixed_physical=0.0000±0.0000 | P_mean_late_fixed_physical=0.0000±0.0000

This is a major successful milestone.
What has now been established
Exact finite replay

The mandatory baseline passed:

text

40/40 frozen seeds
7/7 historical branches
Baseline equivalence: PASSED

This is the first actual saved evidence that the parameterized engine reproduces the frozen AIEF-1.4 run on the complete frozen cohort and current environment.
Regenerated sensitivity grid

The full grid completed:

text

200/200 conditions successful
0 failed conditions
800 expected branch-level scalar rows
freezeable = true

Grid runtime was approximately:
4366.2 s≈72.8 minutes
4366.2 s≈72.8 minutes

excluding the separately executed baseline work.

The printed summaries are descriptive checks only. Do not yet draw formal conclusions from them; the paired regime contrasts, DIDs, topology comparisons, and recovery analyses remain to be run.
Administrative concern: baseline and production used the same directory

Both runs report:

text

runs/b57562a8ee77

Normally, the second invocation should have refused to run because the baseline-only directory already existed, unless:

the directory was deleted between runs;
--overwrite was used in the actual command;
or the baseline-only directory was otherwise recreated.

This is not necessarily a scientific problem because the full production run repeated and passed the entire baseline before running the grid. But verify that the final manifest is a clean full-run manifest and does not retain:

JSON

"execution_mode": "baseline_only"

The separate baseline-only snapshot may no longer exist. Preserve its /tmp/twindow_baseline_only.log if available:

Bash

cp /tmp/twindow_baseline_only.log \
"$PROJECT_ROOT/twindow_baseline_only.log"

Also preserve the grid log:

Bash

```
cp /tmp/twindow_full_grid.log \
"$PROJECT_ROOT/twindow_full_grid.log"
```

If /tmp/twindow_full_grid.log is not the correct path, preserve whichever log contains the complete 200-condition output.

Gate 1 — Audit the production directory

Set the final run directory:

Bash

```
export RUN_DIR="$PROJECT_ROOT/runs/b57562a8ee77"

test -d "$RUN_DIR"
test -f "$RUN_DIR/manifest.json"
test -f "$RUN_DIR/baseline_equivalence_certificate.json"
test -f "$RUN_DIR/baseline_equivalence_by_seed.json"
test -f "$RUN_DIR/twindow_condition_status.csv"
```

Run this comprehensive audit:

Bash

```
python3 - <<'PY'
import hashlib
import json
import os
from pathlib import Path

import numpy as np
import pandas as pd

root = Path(os.environ["RUN_DIR"])
engine_path = Path(os.environ["ENGINE"])
driver_path = Path(os.environ["DRIVER"])
frozen_manifest = Path(os.environ["FROZEN_MANIFEST"])
```

```
frozen_npz = Path(os.environ["FROZEN_NPZ"])
frozen_scalar = Path(os.environ["FROZEN_SCALAR"])


def sha256(path):
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(
lambda: handle.read(1024 * 1024),
b"",
):
digest.update(block)
return digest.hexdigest()


with open(root / "manifest.json", encoding="utf-8") as f:
manifest = json.load(f)


with open(
root / "baseline_equivalence_certificate.json",
encoding="utf-8",
) as f:
certificate = json.load(f)


# ------------------------------------------------------------------
# Full-run manifest
# ------------------------------------------------------------------


assert manifest["config_hash"] == "b57562a8ee77"
assert manifest["freezeable"] is True
assert manifest.get("execution_mode") != "baseline_only"


completion = manifest["completion"]


assert completion["status"] == "completed"
assert completion["n_conditions_requested"] == 200
assert completion["n_conditions_successful"] == 200
assert completion["n_conditions_failed"] == 0
assert completion["n_scalar_rows"] == 800
assert completion["n_missing_primary_outcomes"] == 0
assert completion["n_missing_secondary_outcomes"] == 0
```

```
assert completion["freezeable"] is True


# ---------------------------------------------------------------------
# Baseline certificate
# ---------------------------------------------------------------------

expected_frozen_branches = [
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
]

assert certificate["status"] == "passed"
assert certificate["n_seeds"] == 40
assert len(certificate["seed_results"]) == 40
assert (
certificate["protocol"]["branches"]
== expected_frozen_branches
)

for seed, result in certificate["seed_results"].items():
assert result["status"] == "passed", seed

failed_checks = [
name
for name, passed in result["checks"].items()
if not passed
]
assert not failed_checks, (
seed,
failed_checks[:20],
)

assert not result.get("max_diffs"), (
seed,
result.get("max_diffs"),
```

```
)

# ---------------------------------------------------------------
# Current sources versus run-recorded sources
# ---------------------------------------------------------------

dependencies = manifest["dependencies"]

assert (
sha256(engine_path)
== dependencies[
"aief_experiment_engine_v1"
]["sha256"]
), "Current engine differs from production engine"

assert (
sha256(driver_path)
== dependencies[
"AIEF_1_4_Twindow_sensitivity"
]["sha256"]
), "Current driver differs from production driver"

assert (
sha256(engine_path)
== certificate["engine_source_sha256"]
), "Certificate engine hash mismatch"

# ---------------------------------------------------------------
# Frozen input authorities
# ---------------------------------------------------------------

frozen_hashes = manifest[
"baseline_equivalence"
]["frozen_input_hashes"]

assert (
sha256(frozen_manifest)
== frozen_hashes["manifest_file_sha256"]
)
```

```
assert (
sha256(frozen_npz)
== frozen_hashes["npz_file_sha256"]
)
assert (
sha256(frozen_scalar)
== frozen_hashes["scalar_csv_file_sha256"]
)


# ---------------------------------------------------------------
# Condition status
# ---------------------------------------------------------------


status_path = root / "twindow_condition_status.csv"


status = pd.read_csv(
status_path,
dtype={"seed": str},
)


assert len(status) == 200
assert not status.duplicated(["seed", "rho"]).any()
assert (status["status"] == "success").all()
assert status["seed"].nunique() == 40
assert set(status["rho"].astype(float)) == {
0.5,
0.75,
1.0,
1.5,
2.0,
}


# ---------------------------------------------------------------
# Main scalar and NPZ files
# ---------------------------------------------------------------


scalar_files = list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*"
"_scalar_summary.csv"
```

```
)
)
npz_files = list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*"
"_branch_diagnostics.npz"
)
)

assert len(scalar_files) == 1
assert len(npz_files) == 1

scalar_path = scalar_files[0]
npz_path = npz_files[0]

df = pd.read_csv(
scalar_path,
dtype={"seed": str},
)

assert len(df) == 800
assert df["seed"].nunique() == 40
assert not df.duplicated(
["seed", "rho", "branch"]
).any()

assert set(df["rho"].astype(float)) == {
0.5,
0.75,
1.0,
1.5,
2.0,
}

assert set(df["branch"]) == {
"continue",
"reset_ema",
"reset_B",
"future_shuffle",
}
```

```
required_outcomes = [
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
"P_mean_late_fixed_physical",
"Weighted_similarity_late_normalized_EMA",
"Retention_contrast_late_normalized_EMA",
"P_mean_late_normalized_EMA",
]


for column in required_outcomes:
values = pd.to_numeric(
df[column],
errors="coerce",
)
assert np.isfinite(values).all(), column


# ---------------------------------------------------------------------
# Final output hashes
# ---------------------------------------------------------------------


hash_paths = {
"scalar_csv": scalar_path,
"branch_npz": npz_path,
"condition_status_csv": status_path,
"baseline_certificate":
root / "baseline_equivalence_certificate.json",
"baseline_seed_results":
root / "baseline_equivalence_by_seed.json",
}


for label, path in hash_paths.items():
expected = manifest["output_hashes"][label]
observed = sha256(path)
assert observed == expected, (
label,
expected,
observed,
)
```

```
# ---------------------------------------------------------------
# Condition checkpoints
# ---------------------------------------------------------------

checkpoints = list(
root.glob("condition_seed_*_rho_*.npz")
)
assert len(checkpoints) == 200, len(checkpoints)

print("PRODUCTION AUDIT PASSED")
print(" Run:", root)
print(" Conditions: 200/200")
print(" Scalar rows: 800/800")
print(" Baseline seeds: 40/40")
print(" Frozen branches: 7/7")
print(" Checkpoints: 200")
PY
```

Do not run the analysis unless this prints:

text

```
PRODUCTION AUDIT PASSED
```

Gate 2 — Recheck source hashes

Bash

```
sha256sum -c \
"$PROJECT_ROOT/twindow_source_hashes_before.txt"
```

Ideally all three report OK.

If only the analysis source changed during its final review, that does not invalidate the generated grid, but:

recompile and re-import the final analysis source;
record its new hash;

ensure the analysis manifest records that exact hash.

Any change to the engine or driver after grid generation would invalidate the claim that the current sources produced this run.
Gate 3 — Compile and import the final analysis source

Bash

```
python3 -m py_compile "$ANALYSIS"

python3 - <<'PY'
import AIEF_1_4_Twindow_sensitivity_analysis_v3 as a

assert a.EXPECTED_N_ROWS == 800
assert a.EXPECTED_N_TOPOLOGY == 160
assert a.EXPECTED_N_RECOVERY_GROUPS == 80
assert a.EXPECTED_N_RECOVERY_DID == 24960
assert a.EXPECTED_N_RECOVERY_DID_GROUPS == 624

assert len(
a.FULL_COHORT_RECOVERY_DID_OUTCOMES
| a.CONDITIONAL_RECOVERY_DID_OUTCOMES
) == 13

assert not (
a.FULL_COHORT_RECOVERY_DID_OUTCOMES
& a.CONDITIONAL_RECOVERY_DID_OUTCOMES
)

print("Final analysis source import/schema check passed.")
PY
```

Gate 4 — Ensure the analysis destination is fresh

Bash

```
test ! -e "$RUN_DIR/analysis" || {
echo "Analysis directory already exists."
```

echo "Inspect it before deciding whether to use --overwrite."
exit 1
}

Do not use --overwrite for the first analysis.
Phase 5 — Run the analysis

Bash

set -o pipefail

time python3 "$ANALYSIS" \
"$RUN_DIR" \
--bootstrap-seed 20260808 \
--n-bootstrap 5000 \
2>&1 | tee "$PROJECT_ROOT/twindow_analysis.log"

The analysis script will independently verify:

simulation manifest;
freezeability;
baseline certificate;
input hashes;
exact cohort;
exact rho grid;
exact branch set;
TwindowTwindow;
trajectories;
topology matrices;
recovery counts;
DID counts;
summary support.

Gate 5 — Audit the analysis outputs

Bash

python3 - <<'PY'

```
import json
import os
from pathlib import Path

import pandas as pd

analysis_dir = Path(os.environ["RUN_DIR"]) / "analysis"

expected_rows = {
"twindow_paired_regime_contrasts.csv": 960,
"twindow_paired_regime_contrasts_fixed_physical.csv": 480,
"twindow_paired_regime_contrasts_normalized_EMA.csv": 480,
"twindow_branch_interactions.csv": 2880,
"twindow_branch_interactions_fixed_physical.csv": 1440,
"twindow_branch_interactions_normalized_EMA.csv": 1440,
"twindow_cross_condition_topology.csv": 160,
"twindow_recovery_by_seed.csv": 3200,
"twindow_recovery_summary.csv": 80,
"twindow_recovery_interactions.csv": 24960,
"twindow_recovery_interactions_summary.csv": 624,
"twindow_regime_summary.csv": 12,
"twindow_branch_DID_summary.csv": 36,
"twindow_topology_summary.csv": 4,
"twindow_regime_secondary_summary.csv": 12,
"twindow_branch_DID_secondary_summary.csv": 36,
}

for name, expected in expected_rows.items():
path = analysis_dir / name
assert path.is_file(), name

observed = len(pd.read_csv(path))
assert observed == expected, (
name,
observed,
expected,
)

recovery = pd.read_csv(
analysis_dir / "twindow_recovery_by_seed.csv",
```

```
dtype={"seed": str},
)


recovery_sizes = recovery.groupby(
["rho", "branch", "horizon_system", "metric"]
).size()


assert len(recovery_sizes) == 80
assert (recovery_sizes == 40).all()


recovery_did = pd.read_csv(
analysis_dir / "twindow_recovery_interactions.csv",
dtype={"seed": str},
)


did_sizes = recovery_did.groupby(
[
"rho",
"branch",
"horizon_system",
"metric",
"outcome",
]
).size()


assert len(did_sizes) == 624
assert (did_sizes == 40).all()


assert set(recovery_did["branch"]) == {
"reset_ema",
"reset_B",
"future_shuffle",
}


with open(
analysis_dir / "analysis_manifest.json",
encoding="utf-8",
) as handle:
manifest = json.load(handle)
```

```
assert manifest["validation"]["status"] == "passed"
assert manifest["bootstrap"]["seed"] == 20260808
assert manifest["bootstrap"]["n_iterations"] == 5000

print("ANALYSIS AUDIT PASSED")
PY
```

Gate 6 — Preserve the evidence

Bash

```
sha256sum -c \
"$PROJECT_ROOT/twindow_source_hashes_before.txt"
```

Then create a file inventory outside the run directory:

Bash

```
find "$RUN_DIR" \
-type f \
-print0 \
| sort -z \
| xargs -0 sha256sum \
> "$PROJECT_ROOT/twindow_production_artifact_hashes.txt"
```

Preserve:

text

```
runs/b57562a8ee77/
twindow_source_hashes_before.txt
twindow_frozen_authority_hashes.txt
twindow_baseline_only.log
twindow_full_grid.log
twindow_analysis.log
twindow_production_artifact_hashes.txt
```

Scientific status at this point

You may now state as saved finite evidence:

the parameterized engine exactly reproduced the frozen 40-seed, seven-branch baseline;
all 200 regenerated TwindowTwindow conditions completed;
the production driver reported the run as freezeable.

You should not yet interpret the descriptive lines as inferential conclusions. The paired contrasts, branch DIDs, cross-TT topology, and recovery scaling must first pass the analysis stage and its audits.

The initial test was renamed pivot run and moved to a different directory. Then the full system was ran, as you saw.

text

I think that the next architectural direction that makes the most sens is:
Right now the T_window pipeline already hints at this separation:

* `aief_experiment_engine_v1.py` contains the dynamical system and replay machinery.

* `AIEF_1_4_Twindow_sensitivity.py` defines an intervention protocol.

* `AIEF_1_4_Twindow_sensitivity_analysis_v3.py` defines estimands and statistical summaries.

That is already a three-layer architecture. Your idea is to make the middle layer modular.

Modular experiment architecture

Engine

Dynamics, replay, perturbations

Driver

Intervention protocol

Analysis

Estimands and inference

The advantage is that you can ask many different questions while preserving exactly the same numerical implementation.

What struck me from reading the T_window driver is that most of the code is protocol, not physics. The engine already knows how to:

* replay trajectories,

* manage branches,

* perturb state,

* compute observables,

* save artifacts,

* validate consistency.

The driver mostly chooses when and how to intervene.

That suggests the natural abstraction is not “different scripts,” but different intervention maps.

For example, a driver could specify:

Driver	Intervention
T_window sensitivity	Change EMA horizon
Memory ablation	Reset EMA / B / history
Coherence forcing	Perturb C variables
Branch geometry	Alter winner margins
Delay sensitivity	Change tau

| Noise response | Inject controlled perturbations |

All of those could share the same engine and the same artifact schema.

The deeper benefit is comparability. One thing we struggled with in the spectral work was that every script introduced a slightly different state representation and slightly different assumptions. With a modular driver architecture, every experiment inherits:

* the same replay,

* the same frozen authority,

* the same baseline equivalence,

* the same seed generation,

* the same output schema.

That means results become directly comparable across studies.

I would actually go one step further and make the drivers declarative.

Instead of each driver containing hundreds of lines of orchestration, define a small experiment specification, conceptually something like:

Python

Run

```
Experiment(
intervention=TWindowScale(rho=[0.75,1.0,1.5]),
branches=[Continue(), ResetEMA(), ResetB(), FutureShuffle()],
outcomes=[WeightedSimilarity(), RetentionContrast(), PMean()],
design=PairedSeeds(n=40),
baseline=FrozenReplay()
)
```

text

Then the engine executes that specification.

The important scientific consequence is that you separate implementation validity from hypothesis validity.

Right now the T_window driver is tightly coupled to one hypothesis. In a modular architecture, the engine is validated once, and new hypotheses are introduced by small intervention modules. That makes it much easier to argue that differences between studies arise from the intervention rather than from changes in the numerical machinery.

I also think this would help with the issue that emerged from the spectral analysis. The compressed Jacobian work was trying to infer mechanism from a specialized coordinate representation. A modular driver system would let you test those mechanisms experimentally instead:

* perturb the coherence bundle,

* perturb the B subsystem,

* perturb history,

* measure downstream observables,

all using the same validated engine.

So my strongest recommendation would be to evolve this into a driver API with a small number of required methods: `initialize()`, `intervene()`, `collect_outcomes()`, and `validate()`. The T_window study then becomes just one implementation of that interface, and future studies can be added with relatively small modules rather than new monolithic scripts. That would make the engine a genuine experimental platform rather than a single-purpose simulator.

Moving the pivot baseline run resolves the directory-lineage concern, assuming the full run began from a clean runs/b57562a8ee77 directory. Preserve the pivot certificate separately, but treat the production run’s repeated baseline as the certificate that directly authorizes its grid.

The modular direction is sound, with one important correction:

Not every experimental change is an intervention map.

T_window, dt, tau, and safeguard changes alter the dynamics from initialization onward. They must be represented as model-regime factors requiring full regeneration, not as split-state interventions.
Recommended architecture

text

Validated numerical kernel
↓
Experiment specification / runner
↓
Interventions and forcing policies
↓
Collectors and artifact schema
↓
Study-specific estimands and inference

1. Numerical kernel

The kernel should continue to own:

initialization;
delay-history management;
EMA update;
weight update;
positivity and symmetrization;
global admissibility map;
phase update;
RNG construction;
state cloning;
step diagnostics.

Drivers should not implement or override initialization. Allowing each driver to define initialize() would reintroduce exactly the numerical drift the platform is meant to prevent.
2. Model-regime factors

These modify ModelConfig before initialization:

text

T_window
dt
tau
omega
eta, mu, gamma
capacity parameters
safeguard epsilon

Each factor must declare its regeneration policy:

Python

```
RegimeFactor(
name="T_window_scale",
levels=(0.5, 0.75, 1.0, 1.5, 2.0),
regeneration="full_from_initialization",
)
```

For this study:

text

T_window sensitivity = model-regime factor
not a split-state intervention

The same applies to future dt, tau, and safeguard studies. No split state may be reused across those levels.
3. State interventions

These are transformations applied at a declared protocol event, such as the split:

text

reset EMA
reset B
reset B + EMA
randomize phase/history
preserve mature B while resetting other channels

A safe interface would be narrower than general arbitrary code:

Python

```
class StateIntervention(Protocol):
name: str
version: str
touched_channels: frozenset[str]
requires_rng: bool

def validate_compatibility(
self,
model: ModelConfig,
state: DynamicState,
) -> None: ...

def apply(
self,
state: DynamicState,
rng: np.random.Generator,
) -> None: ...
```

Each intervention should declare exactly what it may touch:

Python

```
ResetEMA(
touched_channels={"coherence_acc"},
requires_rng=False,
)

FullReset(
```

touched_channels={
"B",
"coherence_acc",
"theta",
"history",
},
requires_rng=True,
)

The runner should audit untouched channels before and after intervention.
4. Dynamic forcing policies

Some proposed studies are neither model factors nor one-time state interventions:

text

future_shuffle
continuous coherence forcing
noise injection
time-localized perturbation

These need a separate interface because they act during stepping:

Python

class ForcingPolicy(Protocol):
name: str
version: str
rng_stream_name: str | None

def driver(
self,
step: int,
state: DynamicState,
physical_observable: np.ndarray,
rng: np.random.Generator,
) -> np.ndarray: ...

This distinction matters scientifically:
Study Correct category
TwindowTwindow sensitivity Model-regime factor; full regeneration
Memory ablation Split-state intervention
Future shuffle Dynamic forcing policy
Coherence impulse Scheduled state intervention
Continuous coherence forcing Dynamic forcing policy
Delay sensitivity Model-regime factor; full regeneration
Timestep sensitivity Discretization regime; full regeneration
Safeguard sensitivity Operator/configuration regime; full regeneration
Noise response Stochastic forcing policy with dedicated RNG stream
Changing winner/admissibility rules New operator/model, not merely an intervention

Changing the actual admissibility rule or mirror geometry would constitute a new model/operator version, not a plugin under the already certified engine.
Refined declarative specification

A reasonable first version is an immutable Python dataclass, not yet a YAML or custom DSL:

Python

```
ExperimentSpec(
name="AIEF_1_4_Twindow_sensitivity",
version="v1.0",
base_model="frozen_AIEF_1_4",
regime_axes=(
TWindowScale(
levels=(0.5, 0.75, 1.0, 1.5, 2.0),
reference_level=1.0,
regeneration="full",
),
),
protocol=SplitProtocol(
burn_end=160.0,
split_time=480.0,
future_duration=320.0,
diagnostic_stride=50,
),
branch_rng_namespace=FROZEN_BRANCH_ORDER,
branches=(
BranchSpec("continue", interventions=()),
```

```
BranchSpec("reset_ema", interventions=(ResetEMA(),)),
BranchSpec("reset_B", interventions=(ResetB(),)),
BranchSpec(
"future_shuffle",
forcing=FutureEdgeShuffle(),
),
),
design=PairedSeedDesign(
master_seed=20260714,
n_seeds=40,
),
collectors=(
CoreTrajectoryCollector(),
DualHorizonCollector(
fixed=(17.5, 87.5, 320.0),
normalized=(1.0, 5.0),
),
),
required_gates=(
FrozenBaselineGate(),
ConditionSchemaGate(),
ArtifactIntegrityGate(),
),
)
```

The canonical serialized form of this specification should define the experiment hash. Absolute paths and platform strings should remain provenance metadata, not scientific configuration identity.
Keep analysis separate

Your separation of execution from inference is important and should remain.

The experiment specification may name required raw observables, but it should not silently define inferential conclusions. The analysis layer should continue to own:

fixed-physical primary estimands;
normalized-EMA secondary estimands;
paired regime contrasts;
branch DIDs;
topology comparisons;
recovery/censoring conventions;
bootstrap procedures;

expected row/group counts.

This preserves the distinction between:

implementation validity;
generated finite evidence;
statistical estimands;
scientific interpretation.

A validated engine does not validate a hypothesis, and a valid intervention does not validate an estimand.
Stable artifact envelope, not one universal flat schema

A single immutable schema for every future study is too restrictive. Use:
Stable core envelope

text

manifest
engine and plugin hashes
canonical experiment specification
paired seed list
factor levels
branch RNG namespace
condition status
prefix state arrays
B_final
sample times
density trajectory
similarity trajectory
projection diagnostics
baseline certificate

Versioned study extensions

text

dual-horizon matrices

forcing diagnostics
noise-stream metadata
perturbation amplitudes
operator-specific diagnostics

Every output should record:

text

core_schema_version
study_schema_version
engine_hash
specification_hash
intervention/forcing module hashes

Validation cannot happen only once

A better formulation is:

A particular engine source hash is certified once against the frozen authority, but every production study must verify that it is using that certified hash and must run or reference an authenticated baseline certificate.

Additional requirements:

every intervention module gets unit and channel-touch tests;
every forcing module gets deterministic RNG tests;
every experiment specification gets compatibility validation;
every operator-changing plugin requires a new engine/model certification;
every production run records all module hashes;
reference-level conditions reproduce the certified baseline exactly;
nonreference parameter conditions receive condition-level schema checks.

Repeated execution with the same seeds remains a reproducibility check, not independent evidence.
Migration plan

Do not refactor the current successful pipeline in place.
Phase A — Finish and freeze v1

Audit runs/b57562a8ee77.
Run the current analysis.
Audit all expected output counts.
Freeze source and artifact hashes.
Update evidence lineage.

Phase B — Create the platform beside v1

Create new files such as:

text

aief_experiment_spec_v1.py
aief_experiment_runner_v1.py
aief_interventions_v1.py
aief_forcing_policies_v1.py
aief_collectors_v1.py

Keep:

text

aief_experiment_engine_v1.py
AIEF_1_4_Twindow_sensitivity.py

unchanged as frozen oracles.
Phase C — Encode the existing study declaratively

Implement the current TwindowTwindow study as the first ExperimentSpec.

Required equivalence:

identical seed set;
identical branch RNG assignments;
identical prefixes;
identical branch arrays;

identical scalars;
identical condition keys;
identical analysis inputs.

Compare the modular runner against the current successful run. This is a workflow equivalence test, not a new experiment.
Phase D — Add new studies

A sensible order is:

regenerated timestep sensitivity;
regenerated safeguard sensitivity;
delay sensitivity;
controlled state-channel perturbations;
stochastic forcing/noise response.

Do not mix timestep, safeguard, and delay changes into one first modular study.
Bottom line

The modular-driver idea is the correct architectural direction, but the safe abstraction is broader than “intervention maps”:

text

model factors
+ state interventions
+ dynamic forcing policies
+ collectors
+ validation gates

The engine should retain exclusive ownership of initialization and stepping. Drivers should become small declarative specifications, while analysis remains separately versioned.

Most importantly, finish and freeze the successful TwindowTwindow evidence before beginning the refactor. The current v1 pipeline should become the golden oracle against which the modular platform is certified.

No I will deal with the frozen data and runs, I need to to create very detailed coding instructions for our Coding Agent on how to modify the driver file to create a robust and versatile modular-driver. That may include adding a simple system that organizes the outputs of the different studies by name or number or something....
WriteAIEF_MODULAR_DRIVER_CODING_AGENT_PROMPT.md1034 lines
Coding-Agent Instructions: AIEF Modular Experiment Driver Platform v1MD
Coding-Agent Instructions: AIEF Modular Experiment Driver Platform v1
0. Role and objective

You are implementing a new, versioned modular experiment-driver platform for AIEF. The goal is to extract reusable experiment orchestration from the successful T_window study without changing the validated numerical dynamics.

The current architecture already has three layers:

aief_experiment_engine_v1.py — numerical dynamics, state, RNG hierarchy, replay, branch simulation, diagnostics, and frozen-baseline equivalence.
AIEF_1_4_Twindow_sensitivity.py — study protocol and orchestration.
AIEF_1_4_Twindow_sensitivity_analysis_v3.py — estimands and statistical analysis.

Your task is to modularize the middle layer. Build a general runner plus declarative study specifications so future studies can reuse the same certified engine and artifact machinery.

The first and mandatory reference implementation is the existing T_window sensitivity study. The modular version must reproduce the current driver's scientific behavior and data products.
1. Non-negotiable constraints
1.1 Do not modify frozen or successful production sources

Do not edit or overwrite:

text

aief_experiment_engine_v1.py
AIEF_1_4_Twindow_sensitivity.py
AIEF_1_4_Twindow_sensitivity_analysis_v3.py
AIEF_1_4_memory_ablation_updated.py
runs/5a9f093a2fe0/
runs/b57562a8ee77/

Treat these as authorities/oracles. Add new files. If a compatibility wrapper is eventually desired, create it only after the modular implementation has passed equivalence testing.
1.2 Do not change the numerical model

The modularization must not alter:

text

N
dt
tau
omega
eta
mu
gamma
capacity
B_c
B_init_scale
T_window semantics
delay indexing
EMA update
positivity map
symmetrization
global gate
minimum.outer endpoint scaling
fixed +1e-12 safeguard
RNG streams
seed pairing
split-state protocol
branch RNG namespace
trajectory sampling schedule

Do not rewrite the step map. Import and call the existing engine.
1.3 Do not represent every change as a state intervention

Use four distinct categories:

Model-regime factors — modify ModelConfig before initialization and require full regeneration.
Examples: T_window, dt, tau, safeguard epsilon.

State interventions — transform a state at a scheduled protocol event. Examples: reset EMA, reset B, reset phase/history.
Dynamic forcing policies — modify a driver during future stepping. Example: future_shuffle.
Operator/model changes — changes to the actual admissibility rule, mirror dynamics, delay law, etc. These require a new engine/model version and are not ordinary plugins under engine v1.

T_window scaling is a model-regime factor, not a split-state intervention.
1.4 Full regeneration policy

A changed T_window, dt, tau, safeguard, or any model parameter requires a fresh trajectory from initialization. Never reuse a split state across incompatible model configurations.
1.5 Inferential unit

The seed/graph is the inferential unit. Branches, factor levels, edges, components, horizons, perturbation directions, and repeated runs within a seed are paired measurements, not independent replicates.
1.6 No fake feature support

Engine v1 exposes only its current branch/intervention behavior. Do not claim that arbitrary noise, forcing, delay changes, or user-defined interventions are supported unless the existing engine can actually execute them without modification.

Define compatibility interfaces and fail clearly for unsupported policies. Do not silently implement a different numerical model in the runner.
2. Existing authorities and golden references
2.1 Frozen baseline authority

text

runs/5a9f093a2fe0/manifest.json
runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz
runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv

2.2 Successful production T_window oracle

text

runs/b57562a8ee77/

The successful run established:

text

40/40 frozen seeds passed baseline equivalence
7/7 frozen branches passed
200/200 regenerated (seed, rho) conditions succeeded
800 branch-level scalar rows
freezeable = true

Use its manifest, scalar CSV, consolidated NPZ, condition checkpoints, and certificate as golden reference data. Do not interpret a repeated equivalence run as an independent experiment.
2.3 Compatibility requirement

For the modular T_window implementation, compare logical contents, not compressed-file bytes:

scalar row identity and ordering;
exact seed, rho, and branch keys;
exact scalar values with NaN-aware equality;
exact array shape, dtype, and NaN-aware values;
exact branch RNG assignment;
exact prefix and future outputs;
exact horizon metadata;
exact condition-status coverage.

Do not require NPZ file SHA-256 equality because compression/container metadata may differ. Compare arrays after loading.
3. Required new file structure

Create a new package; do not turn the current driver into another monolith.

Recommended structure:

text

aief_modular/
__init__.py

spec.py
registry.py
factors.py
branches.py
collectors.py
validation.py
artifacts.py
runner.py
catalog.py
studies/
__init__.py
aief_1_4_s01_twindow.py

AIEF_modular_driver_v1.py

Tests:

text

tests_modular/
test_spec_canonicalization.py
test_registry.py
test_condition_expansion.py
test_rng_namespace.py
test_artifact_paths.py
test_atomic_writes.py
test_twindow_one_condition.py
test_twindow_oracle_equivalence.py

If the repository does not use pytest, provide an equivalent deterministic self-test script, but pytest is preferred if already available.
4. Core data model

All specification objects must be immutable dataclasses or equivalent immutable records. They must have deterministic canonical serialization.
4.1 ExperimentSpec

Implement an immutable specification containing at least:

Python

```
@dataclass(frozen=True)
class ExperimentSpec:
study_id: str
study_slug: str
study_version: str
title: str
purpose: str
engine_module: str
base_model: ModelConfig
model_axes: tuple[ModelFactorSpec, ...]
protocol: ProtocolSpec
seed_design: SeedDesignSpec
branch_rng_namespace: tuple[str, ...]
branches: tuple[BranchSpec, ...]
collectors: tuple[CollectorSpec, ...]
baseline_gate: BaselineGateSpec
artifact_policy: ArtifactPolicySpec
validators: tuple[ValidatorSpec, ...]
```

Requirements:

validate uniqueness of factor names, branch names, level IDs, and collector names;
require continue when downstream metrics need a continue reference;
require executed branches to be a subset of the branch RNG namespace;
preserve RNG namespace order exactly;
require positive finite times and valid protocol ordering;
require a reference factor level where applicable;
reject unsupported engine policies before any simulation.

4.2 Canonical serialization

Implement a function equivalent to:

Python

```
canonical_json(spec) -> str
```

Requirements:

sorted dictionary keys;
stable list ordering;
no NaN or Infinity;
full-precision float representation;
no absolute source paths;
no timestamps;
no platform/environment data;
no output-root path;
no Python object repr strings.

Use canonical JSON to compute spec_sha256.
4.3 Scientific run identity

Compute a deterministic scientific run_id from a canonical identity payload containing:

study ID and version;
canonical experiment specification;
engine source hash;
runner source hash;
all study/plugin source hashes;
exact paired seed list and seed-generation rule;
model factor levels;
protocol;
branch RNG namespace;
executed branches;
collectors and schema versions.

Do not include:

absolute paths;
host platform;
Python environment strings;
timestamp;
baseline-only versus production execution mode.

Use a 12- or 16-character prefix for human-facing paths, but retain the full SHA-256 in the manifest.
4.4 Execution identity

Baseline-only and production executions of the same scientific specification must not collide.

Use a separate execution_id, for example:

text

baseline_20260810T153000Z_<short nonce>
production_20260810T170000Z_<short nonce>

The timestamp/nonce belongs to execution provenance, not scientific identity.
5. Study registry

Implement an explicit registry with unique IDs.

Example:

Python

register_study(
"AIEF_1_4_S01",
build_twindow_spec,
)

Registry requirements:

duplicate IDs are fatal;
duplicate slugs are fatal;
registry listing is deterministic;
registry entries expose ID, slug, version, title, and source module;
no arbitrary dynamic code loading from untrusted file paths;
each registered study can produce a validated ExperimentSpec.

Required CLI operations:

Bash

```
python AIEF_modular_driver_v1.py list-studies
python AIEF_modular_driver_v1.py describe AIEF_1_4_S01
python AIEF_modular_driver_v1.py dry-run AIEF_1_4_S01
python AIEF_modular_driver_v1.py baseline-only AIEF_1_4_S01 ...
python AIEF_modular_driver_v1.py run AIEF_1_4_S01 ...
python AIEF_modular_driver_v1.py catalog --output-root experiments
```

6. Model-factor interface

Implement a typed model-factor abstraction.

Python

```
@dataclass(frozen=True)
class ModelFactorSpec:
name: str
levels: tuple[FactorLevel, ...]
reference_level_id: str
regeneration_policy: str
```

Each FactorLevel must have:

Python

```
level_id: str # stable machine ID, e.g. "rho_0p50"
label: str # human label, e.g. "rho=0.5"
value: float
metadata: mapping
```

Implement a resolver that maps (base_model, level) to a new immutable ModelConfig.

For T_window:

Python

```
T_window = rho * T_star
alpha = exp(-dt / T_window) # derived by engine config
regeneration_policy = "full_from_initialization"
```

The runner must reject split-state reuse when the policy is full regeneration.

Do not add dt, tau, or safeguard studies now. Only create interfaces and compatibility failures for future implementation.
7. Branch and intervention interface

For engine v1, use explicit mappings to supported engine branch names.

Python

```
@dataclass(frozen=True)
class BranchSpec:
name: str
engine_branch: str
role: str
intervention_ids: tuple[str, ...]
forcing_policy_id: str | None
requires_rng: bool
```

Initial supported mapping:

text

```
continue -> engine branch continue
reset_ema -> engine branch reset_ema
reset_B -> engine branch reset_B
future_shuffle -> engine branch future_shuffle
```

The T_window study must retain the full frozen namespace:

Python

```
(
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
```

"B_carrier_only",
"full_reset",
"future_shuffle",
)

while executing only:

Python

(
"continue",
"reset_ema",
"reset_B",
"future_shuffle",
)

future_shuffle must continue to receive the seventh frozen child stream, not the fourth.

Do not permit branch list order to redefine RNG child positions.
8. Protocol and condition expansion
8.1 Protocol

Represent the current protocol declaratively:

text

T_sim = 800
burn_end_time = 160
split_time = 480
future_duration = 320
diagnostic_stride = 50

8.2 Condition expansion

Expand conditions deterministically in this exact nested order:

text

for seed_index, seed in paired_seeds:
for factor_level in declared level order:

For T_window, level order must be:

Python

(0.5, 0.75, 1.0, 1.5, 2.0)

Each condition needs:

text

condition_id
seed
seed_index
factor level IDs and values
model config hash
protocol hash
expected branch set

Use a stable human-readable condition ID, e.g.:

text

seed_11242426753702356865__rho_0p50

Retain exact numeric factor values in the manifest; do not infer values later from formatted IDs.
9. Collectors and output schema
9.1 Keep inference out of the runner

Collectors may save raw observables and deterministic condition-level summaries. They must not compute seed-level bootstrap intervals or inferential DIDs. Those remain in the analysis layer.
9.2 Core collector interface

Python

class Collector(Protocol):
collector_id: str
schema_version: str

def validate_engine_result(...): ...
def scalar_row(...): ...
def array_payload(...): ...

9.3 T_window compatibility collector

Implement TWindowLegacyCollectorV1 that preserves the current production data contract used by AIEF_1_4_Twindow_sensitivity_analysis_v3.py.

It must preserve:

scalar column names;
seed/rho/branch condition coverage;
primary fixed-physical fields;
secondary normalized-EMA fields;
trajectory names;
B_final names;
prefix names;
NPZ key format, including legacy rho formatting;
condition status schema.

Do not change the current analysis script in this first modularization task.
9.4 Stable core schema versus extensions

Record in the manifest:

text

core_schema_version
study_schema_version
collector IDs and versions
artifact naming policy ID

A future study may add namespaced fields without pretending that every study has an identical flat schema.

10. Output organization

Use a study-organized hierarchy:

text

```
experiments/
AIEF_1_4_S01_twindow_sensitivity/
v1/
<run_id>/
baseline/
<execution_id>/
... baseline artifacts ...
production/
<execution_id>/
... production artifacts ...
```

The leaf execution directory is the run directory passed to analysis.

10.1 Leaf directory layout

Preserve current root-level compatibility files for the T_window study:

text

```
manifest.json
baseline_equivalence_certificate.json
baseline_equivalence_by_seed.json
twindow_condition_status.csv
AIEF_1_4_Twindow_sensitivity_<run_id>_scalar_summary.csv
AIEF_1_4_Twindow_sensitivity_<run_id>_branch_diagnostics.npz
```

Additional organization:

text

```
conditions/
```

condition_seed_<seed>_rho_<level>.npz
logs/
analysis/ # reserved for analysis driver
failure/ # optional structured failure diagnostics

Keep the consolidated CSV/NPZ at the leaf root so the existing analysis script can run unchanged.
10.2 Authority rules

manifest.json is the execution authority.
Consolidated CSV/NPZ are authoritative study outputs.
Per-condition checkpoints are diagnostic and explicitly non-authoritative.
analysis/ is written only by the analysis layer.
Optional catalogs are indexes, not authorities.

10.3 No mutable “latest” authority

Do not make a mutable latest symlink an authority. If created for convenience, mark it non-authoritative and rebuildable.
11. Rebuildable catalog

Implement a catalog command that scans leaf manifests and writes a catalog atomically.

Suggested output:

text

experiments/catalog.csv
experiments/catalog.json

Columns/fields:

text

study_id
study_slug
study_version
run_id
full_spec_sha256

execution_id
execution_mode
created_utc
status
freezeable
engine_sha256
runner_sha256
seed_count
condition_count
path

Requirements:

catalog is derived from manifests;
never use catalog as run authority;
deterministic sorting;
detect duplicate (study_id, version, run_id, execution_id);
flag malformed or incomplete manifests;
atomic write;
no silent overwrites.

12. Runner lifecycle

Implement a generic runner with this lifecycle:

Load registered study.
Validate spec.
Compute source/plugin hashes.
Compute canonical spec hash and scientific run ID.
Allocate a new execution directory and lock it.
Write initial manifest atomically.
Validate frozen authority paths.
Run mandatory frozen-baseline gate.
Save baseline certificate atomically.
If mode is baseline-only, mark completed and exit before condition expansion.
Expand all conditions deterministically.
For each condition:
build a fresh model config;
enforce regeneration policy;
create branch RNGs from the full namespace;

simulate prefix fresh;
simulate declared branches;
validate prefix, branches, arrays, metadata, and required outcomes;
write condition checkpoint atomically;
update condition status atomically.
Consolidate scalar and array outputs.
Verify exact expected condition and row counts.
Hash final outputs.
Write final manifest with freezeable status.
Release the execution lock.
Exit nonzero on any incomplete/non-freezeable production execution.

12.1 Condition status durability

Unlike the current end-only status write, persist condition status after every successfully committed condition or at a documented small interval. Use atomic whole-file replacement.

Do not call this “resume support.” It is crash diagnostics only in v1.
12.2 Resume policy

Do not implement resume in modular platform v1. A safe resume system requires strict validation of every checkpoint against spec, engine, schema, and source hashes. Add a clear failure if --resume is requested.
12.3 Locking

Prevent two processes from writing the same execution directory. Use an atomic lock file or OS file lock. Record PID, host, and start time. Stale-lock handling must require explicit user action; do not silently steal locks.
13. Baseline gate

Every production execution must run the engine’s mandatory baseline gate. There is no --skip-baseline production option.

For T_window, require:

text

40/40 frozen seeds

7/7 frozen branches
all frozen prefix arrays exact
all frozen matrices/trajectories exact
all 42 frozen scalar fields exact
all rho=1 extension identities exact
all horizon metadata exact

Store:

text

frozen manifest file hash
frozen NPZ file hash
frozen scalar CSV file hash
frozen production source hash
engine source hash
runner source hash
study module source hash
certificate path
seed-level certificate path

A certificate is reusable only if all linked source, spec, authority, and environment constraints match exactly. Do not implement certificate reuse in v1; always rerun it.
14. Validation gates

Implement explicit validators rather than message-string heuristics.

Suggested exceptions:

Python

class SpecValidationError(Exception): ...
class BaselineGateError(Exception): ...
class ConditionSchemaError(Exception): ...
class ArtifactIntegrityError(Exception): ...
class UnsupportedPolicyError(Exception): ...

Do not decide fatality by searching exception text for words like schema or missing.

Required condition checks for T_window:

prefix keys, shape, dtype, and finiteness;
all required branch keys;
all 12x12 matrix shapes and finiteness;
exact 640-point sample-time schedule;
density finite and in [0,1];
similarity finite entries in [-1,1], permitting NaN zero-vector cosine;
exact rho and T_window metadata;
exact window_steps, recovery_end_ema, fixed horizons;
finite primary and secondary outcomes;
no output-key collisions;
exact branch set;
exact condition count and scalar-row count.

15. Atomic artifacts and failure handling

All JSON, CSV, NPZ, and text writes must be atomic:

write temporary file in the destination directory;
flush;
fsync file;
atomic replace;
preferably fsync directory.

On failure:

preserve already committed condition checkpoints;
write FAILED_manifest.json or a final manifest with structured failure state;
record exception type, message, condition ID, and traceback path;
mark freezeable=false;
exit nonzero;
do not emit a misleading “completed” status.

Do not catch broad exceptions and continue after global spec/schema incompatibility.
16. CLI requirements

Implement subcommands rather than many loosely related flags.
List

Bash

python AIEF_modular_driver_v1.py list-studies

Describe

Bash

python AIEF_modular_driver_v1.py describe AIEF_1_4_S01

Print canonical study specification and expected counts.
Dry run

Bash

python AIEF_modular_driver_v1.py dry-run AIEF_1_4_S01

Must not create a run directory. Validate:

imports;
registry;
engine compatibility;
factor levels;
protocol;
branch namespace;
expected seeds/conditions/rows;
canonical spec hash and run ID.

Baseline only

Bash

python AIEF_modular_driver_v1.py baseline-only AIEF_1_4_S01 \
--frozen-manifest ... \

--frozen-npz ... \
--frozen-scalar ... \
--output-root experiments

Production run

Bash

```
python AIEF_modular_driver_v1.py run AIEF_1_4_S01 \
--frozen-manifest ... \
--frozen-npz ... \
--frozen-scalar ... \
--output-root experiments
```

Catalog

Bash

```
python AIEF_modular_driver_v1.py catalog \
--output-root experiments
```

Safety

no production --skip-baseline;
no default overwrite;
--overwrite must require an exact leaf execution path and an additional confirmation flag, or omit overwrite entirely in v1;
no resume in v1;
deterministic nonzero exit codes.

17. T_window study specification

Register the first study as:

text

```
study_id: AIEF_1_4_S01
```

study_slug: twindow_sensitivity
study_version: v1

Required model levels:

Python

```
rho = (0.5, 0.75, 1.0, 1.5, 2.0)
T_star = 25 / mean(omega)
T_window = rho * T_star
```

Required protocol:

text

```
T_sim = 800
burn = 160
split = 480
future = 320
diagnostic stride = 50
```

Full RNG namespace:

Python

```
(
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
)
```

Executed branches:

Python

(
"continue",
"reset_ema",
"reset_B",
"future_shuffle",
)

Expected production counts:

text

40 seeds
5 rho levels
200 conditions
4 branches per condition
800 scalar rows
200 condition checkpoints

Primary and secondary execution outputs must remain compatible with the existing analysis driver.
18. Tests and staged acceptance
18.1 Static tests

compile all new files;
import all modules;
list/describe study;
canonical serialization deterministic across repeated calls;
no duplicate registry IDs;
run ID independent of output root and absolute source path;
execution IDs differ between baseline and production modes;
schema versions recorded.

18.2 Unit tests

factor level expansion order;
rho -> T_window exact mapping;
full-regeneration policy enforcement;
branch RNG child assignment, especially future_shuffle child index;
exact seed generation;

condition ID uniqueness;
atomic file writes;
failure manifest behavior;
catalog rebuild and duplicate detection;
unsupported policy failure.

18.3 One-condition smoke

Use one frozen seed and nonreference rho=0.5:

fresh prefix;
all four study branches;
full RNG namespace;
complete schema validation;
checkpoint write/read round-trip;
scalar extraction;
array extraction.

18.4 Frozen baseline certification

Run complete 40-seed, seven-branch baseline equivalence with the new runner. It must pass exactly.
18.5 Selected oracle comparisons

Before a full 200-condition rerun, compare modular outputs with runs/b57562a8ee77 for:

at least two seeds, including seed 4619122171158768524;
all five rho levels;
all four study branches;
all scalar fields;
all required arrays;
all condition metadata.

Use exact NaN-aware equality.
18.6 Full oracle equivalence

Before declaring the modular runner a replacement, run the complete 200-condition study and compare logical contents against runs/b57562a8ee77:

text

200/200 condition keys
800/800 scalar rows
all required arrays exact
all metadata exact
all branch RNG assignments exact

This is a workflow reproducibility test, not an independent experiment.
19. Acceptance criteria

The task is complete only when all of the following are true:

Existing engine, driver, analysis, and successful run are unchanged.
The modular package imports cleanly.
Registry lists AIEF_1_4_S01 deterministically.
Dry-run creates no output directory.
Scientific run ID is stable and portable.
Baseline and production executions cannot collide.
Output hierarchy is organized by study ID, slug, version, run ID, and execution mode.
Every production run has immutable source/spec/authority hashes.
Full branch RNG namespace is preserved.
Every rho level regenerates from initialization.
Baseline gate is mandatory.
Per-condition status is durable and exact.
Consolidated outputs remain compatible with the current analysis driver.
Catalog is rebuildable and non-authoritative.
One-condition smoke passes.
Full frozen baseline passes.
Selected oracle comparisons pass.
Full oracle equivalence passes before replacement status.

20. Prohibited shortcuts

Do not:

mutate the current successful driver in place;
modify engine v1;
copy numerical step code into the runner;

treat T_window as a split intervention;
reuse split states across rho;
shorten or reorder the frozen RNG namespace;
use branch execution order as RNG child order;
omit the complete baseline gate;
create a broad generic plugin that silently changes dynamics;
claim support for unsupported intervention/forcing types;
conflate collectors with inferential analysis;
use edges or branches as independent replicates;
add a new seed cohort;
implement unsafe resume;
use absolute paths or timestamps in scientific run IDs;
make a mutable catalog or latest symlink authoritative;
compare compressed NPZ file hashes instead of logical array contents for oracle equivalence;
call a partially completed run freezeable.

21. Required coding-agent report

When finished, report:

new files created;
existing files modified, which should be none unless explicitly authorized;
CLI examples;
package/module diagram;
canonical identity payload definition;
output directory specification;
study registry contents;
validation and failure model;
tests run and exact results;
one-condition smoke result;
baseline certificate result, if executed;
oracle-comparison result, if executed;
remaining unsupported features;
known limitations and next safe extension.

Do not report “production ready” unless the full acceptance criteria, including baseline and oracle equivalence, have passed.
Open

Created and opened:

It gives the coding agent detailed requirements for:

preserving the frozen engine and successful monolithic driver as oracles;
separating model factors, state interventions, dynamic forcing, and operator changes;
an immutable declarative ExperimentSpec;
canonical scientific run IDs versus execution IDs;
a study registry with numbered IDs such as AIEF_1_4_S01;
organized output paths by study, version, run ID, and execution mode;
a rebuildable, non-authoritative run catalog;
branch RNG namespace preservation;
full-regeneration enforcement;
collectors and schema versioning;
atomic artifacts, locking, failure handling, and no unsafe resume;
CLI subcommands for listing, describing, dry-run, baseline-only, production, and catalog operations;
exact compatibility with the existing TwindowTwindow analysis;
staged tests against the frozen baseline and runs/b57562a8ee77;
complete acceptance criteria and prohibited shortcuts.

The instructions explicitly require the agent to create new modular files rather than modifying the now-successful engine or production driver in place.

The new pipeline still inherits the original numerical and analytical behaviors. It fixes orchestration defects, but it does not correct scientific semantics because it delegates to engine v1 and preserves compatibility with analysis v3.
Status by category
1. Fixed by the new pipeline

These were middle-layer orchestration problems:

Portable, path-independent scientific run identity

Separate baseline and production executions

Durable per-condition status

Transactional condition commits

Typed failures instead of message-string heuristics

Execution locking

No unsafe overwrite

Structured failure manifests

Deterministic condition expansion

Explicit full-regeneration enforcement

Explicit seven-branch RNG namespace

Exact 40-seed and 200-condition design

Logical rather than compressed-byte oracle comparison

These problems are not intentionally carried forward.
2. Still present in generated engine outputs

Because the new runner calls engine v1 unchanged:

| Inherited behavior | Effect in S01 |

|---|---|

| r_mean_* is averaged only at diagnostic samples | Still present in the CSV, although it is not a primary S01 outcome |

| Time trajectories start at 0.01 and end at 319.51 | Still present exactly |

| Exact reproducibility is environment-dependent | Still true; the mandatory baseline gate detects disagreement but does not make floating-point operations portable |

| Engine caches are writable globals | Still present |

| Strict floating-point divisibility rules | Still present, but all S01 configurations satisfy them |

| Fixed horizon code does not generally enforce a 320-unit endpoint | Still an engine limitation, but S01 fixes future_duration=320, so it has no effect here |

| Empty horizons become zero matrices | Still possible in engine v1, but none of the five S01 rho levels has an empty late horizon |

| Baseline authority lacks an independent pinned trust anchor | Partly still present; hashes are recorded, but the modular runner does not contain externally pinned expected artifact hashes |

Thus, some engine problems remain observable, while others are prevented by the narrowly validated S01 specification.
3. Still present when analysis v3 is run

The modular runner produces files compatible with the existing analysis script. Consequently, analysis v3 still has:

The held-duration one-sample-interval semantic issue

Recovery “horizon systems” that select targets but do not restrict observation horizons

Duplicated density recovery rows across horizon labels

Duplicated similarity AUC across horizon labels

“Full future” AUC over only 0.01–319.51

Inconsistent treatment of undefined cosine values

Complete-case selection in conditional recovery DID

A rho-dependent held-event definition

Unadjusted pointwise bootstrap intervals across many comparisons

Nontransactional replacement of analysis outputs

The new pipeline neither introduces nor fixes these; they reappear when analysis v3 consumes its compatible outputs.

4. Compatibility inefficiencies intentionally retained

To pass logical oracle equivalence, the new collector currently preserves:

Scalars duplicated in the consolidated NPZ

The large flat NPZ key namespace

In-memory consolidated payload construction

The complete baseline rerun before every production execution

The original diagnostic trajectory schedule

Bottom line

The new pipeline is:

Orchestration-corrected

Numerically legacy-compatible

Analytically legacy-compatible

Not scientifically corrected for the previously identified engine/analysis issues

For S01’s primary outcomes, several general engine hazards are neutralized by the fixed protocol. But if you run the current analysis v3, its recovery and inferential issues remain.

A safe future structure would preserve legacy fields for oracle compatibility while adding:

Stricter runner validation that rejects configurations with empty/truncated horizons.

Namespaced all-step order-parameter metrics from a new engine version.

A separate analysis v4 with corrected recovery-time semantics.

Independently pinned frozen-authority hashes.

A compact schema in a new collector version.

Those should be versioned changes, not silently substituted into S01 v1.

This is a strong and mostly accurate compatibility statement. The central conclusion is correct:

The modular pipeline can correct orchestration while remaining scientifically and numerically legacy-compatible.

A few items should be worded more precisely, and the “fixed” claims remain provisional until the modular implementation passes oracle-equivalence tests.
Evidence status

At present, distinguish:

Established: the monolithic engine/driver passed the frozen baseline and completed the 200-condition run.
Architectural intention: the modular runner fixes the listed orchestration problems.
Not yet established here: that the modular runner actually preserves every numerical output and fixes every orchestration defect.

Do not call the modular pipeline “orchestration-corrected” as a saved fact until it passes:

text

modular 40-seed baseline gate

selected condition oracle comparisons
full 200-condition logical oracle comparison
failure/locking/transaction tests

A safer current phrase is:

The modular pipeline is designed to correct orchestration defects while preserving Engine v1 and Analysis v3 compatibility. This status becomes established only after oracle certification.

Refinements to the inherited-analysis claims
1. Held-duration semantics

This is accurate, but phrase it conditionally:

The current held event uses ceil(T_window / saved_sample_spacing) consecutive saved samples. Such a run spans one fewer intersample intervals than the sample count suggests. It is a valid discrete-sample estimand, but it is not identical to continuous physical persistence for a full TwindowTwindow.

It is not inherently wrong if explicitly defined as a saved-sample event. It is wrong only if reported as an exact continuous-time duration.
2. Recovery “horizon systems”

Correct. They are more accurately:

text

target-definition systems

Both systems search the same full saved trajectory. They differ principally through the fixed-physical versus normalized-EMA late similarity target.

Analysis v4 should separate:

text

target_definition
observation_window

censoring_horizon
AUC_support
held_duration

3. Undefined cosine handling

“Inconsistent” is slightly too strong. It is context-dependent:

raw trajectory: NaN;
crossing/held tests: NaN is treated as below threshold;
AUC: NaN is imputed to zero;
some engine summaries ignore nonfinite similarity samples.

For positive similarity targets, “below threshold” and zero imputation have similar operational consequences, but they are not the same mathematical statement.

Recommended wording:

Undefined zero-vector cosine remains NaN in raw outputs but is treated as failure-to-recover for event indicators and imputed as zero for recovery AUC. This is an explicit analysis convention, not an implementation identity.

4. Rho-dependent held event

Correct, but it is partly intentional. Holding for one current TwindowTwindow is the normalized-time estimand.

The interpretive limitation is:

Recovery probabilities at different rho levels correspond to different physical hold durations.

Analysis v4 should provide both:

one-current-TwindowTwindow held recovery;
one-fixed-physical-duration held recovery.

5. Unadjusted bootstrap intervals

Correct, but pointwise intervals are not invalid by themselves. The problem arises if they are interpreted jointly across the large family of outcomes.

Use:

Intervals are pointwise descriptive bootstrap intervals and are not simultaneous or multiplicity-adjusted.

Analysis v4 could add:

prespecified primary-family adjustment;
simultaneous max-tt bootstrap bands;
false-discovery-rate summaries for exploratory families;
clear separation of confirmatory and exploratory outcomes.

6. Nontransactional analysis replacement

Correct at the bundle level:

each individual output is written atomically;
the complete set is not committed as one transaction;
a crash can leave a partial new analysis directory.

Call it:

Per-file atomic but not transactionally committed as a complete analysis bundle.

Additional inherited analytical limitations

Two important points should be added.
1. Recovery times are saved-grid crossing times

The trajectories are sampled every 0.5 time units:

text

0.01, 0.51, 1.01, ..., 319.51

Therefore:

first crossing is the first saved crossing;
the actual continuous/discrete-step crossing occurred somewhere since the previous saved sample;
recovery times are interval-censored at the diagnostic resolution.

They should not be described as exact step-level or continuous-time event times.
2. Nominal versus effective EMA horizon

The model stores continuous:
Twindow=ρT∗,
Twindow=ρT∗,

but normalized horizon boundaries use:

Python

window_steps = round(T_window / dt)

Thus the effective horizon is:
Teff=
Teff=

\operatorname{round}(T_{\rm window}/dt),dt.
]

Recovery normalization currently divides by nominal TwindowTwindow, while horizon membership uses TeffTeff. The discrepancy is small but should be recorded.

Analysis v4 should report both:

text

T_window_nominal
window_steps

T_window_effective
rounding_error

Recommended version structure
Legacy-compatible track

text

Engine v1
Modular runner v1
Legacy collector v1
Analysis v3

Status:

text

orchestration-modernized
numerically Engine-v1 compatible
artifact-compatible
analysis-v3 compatible
known semantics preserved

Corrected-analysis track

text

Engine v1
Modular runner v1
Legacy collector v1
Analysis v4

Analysis v4 can correct many issues without regenerating trajectories:

explicit target versus observation horizons;
deduplicated density/AUC summaries;
corrected discrete held semantics;
restricted outcomes as primary;

explicit complete-case labels;
pointwise versus simultaneous intervals;
nominal/effective TwindowTwindow;
bundle-transaction output.

It cannot reconstruct unsaved event times or missing endpoint values.
New-observation track

text

Engine v2
Modular runner v2-compatible
Collector v2
Analysis v4+

Engine v2 is needed for:

all-step order-parameter averages;
step-resolution recovery events;
explicit t=0t=0 and t=320t=320 observations;
read-only/cache hardening;
compact native artifact schema;
any changed model/operator semantics.

That would require fully regenerated trajectories and a separately certified baseline.
Pinned authority design

Add an authority lock file stored independently of the run being validated:

JSON

{
"authority_id": "AIEF_1_4_FROZEN_5a9f093a2fe0",
"manifest_sha256": "...",
"npz_sha256": "...",
"scalar_csv_sha256": "...",
"production_source_sha256": "...",
"seed_count": 40,

"branch_count": 7
}

The modular runner should require this lock in production mode and verify:

text

lock → frozen artifacts
frozen manifest → production source and seeds
certificate → engine/spec/execution

The lock itself should be committed to the evidence lineage or otherwise independently authenticated. Merely generating it beside the artifacts at runtime would not create an independent trust anchor.
Compact schema migration

Do not remove legacy outputs from S01 v1. Instead use two collectors:

text

TWindowLegacyCollectorV1
CompactCoreCollectorV2

During migration, a run can emit both:

text

legacy/ exact analysis-v3 compatibility
compact/ future normalized schema

The compact schema should avoid:

scalar duplication inside NPZ;
float-formatted identifiers as primary keys;
one enormous flat namespace;
full in-memory consolidation.

Use condition IDs and structured per-condition groups or files. But do not silently substitute compact output for the legacy oracle contract.
Recommended formal statement

I would revise the bottom line to:

The modular platform is intended to be orchestration-corrected while remaining numerically compatible with Engine v1 and artifact/estimand compatible with Analysis v3. It deliberately preserves legacy numerical schedules, observables, and recovery conventions in S01 v1. Those inherited behaviors are documented limitations, not silently corrected semantics. Orchestration correctness and legacy equivalence remain subject to modular baseline and full-oracle certification.

Once certification passes, change “is intended to be” to “is established to be.”
Immediate sequencing

Freeze the successful monolithic S01 evidence.
Implement and test the modular runner.
Certify modular baseline and full logical oracle equivalence.
Freeze modular S01 v1.
Design Analysis v4 separately.
Design Engine v2 only for semantics that cannot be corrected from existing trajectories.

That prevents orchestration refactoring, analytical corrections, and model changes from becoming entangled in one unverifiable revision.

Think about this"
Stopped

Think about this:

text

I’ve reviewed the architecture and numerical semantics. The engine is thoughtfully designed, but I would **not freeze it as the reference implementation yet**.

What the engine is doing

It implements a delayed adaptive oscillator network with:

- Delayed phase coherence and an EMA memory state
- Adaptive symmetric coupling matrix \(B\)
- A row-capacity feasibility correction
- A common prefix followed by seven intervention branches
- Independent branch RNG namespaces
- Legacy and dual-horizon outcome calculations
- Exact comparison against frozen prefix, trajectory, matrix, and scalar outputs

The separation into `ModelConfig`, `ProtocolConfig`, `DynamicState`, core dynamics, interventions, and validation is good.

Freeze-blocking issues

1. The “frozen” artifacts have no external trust anchor

The validator computes hashes, but never checks them against trusted expected hashes. A modified manifest, NPZ, and CSV could be supplied together and still pass.

Also, the code does not actually enforce:

- Exactly 40 seeds
- The authoritative seed list independent of the manifest
- Exactly seven authoritative branches

It only checks supplied values against the supplied manifest.

For a certification-grade baseline, pin at least the manifest hash in code or a signed release file, and have the manifest contain expected hashes for the NPZ and CSV:

```python
EXPECTED_MANIFEST_SHA256 = "..."

if file_sha256(frozen_manifest_path) != EXPECTED_MANIFEST_SHA256:
return {"status": "failed", "error": "Untrusted frozen manifest"}

if file_sha256(frozen_npz_path) != frozen_manifest["artifacts"]["npz_sha256"]:
```

...

The frozen seed and branch authorities should also be explicit.
2. Empty horizons silently become zero-valued scientific outcomes

When a horizon contains no steps, the code returns a zero matrix:

Python

```
P_ema = {
name: P_sums_ema[name] / counts_ema[name]
if counts_ema[name] > 0
else np.zeros((N, N))
}
```

This makes “not observed” look like “observed persistence was zero.”

This can happen in parameter studies when:

```
future_duration <= 87.5
5 * window_steps >= future_steps
T_window is large relative to the future
```

It then produces quantities such as P_mean_late == 0.0 and potentially retention values of zero, even though there was no late period.

Better choices:

Reject protocols lacking required horizons, or
Return NaN matrices/scalars and explicit availability/count metadata.

For example:

Python

```
def mean_or_nan(total: np.ndarray, count: int) -> np.ndarray:
if count == 0:
```

return np.full_like(total, np.nan, dtype=float)
return total / count

Expose all horizon counts in the result.
3. The declared fixed future horizon of 320 is not implemented

You define:

Python

FIXED_PHYSICAL_FUTURE_END = 320.0

but never use it. Fixed “full” and “late” outcomes extend to whatever protocol.future_duration happens to be.

Therefore:

A 500-unit future includes data after 320.
A 200-unit future silently truncates the estimand.
The result is still called a fixed-physical outcome.

Either enforce a 320-unit future for these estimands or accumulate only through:

Python

fixed_future_end = min(
int(round(FIXED_PHYSICAL_FUTURE_END / config.dt)),
future_steps,
)

Truncation should be reported explicitly.
4. r_mean_* depends on diagnostic_stride

The order parameter is accumulated only inside:

Python

if step % protocol.diagnostic_stride == 0:

Consequently, changing a parameter described as a diagnostics setting changes these scientific outcomes:

r_mean_future
r_mean_late
r_mean_late_fixed_physical
r_mean_late_normalized_EMA

That is undesirable unless the intended estimand is specifically a diagnostic-grid average.

For legacy equivalence, preserve the existing fields. For parameterized studies, add all-step metrics:

Python

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))
r_all_sum += r_value
r_all_count += 1

if step % protocol.diagnostic_stride == 0:
trajectory storage only

A related issue is that the sampled trajectory starts at t=0.01, not intervention time zero, and does not include the final state at t=320. This matters for time-to-event and minimum-similarity interpretation.
5. Exact reproducibility is environment-scoped

Bit-for-bit equality involving these operations is not guaranteed across arbitrary platforms and library versions:

np.sin
np.exp
np.linalg.norm
np.dot
np.corrcoef
default_rng

The validation result should record and enforce, or at least report:

Python version
NumPy version
Platform and machine architecture
BLAS/LAPACK implementation
Bit generator type
Floating-point information

For a true frozen run, distribute a lock file or container image. Otherwise, distinguish:

Bitwise equivalence in the certified environment
Numerical equivalence elsewhere

Important parameterization issues
Strict floating-point divisibility is brittle

Checks such as:

Python

np.isclose(value / dt, round(value / dt), rtol=0.0, atol=0.0)

can reject mathematically compatible decimal inputs because of binary floating-point representation.

This is acceptable for one specifically frozen configuration, but conflicts with the claim that this is a general parameterized engine.

A better design is to make integer step counts authoritative, or use a tightly controlled tolerance and then reconstruct times from steps.
EMA horizon wording does not exactly match the implementation

The code uses:

Python

window_steps = round(T_window / dt)
recovery_end_ema = 5 * window_steps

This is not generally equal to:

Python

round(5 * T_window / dt)

At the baseline:

round(T_window / dt) = 1750
5 * 1750 = 8750
round(5 * T_window / dt) = 8751

The code intentionally matches frozen production, but documentation should call this five discretized EMA windows, not an exact boundary at 5Twindow5Twindow.
“No hidden module globals” is not literally true

Core behavior and metrics still depend on:

FROZEN_PROJECTION_NORM_TOL
FROZEN_ZERO_TOL
FROZEN_GAP_TOL
Literal 17.5 and 87.5
Hard-coded 1e-12 values in metric functions

Some are numerical-policy constants rather than model parameters, but they still affect outputs or validation. They should be collected into an explicit numerical-policy configuration if parameterized studies need auditability.
Configuration immutability is incomplete

ModelConfig is frozen, but Python does not enforce the annotation:

Python

omega: tuple[float, ...]

A caller can pass a mutable list. That list later fails as a cache key.

The cached arrays are also writable:

Python

get_omega_array(...)
compute_edges(...)

External mutation could corrupt all later simulations. Canonicalize inputs and mark cached arrays read-only.
Baseline-validator hardening

Additional issues worth correcting:

Malformed manifest handling catches JSON and I/O errors, but later KeyError, TypeError, and invalid nested structures can still escape.
CSV blank values, commonly produced by pandas for NaN, are not parsed as NaN.
Exact scalar authority should preferably be binary NPZ data, not decimal CSV.
exact_array_equal_with_nan is overly permissive for complex values containing a partially NaN component.
The CSV-row error says “40 seeds × 7 branches” even when inputs differ.
Extra or duplicate NPZ archive entries are not rejected.
frozen_csv_path is missing from the normal result metadata.
Hashes are recorded but, as noted, not verified.

Performance

The full baseline performs:
40×(48,000+7×32,000)=10,880,000
40×(48,000+7×32,000)=10,880,000

Python-level time steps.

Straightforward improvements that should preserve arithmetic order include:

Replace list pop(0) history with a ring buffer.
Remove the redundant clone before simulate_future, since it clones internally.
Avoid accumulating unused B_mean_full/immediate/recovery matrices.

Avoid duplicate fixed/EMA full accumulators.
Replace per-step diagnostic dictionaries with a small tuple or structure.
Parallelize independent seeds using processes while preserving per-seed RNG streams.
Add checkpoint/resume support for the 40-seed certification run.

Recommended design decision

I would separate two modes:

frozen_1_4 compatibility mode
Preserves every legacy boundary, sampled metric, RNG call, and floating-point operation.
Runs only in a pinned environment.
Validates against cryptographically pinned artifacts.

Parameterized study mode
Uses robust step-based configuration.
Treats empty horizons as missing or invalid.
Computes scientific averages independently of diagnostic sampling.
Defines exact horizon and truncation semantics.
Uses stronger runtime and property-based validation.

Trying to make one path simultaneously bit-exact to legacy behavior and scientifically clean for arbitrary parameters is the main architectural tension.
Overall assessment

Strong foundation, not freeze-ready. The dynamics and RNG structure look internally coherent, and the row-capacity correction does enforce feasibility. The main risks are in certification trust, horizon semantics, sampled estimands, and portable exactness—not the central state-update equations.

Actual “40/40” certification cannot be established from this source alone; it requires the frozen manifest, NPZ, CSV, production source, and the original numerical environment.

The critique is technically strong, but its conclusion—“not freeze-ready”—is too broad given what has now actually been demonstrated.

The correct distinction is:

Engine v1 is freeze-ready as a legacy-compatibility reference in the certified environment, but it is not suitable as the scientifically cleaned, general-purpose parameterized engine for arbitrary future studies.

That distinction resolves most of the apparent conflict.
1. What is now established

The statement:

Actual 40/40 certification cannot be established from source alone.

is true in principle, but no longer describes the current evidence state. You have now run the certification:

text

40/40 frozen seeds passed
7/7 frozen branches passed
all required frozen comparisons passed
200/200 T_window conditions succeeded
freezeable = true

Therefore, you can state:

Engine v1, with the certified source hash and current numerical environment, exactly reproduces the frozen AIEF-1.4 finite replay on the complete frozen cohort.

You cannot state:

Engine v1 is bitwise portable across arbitrary systems.

Nor can you state:

Engine v1 defines ideal scientific semantics for arbitrary parameter studies.

Those are different claims.
2. The right freeze classification

I recommend freezing it under a deliberately narrow designation such as:

text

AIEF Experiment Engine v1
Status: legacy-compatibility certified
Authority: AIEF-1.4 frozen finite replay
Environment scope: recorded certified environment
General parameter semantics: not certified

Avoid calling it simply:

text

the canonical AIEF engine

without qualification.

A useful manifest distinction would be:

JSON

```
{
"certification_status": "legacy_compatibility_certified",
"certification_scope": {
"workflow": "AIEF-1.4",
"seed_count": 40,
"branch_count": 7,
"bitwise_environment_scoped": true,
"arbitrary_parameter_semantics_certified": false
}
}
```

3. Evaluation of the freeze-blocking claims
External trust anchor

This is the strongest remaining certification criticism.

The current system verifies internal consistency among:

text

manifest
NPZ
CSV
engine output

and records their hashes. But a coordinated replacement of all supplied authorities could still pass if no independently trusted hash is consulted.

That is an authority authentication issue, not a numerical replay issue.

It can be fixed without changing Engine v1 by creating an independent lock:

JSON

{
"authority_id": "AIEF_1_4_5a9f093a2fe0",
"manifest_sha256": "bf4d8c4152efafac352c8e1e56759ee7867fdd418cb04a8ee1f2fc87c2315a33",
"branch_npz_sha256": "8fa9cc059117a7dbef0ed14e52907ec63f7626b50e4b3c8815977547e4590f0f",
"scalar_csv_sha256": "f4bcdd1403796f0e2fa4f6fbedaee9d79fd5c970b4c33d49ac995b1f0c3bffda"
}

That lock must be stored independently—in source control, the evidence lineage, or a signed release—not generated beside the artifacts during validation.
Seed/branch criticism

The claim that exactly 40 seeds and seven branches are not enforced is now partly outdated:

the engine requires the supplied cohort to equal the manifest cohort;
the production driver generates 40 seeds;
the baseline protocol executes the exact seven manifest branches;
the certificate records 40 seeds and seven branches.

What remains true is that these authorities ultimately derive from the supplied manifest rather than an independently pinned lock. The lock resolves that.

Empty horizons

Correct general criticism.

Engine v1 maps an unavailable horizon to zeros, which conflates:

text

no observations

with:

text

observed scientific value of zero

But it does not affect S01:

future duration is exactly 320;
all five rho levels have nonempty immediate, recovery, and late intervals;
the runner validates the allowed levels and metadata.

Therefore:

text

general Engine v1 hazard: yes
S01 v1 defect: no

For a modular runner, reject incompatible configurations before execution:

Python

if window_steps <= 0:
raise SpecValidationError(...)

```
if 5 * window_steps >= future_steps:
raise SpecValidationError(
"normalized late horizon would be empty"
)

if fixed_recovery_end >= future_steps:
raise SpecValidationError(
"fixed late horizon would be empty"
)
```

Engine v2 should return NaN plus explicit horizon availability/count metadata.
Fixed future endpoint

Correct general criticism.

The engine’s “fixed physical” late interval really means:

text

from 87.5 to the configured end

not intrinsically:

text

from 87.5 to 320

S01 fixes:

text

future_duration = 320

so the implemented and intended estimands coincide.

The modular S01 specification should reject any other future duration. Engine v2 should make endpoint semantics explicit.
Diagnostic-grid r_mean

Correct.

The current r_mean_* fields are averages over saved diagnostic samples, not all simulation steps. They depend on diagnostic_stride.

This does not affect S01's primary outcomes:

text

Weighted_similarity_late_fixed_physical
Retention_contrast_late_fixed_physical
P_mean_late_fixed_physical

Therefore retain the legacy fields unchanged in compatibility output, but rename their semantics in documentation:

text

r_mean_future_diagnostic_grid
r_mean_late_diagnostic_grid

Engine v2 can add:

text

r_mean_future_all_steps
r_mean_late_all_steps

Do not silently substitute all-step values under the old names.
Environment-scoped exactness

Correct and important.

Your successful baseline establishes bitwise equivalence on the tested system/environment. It does not make transcendental or linear algebra operations universally portable.

The frozen certificate should record more than it currently does:

text

Python implementation/version
NumPy version
machine architecture
platform
bit generator
BLAS/LAPACK vendor
thread settings
float information

The thread settings used for production should also be preserved:

text

OMP_NUM_THREADS=2
MKL_NUM_THREADS=2
OPENBLAS_NUM_THREADS=2
NUMEXPR_NUM_THREADS=2

A future release could provide:

text

requirements lock
container image
Nix/Conda lock
environment fingerprint

Outside the certified environment, use two gates:

exact bitwise equivalence;
separately specified numerical-tolerance equivalence.

Do not silently fall back from one to the other.
4. Parameterization criticisms
Strict divisibility

Correct for a general engine, harmless for S01.

Engine v2 should make integer counts authoritative:

Python

n_steps: int
delay_steps: int
burn_end_step: int
split_step: int
future_steps: int

Then derive display times:
tn=n dt.
tn=ndt.

For v1 compatibility, retain existing checks.
Five discretized windows

Correct and worth documenting prominently.

The code defines:
nW=round(Twindow/dt),nrecovery=5nW.
nW=round(Twindow/dt),nrecovery=5nW.

It does not define:
nrecovery=
nrecovery=

\operatorname{round}(5T_{\rm window}/dt).
]

At baseline these are 8750 and 8751 respectively.

Correct terminology:

The normalized recovery horizon ends after five discretized EMA windows.

The analysis should record:

text

T_window_nominal
window_steps
T_window_effective = window_steps * dt
recovery_end_steps = 5 * window_steps

Hidden numerical policies

Correct.

These should become an explicit NumericalPolicyConfig in Engine v2:

Python

@dataclass(frozen=True)
class NumericalPolicyConfig:
zero_tol: float
gap_tol: float
projection_norm_tol: float
scale_epsilon: float
fixed_immediate_time: float
fixed_recovery_time: float
fixed_future_time: float

Do not retrofit this into v1 if it risks changing certified behavior.
Immutability and writable caches

Correct.

Engine v2 should canonicalize:

Python

omega = tuple(float(x) for x in omega)

and make cached arrays read-only:

Python

array.setflags(write=False)

Again, this is hardening rather than an observed S01 failure.
5. Baseline-validator criticism

Most of these are legitimate hardening ideas, but their severity differs.
CSV NaN parsing

The current parser does not silently accept blank NaNs; it fails when converting the field for exact comparison. Since the actual frozen CSV passed all 40 seeds, its NaN representation is compatible.

For a future binary scalar authority, save scalars in NPZ or another typed format. But the existing frozen NPZ does not contain them, so switching authority now would require a new frozen release.
Partially NaN complex values

Correct edge case. The current comparison treats a complex element as NaN if either component is NaN, potentially ignoring the finite component.

This does not affect the actual frozen coherence arrays if they contain no NaNs. Engine v2 should compare real and imaginary parts independently.
Extra NPZ entries

Rejecting extras is useful for a canonical release, but additional harmless keys do not invalidate required logical content. A pinned archive hash makes this issue irrelevant for the frozen authority.
Missing CSV path in result metadata

Minor and already compensated for by the production manifest recording the supplied path and file hash.
6. Performance recommendations

Most are reasonable for v2:

ring buffer/deque history;
remove double clone;
avoid unused accumulators;
avoid duplicate full accumulators;
structured diagnostics instead of dictionaries;
process-level seed parallelism.

However, do not apply them to certified Engine v1. Even changes that should preserve arithmetic can alter:

text

source hash
execution order
environmental behavior
memory behavior
failure modes

For modular platform v1, prioritizing exact compatibility over performance is correct.
Resume support

Resume is not automatically an improvement. Unsafe checkpoint reuse is more dangerous than restarting.

A future resume system must validate each checkpoint against:

text

engine hash
study-spec hash
model-config hash
seed

factor level
branch namespace
schema version
collector version

Until then, durable checkpoints should remain crash diagnostics, not resume authority.
7. The two-mode recommendation is exactly right

This is the strongest part of the critique.
Mode 1: frozen compatibility

text

Engine v1
legacy numerical policy
legacy RNG and horizons
legacy diagnostics
legacy artifact collector
mandatory pinned baseline
certified environment

Purpose:

text

reproduce and compare with historical AIEF-1.4 evidence

Mode 2: scientifically cleaned parameter studies

text

Engine v2
step-authoritative configuration
explicit numerical policies
horizon availability
all-step observables
endpoint-inclusive sampling
typed binary scalar artifacts

compact schema
analysis v4

Purpose:

text

new experiments with corrected semantics

Do not place both modes behind a hidden Boolean that changes meanings while keeping field names.
Use explicit versions and distinct schemas.
8. Analysis v3 status

The critique is right that modular orchestration does not repair Analysis v3.

Several points are limitations rather than outright errors:

one-current-TwindowTwindow hold is intentionally rho-dependent;
pointwise bootstrap intervals are valid as pointwise descriptive intervals;
complete-case raw recovery DID is valid if explicitly labelled conditional.

But they must not be overinterpreted.

For the successful S01 run:

run Analysis v3 because it is the prespecified compatibility analysis;
freeze its results as legacy-compatible evidence;
later run Analysis v4 on the same raw trajectories as a separately versioned reanalysis.

Analysis v4 should prioritize:

restricted/censoring-aware outcomes;
explicit saved-grid event-time semantics;
fixed-duration and normalized-duration hold events;
target definition separate from observation horizon;
observed-support AUC rather than “full 0–320” wording;
deduplicated horizon-independent rows;

explicit undefined-cosine conventions;
simultaneous or adjusted interval families.

The v3 and v4 analyses use the same seed cohort and raw run. They are not independent experiments.
9. Final assessment of the critique

I would revise:

Strong foundation, not freeze-ready.

to:

Strong foundation. Freeze-ready as a certified legacy-compatibility implementation after adding an independent authority lock; not suitable as the final general-purpose scientifically cleaned engine.

That is the accurate status.
Recommended next actions

Preserve Engine v1 unchanged.
Add an independently stored authority lock using the recorded frozen hashes.
Freeze Engine v1 as legacy_compatibility_certified.
Complete modular-runner oracle certification.
Run and freeze Analysis v3 for S01.
Design Analysis v4 separately.
Design Engine v2 only after the v1 evidence lineage is closed.

The central adaptive-network dynamics are not the current concern. The remaining work is versioned semantics, authority authentication, and avoiding silent reuse of legacy field names for corrected estimands.

Then they said:

text

Received. This is the second of the three additional files, so one file remains. I'll continue treating conclusions as provisional.

Integrated understanding

This analysis implements:

- Paired continue-regime contrasts against \(\rho=1\)
- Branch difference-in-differences against the continue branch
- Cross-\(T_{\text{window}}\) final-topology comparisons
- Recovery metrics from saved density and similarity trajectories
- Restricted-time handling for administratively censored recovery
- Seed-level percentile-bootstrap confidence intervals
- Strict validation of simulation hashes, condition coverage, baseline certification, and output counts

The primary/secondary separation is consistent with the simulation driver:

- **Primary:** fixed-physical late horizons
- **Secondary:** normalized-EMA late horizons

The regime-contrast and ordinary branch-DID formulas are implemented correctly.

Important provisional findings

1. The held-duration calculation is one sample interval short

The implementation uses:

```python
required_samples = int(np.ceil(duration / sample_step))
```

However, kk instantaneous samples span only (k−1)Δt(k−1)Δt. For example, 35 samples at 0.5 spacing span 17.0 time units, not 17.5.

If “held for at least T_window” is intended literally, it needs either:

Python

required_samples = int(np.ceil(duration / sample_step)) + 1

or, preferably, a timestamp-based condition:

Python

times[j] - times[i] >= duration

The current implementation is internally documented as a discrete convention, but that convention does not match the function's "at least the specified duration" description.
2. Recovery "horizon systems" do not actually restrict the recovery horizon

Every recovery metric is evaluated over the complete saved trajectory from 0.01 through 319.51.

The horizon label changes only the similarity target:

Fixed target from fixed-physical late similarity
Normalized target from normalized-EMA late similarity

For density, even the target is identical across horizon systems. Therefore, these are more accurately target systems, not distinct recovery observation horizons.
3. Several recovery results are duplicated

The code explicitly duplicates all density recovery rows across:

fixed_physical
normalized_EMA

In addition, similarity AUC is also identical across those labels because it does not use the horizon-specific similarity target:

Python

sim_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_similarity_trajectory, similarity_values
)

Consequently:

Density recovery outputs are duplicated wholesale.
Density recovery DID outputs are duplicated.
Similarity AUC and its DID are duplicated.
The expected 624 recovery-DID groups include these duplicate estimands.

The density duplication is documented, but the similarity-AUC duplication is not equally explicit.
4. “Full future” AUC is not over the complete 320-unit future

The integration grid runs from:

text

0.01 through 319.51

Thus the AUC omits:

The intervention state at time zero
The interval from 319.51 through 320.0

Calling this auc_horizon="full_future" is slightly inaccurate. It is the AUC over the saved diagnostic trajectory.

Likewise, “sustained for the rest of the trajectory” means the rest of the saved trajectory, not the final unsampled 0.49 time units.
5. The NaN recovery convention is not fully consistent

The documentation says undefined cosine values are imputed as zero for recovery and AUC.

AUC does that explicitly. Event detection instead uses:

Python

np.isfinite(values) & (values >= threshold)

That treats NaN as categorically below the threshold, not necessarily as zero. The distinction matters if the target is zero or negative.

Either the implementation or the stated convention should be made consistent.
6. Conditional recovery DID uses a selected complete-case cohort

For outcomes such as:

first_crossing_over_Twindow
sustained_time_over_Twindow
held_time_over_Twindow

a DID is finite only when all four component recovery events are observed:
Rρ,b,Rρ,c,R1,b,R1,c.
Rρ,b,Rρ,c,R1,b,R1,c.

The resulting group can therefore be a highly selected subset of seeds, potentially different for every rho, branch, metric, and outcome. The code correctly permits n<40n<40, but reports should state that these are four-cell complete-case conditional DIDs, not cohort-level effects.

The restricted-time and recovery-indicator DIDs are the stronger cohort-level estimands.
7. The held-event definition changes with rho

Because the required hold duration is T_window, a held event means something different at each rho:

Approximately 8.75 units at ρ=0.5ρ=0.5
Approximately 17.5 at ρ=1ρ=1
Approximately 35.0 at ρ=2ρ=2

A DID of held times across rho therefore mixes:

Changes in recovery behavior
Changes in the event definition itself

That may be intentional for scaling analysis, but a parallel fixed-duration held metric would provide a cleaner physical-time comparison.
8. Bootstrap intervals are pointwise, not simultaneous

The seed-level bootstrap is appropriate for marginal group means because pairing has already been incorporated into the contrasts. However, hundreds of pointwise 95% intervals are produced, including 624 recovery-DID groups.

The report should explicitly label these as unadjusted pointwise intervals. If inferential claims will be made across the full family, multiplicity control or prespecified primary comparisons are needed.

A joint seed bootstrap—resampling the same 40 seeds across all rho and branches—would also support simultaneous trend bands and preserve cross-condition covariance.
9. The analysis confirms that scalar duplication in the NPZ is unnecessary

This file reads scalar outcomes exclusively from the CSV. From the NPZ it only needs:

B_final
sample_times
density_trajectory
similarity_trajectory

Therefore, the simulation driver can stop writing every branch scalar into the NPZ while retaining the existing flat array-key convention expected here.
10. Analysis output replacement is not transactional

With --overwrite, old analysis outputs are deleted before Stages 2–5 execute. If a later calculation fails, the prior valid analysis has already been removed and the new output directory may be partial.

A safer design is:

Write the entire analysis into a temporary sibling directory.
Validate all expected outputs and hashes.
Atomically rename or promote the completed directory.

Positive assessment

The validation layer is strong in several respects:

Exact cohort and condition coverage
Duplicate detection
Simulation-output hash verification
Baseline-certificate inspection
Strict topology-matrix validation
Explicit full-cohort versus conditional recovery counts

Reproducible bootstrap seed
Atomic individual-file writes with directory fsync

I’ll incorporate this with the engine and study driver after receiving the final project file.

This second review is substantially correct. It identifies analysis-layer semantics rather than faults in the adaptive-network dynamics. None of these points invalidates the completed S01 trajectories or its three prespecified primary scalar outcomes, but they limit how recovery and interval results should be described.
Claim-by-claim assessment
Finding Assessment
Held duration is one saved interval short Correct if interpreted as physical elapsed duration
“Horizon systems” only change targets Correct
Density recovery is duplicated Correct
Similarity AUC is duplicated Correct
“Full future” AUC covers only saved support Correct
NaN handling differs by derived metric Correct, though harmless for positive targets
Conditional recovery DID is complete-case selected Correct and important
Held-event definition changes with rho Correct and partly intentional
Bootstrap intervals are pointwise Correct
Scalar NPZ duplication is unnecessary for v3 Correct, except for legacy compatibility
Analysis replacement is not bundle-transactional Correct
1. Held-duration semantics

The mathematical criticism is correct:
k saved samples span $(k-1)\Delta t$.
k saved samples span $(k-1)\Delta t$.

The current implementation uses:
$k=\lceil T_{window}\Delta t_{sample}\rceil$.
$k=\lceil \Delta t_{sample}T_{window}\rceil$.

Therefore, the elapsed span is:
$(k-1)\Delta t_{sample}<T_{window}$
$(k-1)\Delta t_{sample}<T_{window}$

in general.

One numerical detail in the review’s example should be corrected. At baseline the actual value is:

Twindow=17.502917…
Twindow=17.502917…

and the diagnostic interval is 0.50.5, so the implementation requires:
⌈17.502917/0.5⌉=36
⌈17.502917/0.5⌉=36

samples, not 35. Those 36 samples span:
(36−1)(0.5)=17.5,
(36−1)(0.5)=17.5,

which is still slightly shorter than 17.502917…17.502917….

The timestamp-based definition is preferable:

Python

```
for i in range(len(times)):
for j in range(i, len(times)):
if times[j] - times[i] >= duration:
if np.all(condition[i : j + 1]):
return float(times[i]), True
break
```

A more efficient implementation can use a moving right endpoint, but the semantic condition should be:

Python

```
times[j] - times[i] >= duration
```

For legacy v3, preserve the existing field and rename its meaning as a consecutive-saved-sample event. Analysis v4 should add the timestamp-corrected event under a new field name.
2. “Horizon system” is misleading terminology

Correct.

The current recovery calculation always searches:

text

0.01 through 319.51

The two labels affect the similarity target:

text

fixed-physical late target
normalized-EMA late target

They do not create distinct observation windows.

The data model should instead separate:

text

target_system
observation_support
censoring_horizon
hold_duration_system
AUC_support

For v3 compatibility:

text

target_system = fixed_physical | normalized_EMA
observation_support = full_saved_future

Renaming alone would improve interpretation without changing results.
3. Duplicate recovery estimands

Correct.

Density

Every density quantity is duplicated across the two horizon labels because neither the trajectory nor target changes.
Similarity

Similarity event metrics may differ because their thresholds differ, but similarity AUC is duplicated because it does not depend on either threshold.

For the 624 recovery-DID groups, duplicated group pairs include:

156 redundant density groups;
12 redundant similarity-AUC groups.

Thus the 624 rows represent 456 unique estimand groups if these exact horizon-label duplicates are collapsed:
624−156−12=456.
624−156−12=456.

This does not make the table numerically wrong, but duplicated rows must not be pooled, counted as separate evidence, or included twice in multiplicity accounting.

Analysis v4 should use a normalized long schema where horizon-independent metrics have one row per condition rather than duplicated labels.
4. AUC support

Correct.

The integration support is:
[0.01,319.51],
[0.01,319.51],

with total elapsed integration width:
319.51−0.01=319.5.
319.51−0.01=319.5.

It omits:

text

[0, 0.01)
(319.51, 320]

The current engine does not save enough information to reconstruct an exact diagnostic-series AUC over [0,320][0,320] without imposing endpoint assumptions.

Analysis v4 can correct the wording using:

text

AUC over saved diagnostic support
support_start = 0.01
support_end = 319.51

A genuinely endpoint-complete AUC requires Engine/Collector v2 to save the intervention-time and terminal observations.

The same limitation applies to “sustained for rest of trajectory”: it means sustained through the final saved observation, not necessarily through t=320t=320.
5. Undefined-cosine convention

The reviewer is correct that there are two operations:
Recovery events

Python

np.isfinite(values) & (values >= threshold)

NaN is classified as not recovered.
Recovery AUC

Python

NaN -> 0

For the actual S01 similarity targets, which should be positive, these conventions usually have the same qualitative effect. But they are not mathematically identical, especially if a target is zero or negative.

A clean v4 convention would explicitly create:

Python

```
similarity_for_recovery = np.nan_to_num(
similarity_raw,
nan=0.0,
)
```

and then use that same array for:

crossing;
sustained;
held;
AUC.

The raw saved trajectory must remain NaN. The imputed array is a derived analysis representation only.

Alternatively, retain “undefined means unrecovered” for events and zero imputation for AUC, but document them as two distinct conventions rather than one unified convention.
6. Complete-case conditional recovery DID

This is one of the most important findings.

For raw event-time DID:
[Rρ,b−Rρ,c]−
[Rρ,b−Rρ,c]−

[R_{1,b}-R_{1,c}],
]

all four event times must be observed. The resulting estimate is conditional on a group-specific subset:

text

seed recovered in rho branch
and rho continue
and baseline branch
and baseline continue

The selected seed set can differ across every:

text

rho
branch
horizon/target system
metric
event definition

Therefore, these are not cohort-level effects.

Reports should include:

text

n complete cases
fraction of 40 seeds
exact complete-case seed list or its hash

The inferential hierarchy should be:
Primary recovery interaction estimands

restricted recovery-time DID;
recovery-indicator DID;
AUC DID.

These retain all 40 seeds.
Secondary conditional estimands

raw first-crossing DID;
raw sustained-time DID;
raw held-time DID.

These are explicitly four-cell complete-case analyses.
7. Rho-dependent held event

Correct, but this is intentional for normalized scaling.

At different rho:
Thold=ρT∗.
Thold=ρT∗.

Thus held-recovery probability itself changes definition across rho. A DID then combines:

changed trajectory behavior;
changed required hold duration.

Analysis v4 should compute both:
Normalized hold
Thold=Twindow(ρ)
Thold=Twindow(ρ)
Fixed physical hold

For example:
Thold=17.5
Thold=17.5

at every rho.

The normalized version asks whether recovery persists for one current memory horizon. The fixed version asks a cleaner physical-time counterfactual.

Neither should silently replace the other.
8. Bootstrap intervals

Correct.

The current intervals are valid as marginal pointwise intervals for seed-level quantities. They do not provide joint family coverage across hundreds of outcomes.

Label them:

text

unadjusted pointwise percentile-bootstrap 95% intervals

For v4, use one joint seed bootstrap:

Python

```
for bootstrap_iteration:
sampled_seed_indices = rng.integers(
0,
40,
size=40,
)

# Reuse these same sampled seeds for every
# rho, branch, metric, and outcome.
```

This preserves cross-condition covariance and permits:

simultaneous max-deviation bands;
trend bands across rho;
familywise primary-outcome intervals;
coherent covariance estimates.

Multiplicity adjustment should focus on prespecified primary families rather than indiscriminately adjusting every exploratory metric.
9. Scalar duplication in NPZ

Correct.

Analysis v3 uses the CSV for scalar outcomes and needs only these NPZ arrays:

text

B_final
sample_times
density_trajectory
similarity_trajectory

It also needs prefix or other matrices only if future analyses request them.

For legacy collector v1, preserve scalar duplication until modular oracle equivalence is certified.

For compact collector v2:

text

CSV/Parquet: scalar table
NPZ/Zarr/HDF5: arrays only

Do not change the existing legacy contract in place.
10. Analysis transactionality

Correct.

Each output file is atomic, but the analysis directory is not committed atomically as a set.

A robust analysis runner should use:

text

analysis_attempt_<execution_id>.tmp/

Workflow:

create fresh staging directory;
write every output;

validate expected files, row counts, and hashes;
write final analysis manifest last;
fsync files and staging directory;
atomically rename staging directory to its final execution directory;
never delete a previous valid analysis before the replacement is complete.

An even simpler policy is never to overwrite analyses:

text

analysis/v3/<analysis_execution_id>/
analysis/v4/<analysis_execution_id>/

Then a rebuildable catalog identifies successful analyses.
Implications for the modular platform

These findings reinforce the need to record the analysis contract separately:

JSON

{
"analysis_contract": {
"analysis_id": "AIEF_1_4_TWINDOW_ANALYSIS_V3",
"semantics": "legacy_compatible",
"target_systems": [
"fixed_physical",
"normalized_EMA"
],
"observation_support": [
0.01,
319.51
],
"held_definition": "consecutive_saved_samples",
"interval_type": "pointwise_unadjusted_percentile_bootstrap"
}
}

The modular runner should not claim to fix any of these analysis semantics merely because it improves orchestration.

Recommended version path
Preserve and run v3

Run Analysis v3 because it is the prespecified compatibility analysis. Freeze it with explicit limitations.
Add Analysis v4 as a reanalysis

Analysis v4 can use the existing S01 trajectories to add:

timestamp-corrected held events;
fixed-duration and one-current-window holds;
target-system terminology;
deduplicated horizon-independent outcomes;
saved-support AUC terminology;
unified NaN convention;
restricted/indicator primary recovery DIDs;
complete-case seed-set reporting;
joint seed bootstrap;
transactional output directories.

Engine v2 only where necessary

Engine v2 is needed for:

t=0t=0 observations;
t=320t=320 observations;
step-resolution event times;
all-step order parameter means;
endpoint-complete AUC.

Bottom line

This review is accurate and should be integrated into the known-limitations registry.

It does not invalidate:

the frozen baseline reproduction;
the 200 regenerated trajectories;
the fixed-physical primary scalar outcomes;

ordinary paired regime contrasts;
ordinary branch DIDs;
cross-TT topology.

It does constrain interpretation of:

held recovery;
recovery “horizon” labels;
duplicated recovery rows;
AUC support;
undefined-cosine handling;
conditional raw recovery DIDs;
large families of pointwise intervals.

The right response is versioning, not silent correction:

text

Analysis v3 = prespecified legacy-compatible analysis
Analysis v4 = corrected recovery/inference semantics

Here is the pipe line please evaluate it:

init :

text

""AIEF Modular Experiment Driver Platform v1.

This package orchestrates certified engine-v1 studies without reimplementing
numerical dynamics. Built-in studies are registered explicitly via
``aief_modular.registry.load_builtin_studies``.
"""

from .registry import REGISTRY, load_builtin_studies
from .runner import RUNNER_VERSION

```
__all__ = ["REGISTRY", "RUNNER_VERSION", "load_builtin_studies"]
__version__ = RUNNER_VERSION
```

artifacts.py :

```
text
```

```
"""Atomic artifact I/O, execution paths, hashing, and write locks."""

from __future__ import annotations

import csv
import datetime as dt
import hashlib
import json
import math
import os
import platform
import secrets
import socket
import tempfile
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable, Mapping, Sequence

import numpy as np

from .spec import ExperimentSpec
from .validation import ArtifactIntegrityError, ExecutionLockError



def file_sha256(path: str | os.PathLike[str]) -> str:
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(lambda: handle.read(1024 * 1024), b""):
digest.update(block)
return digest.hexdigest()
```

```
def _fsync_directory(directory: Path) -> None:
flags = os.O_RDONLY | getattr(os, "O_DIRECTORY", 0)
descriptor = os.open(str(directory), flags)
try:
os.fsync(descriptor)
finally:
os.close(descriptor)
```

```
def _temp_path(destination: Path, suffix: str = ".tmp") -> tuple[int, Path]:
destination.parent.mkdir(parents=True, exist_ok=True)
descriptor, name = tempfile.mkstemp(
dir=str(destination.parent),
prefix=f".{destination.name}.",
suffix=suffix,
)
return descriptor, Path(name)
```

```
def _replace_and_sync(temporary: Path, destination: Path) -> None:
os.replace(temporary, destination)
_fsync_directory(destination.parent)
```

```
def json_safe(value: Any) -> Any:
if isinstance(value, Mapping):
return {str(key): json_safe(item) for key, item in value.items()}
if isinstance(value, (tuple, list)):
return [json_safe(item) for item in value]
if isinstance(value, np.ndarray):
return json_safe(value.tolist())
if isinstance(value, (np.integer, np.floating, np.bool_)):
return json_safe(value.item())
if isinstance(value, float):
return value if math.isfinite(value) else None
if isinstance(value, (str, int, bool)) or value is None:
return value
return str(value)
```

```
def atomic_write_text(path: str | Path, text: str) -> None:
destination = Path(path)
descriptor, temporary = _temp_path(destination)
try:
with os.fdopen(descriptor, "w", encoding="utf-8", newline="") as handle:
handle.write(text)
handle.flush()
os.fsync(handle.fileno())
_replace_and_sync(temporary, destination)
finally:
if temporary.exists():
temporary.unlink()



def atomic_write_json(path: str | Path, value: Any) -> None:
text = json.dumps(json_safe(value), indent=2, sort_keys=True, allow_nan=False) + "\n"
atomic_write_text(path, text)



def atomic_write_csv(
path: str | Path,
rows: Sequence[Mapping[str, Any]],
fieldnames: Sequence[str] | None = None,
) -> None:
destination = Path(path)
if fieldnames is None:
if not rows:
raise ArtifactIntegrityError("Cannot infer CSV fields from an empty row set")
fieldnames = tuple(rows[0])
descriptor, temporary = _temp_path(destination)
try:
with os.fdopen(descriptor, "w", encoding="utf-8", newline="") as handle:
writer = csv.DictWriter(handle, fieldnames=list(fieldnames), extrasaction="raise")
writer.writeheader()
writer.writerows(rows)
handle.flush()
os.fsync(handle.fileno())
_replace_and_sync(temporary, destination)
```

```
finally:
if temporary.exists():
temporary.unlink()


def atomic_write_npz(path: str | Path, payload: Mapping[str, Any]) -> None:
destination = Path(path)
descriptor, temporary = _temp_path(destination, suffix=".tmp.npz")
try:
with os.fdopen(descriptor, "wb") as handle:
np.savez_compressed(handle, **payload)
handle.flush()
os.fsync(handle.fileno())
_replace_and_sync(temporary, destination)
finally:
if temporary.exists():
temporary.unlink()


def load_npz(path: str | Path) -> dict[str, Any]:
payload: dict[str, Any] = {}
try:
with np.load(path, allow_pickle=False) as archive:
if len(archive.files) != len(set(archive.files)):
raise ArtifactIntegrityError(f"NPZ contains duplicate keys: {path}")
for key in archive.files:
value = archive[key]
payload[key] = value.item() if value.ndim == 0 else value.copy()
except ArtifactIntegrityError:
raise
except Exception as exc:
raise ArtifactIntegrityError(f"Cannot load NPZ {path}: {exc}") from exc
return payload


def utc_now_text() -> str:
return dt.datetime.now(dt.timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")
```

```
def make_execution_id(mode: str) -> str:
if mode not in {"baseline", "production"}:
raise ValueError(f"Invalid execution mode: {mode}")
timestamp = dt.datetime.now(dt.timezone.utc).strftime("%Y%m%dT%H%M%SZ")
return f"{mode}_{timestamp}_{secrets.token_hex(4)}"



@dataclass(frozen=True)
class ExecutionPaths:
leaf: Path
manifest: Path
failed_manifest: Path
baseline_certificate: Path
baseline_by_seed: Path
condition_status: Path
scalar_csv: Path
branch_npz: Path
conditions: Path
logs: Path
analysis: Path
failure: Path
lock_file: Path



def execution_paths(
output_root: str | Path,
spec: ExperimentSpec,
run_id: str,
mode: str,
execution_id: str,
) -> ExecutionPaths:
root = Path(output_root)
study_directory = f"{spec.study_id}_{spec.study_slug}"
leaf = root / study_directory / spec.study_version / run_id / mode / execution_id
prefix = spec.artifact_policy.scalar_filename_prefix
scalar_name = f"{prefix}_{run_id}_scalar_summary.csv"
array_name = f"{spec.artifact_policy.array_filename_prefix}_{run_id}_branch_diagnostics.npz"
return ExecutionPaths(
leaf=leaf,
manifest=leaf / "manifest.json",
failed_manifest=leaf / "FAILED_manifest.json",
```

```
baseline_certificate=leaf / "baseline_equivalence_certificate.json",
baseline_by_seed=leaf / "baseline_equivalence_by_seed.json",
condition_status=leaf / "twindow_condition_status.csv",
scalar_csv=leaf / scalar_name,
branch_npz=leaf / array_name,
conditions=leaf / "conditions",
logs=leaf / "logs",
analysis=leaf / "analysis",
failure=leaf / "failure",
lock_file=leaf / ".execution.lock",
)



def create_execution_directory(paths: ExecutionPaths) -> None:
if paths.leaf.exists():
raise ArtifactIntegrityError(
f"Execution directory already exists and overwrite is unsupported: {paths.leaf}"
)
paths.leaf.mkdir(parents=True, exist_ok=False)
for directory in (paths.conditions, paths.logs, paths.analysis, paths.failure):
directory.mkdir()
_fsync_directory(paths.leaf.parent)



class ExecutionLock:
"""Atomic create-only lock. Existing locks are never stolen."""


def __init__(self, path: str | Path):
self.path = Path(path)
self._held = False


def acquire(self) -> None:
record = {
"pid": os.getpid(),
"host": socket.gethostname(),
"started_utc": utc_now_text(),
}
try:
descriptor = os.open(
self.path,
```

os.O_WRONLY | os.O_CREAT | os.O_EXCL,
0o644,
)
except FileExistsError as exc:
raise ExecutionLockError(
f"Execution lock already exists: {self.path}. "
"Stale locks require explicit manual investigation and removal."
) from exc
try:
payload = (json.dumps(record, sort_keys=True) + "\n").encode("utf-8")
os.write(descriptor, payload)
os.fsync(descriptor)
finally:
os.close(descriptor)
_fsync_directory(self.path.parent)
self._held = True

def release(self) -> None:
if self._held and self.path.exists():
self.path.unlink()
_fsync_directory(self.path.parent)
self._held = False

def __enter__(self) -> "ExecutionLock":
self.acquire()
return self

def __exit__(self, exc_type: Any, exc: Any, traceback: Any) -> None:
self.release()

Branches.py

text

"""Engine-v1 branch compatibility mappings and RNG namespace handling."""

from __future__ import annotations

from typing import Any

```
from .spec import BranchSpec, ExperimentSpec
from .validation import SpecValidationError, UnsupportedPolicyError



FROZEN_ENGINE_V1_NAMESPACE = (
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
)

SUPPORTED_EXECUTED_BRANCHES: dict[str, dict[str, Any]] = {
"continue": {
"role": "control",
"intervention_ids": (),
"forcing_policy_id": None,
},
"reset_ema": {
"role": "state_intervention",
"intervention_ids": ("engine_v1.reset_ema",),
"forcing_policy_id": None,
},
"reset_B": {
"role": "state_intervention",
"intervention_ids": ("engine_v1.reset_B",),
"forcing_policy_id": None,
},
"future_shuffle": {
"role": "dynamic_forcing",
"intervention_ids": (),
"forcing_policy_id": "engine_v1.future_shuffle",
},
}



def validate_branch_compatibility(spec: ExperimentSpec) -> None:
```

```
if tuple(spec.branch_rng_namespace) != FROZEN_ENGINE_V1_NAMESPACE:
raise UnsupportedPolicyError(
"The v1 T_window implementation requires the complete frozen seven-branch RNG namespace"
)
for branch in spec.branches:
expected = SUPPORTED_EXECUTED_BRANCHES.get(branch.engine_branch)
if expected is None:
raise UnsupportedPolicyError(
f"Engine-v1 modular runner does not execute branch {branch.engine_branch!r}"
)
if branch.name != branch.engine_branch:
raise UnsupportedPolicyError(
"Aliased branch names are not supported by the legacy compatibility collector"
)
for field in ("role", "intervention_ids", "forcing_policy_id"):
if getattr(branch, field) != expected[field]:
raise SpecValidationError(
f"Branch {branch.name!r} {field}={getattr(branch, field)!r}; "
f"expected {expected[field]!r}"
)



def make_branch_rngs(engine: Any, seed: int, namespace: tuple[str, ...]) -> dict[str, Any]:
"""Delegate RNG creation to the certified engine using the full namespace."""
if tuple(namespace) != FROZEN_ENGINE_V1_NAMESPACE:
raise UnsupportedPolicyError("Cannot shorten or reorder the engine-v1 RNG namespace")
rngs = engine.make_branch_rngs(int(seed), tuple(namespace))
if tuple(rngs) != tuple(namespace):
raise RuntimeError(
f"Engine returned RNG mapping order {tuple(rngs)!r}, expected {tuple(namespace)!r}"
)
return rngs



def branch_rng_child_index(spec: ExperimentSpec, engine_branch: str) -> int:
try:
return spec.branch_rng_namespace.index(engine_branch)
except ValueError as exc:
raise KeyError(engine_branch) from exc
```

catalog.py

text

"""Rebuildable, non-authoritative experiment catalog."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from .artifacts import atomic_write_csv, atomic_write_json
from .validation import CatalogError

CATALOG_FIELDS = (
"study_id",
"study_slug",
"study_version",
"run_id",
"full_spec_sha256",
"execution_id",
"execution_mode",
"created_utc",
"status",
"freezeable",
"engine_sha256",
"runner_sha256",
"seed_count",
"condition_count",
"path",
"malformed_error",
)

def _catalog_row(root: Path, manifest_path: Path) -> dict[str, Any]:
relative = manifest_path.parent.relative_to(root).as_posix()

```
try:
with open(manifest_path, "r", encoding="utf-8") as handle:
manifest = json.load(handle)
source_hashes = manifest.get("source_hashes", {})
completion = manifest.get("completion", {})
row = {
"study_id": manifest["study_id"],
"study_slug": manifest["study_slug"],
"study_version": manifest["study_version"],
"run_id": manifest["run_id"],
"full_spec_sha256": manifest["spec_sha256"],
"execution_id": manifest["execution_id"],
"execution_mode": manifest["execution_mode"],
"created_utc": manifest["created_utc"],
"status": manifest.get("status", completion.get("status", "unknown")),
"freezeable": bool(manifest.get("freezeable", False)),
"engine_sha256": source_hashes.get("engine", "MISSING"),
"runner_sha256": source_hashes.get("runner", "MISSING"),
"seed_count": int(manifest.get("study", {}).get("n_seeds", 0)),
"condition_count": int(completion.get("n_conditions_requested", 0)),
"path": relative,
"malformed_error": "",
}
except Exception as exc:
row = {key: "" for key in CATALOG_FIELDS}
row.update(
{
"status": "malformed",
"freezeable": False,
"path": relative,
"malformed_error": f"{type(exc).__name__}: {exc}",
}
)
return row



def rebuild_catalog(output_root: str | Path, *, replace: bool = False) -> tuple[Path, Path]:
root = Path(output_root)
if not root.exists():
raise CatalogError(f"Output root does not exist: {root}")
csv_path = root / "catalog.csv"
json_path = root / "catalog.json"
```

```
if not replace and (csv_path.exists() or json_path.exists()):
raise CatalogError(
"Catalog output already exists; pass --replace-catalog to rebuild it explicitly"
)


manifests = sorted(
path
for path in root.rglob("manifest.json")
if path.parent.name != "analysis"
)
rows = [_catalog_row(root, path) for path in manifests]
rows.sort(
key=lambda row: (
str(row["study_id"]),
str(row["study_version"]),
str(row["run_id"]),
str(row["execution_id"]),
str(row["path"]),
)
)


seen: dict[tuple[str, str, str, str], str] = {}
for row in rows:
if row["status"] == "malformed":
continue
identity = (
str(row["study_id"]),
str(row["study_version"]),
str(row["run_id"]),
str(row["execution_id"]),
)
if identity in seen:
raise CatalogError(
f"Duplicate catalog execution identity {identity!r}: "
f"{seen[identity]} and {row['path']}"
)
seen[identity] = str(row["path"])


atomic_write_csv(csv_path, rows, CATALOG_FIELDS)
atomic_write_json(
json_path,
```

```
{
"catalog_schema": "aief-catalog-v1",
"authority": "derived_non_authoritative",
"entries": rows,
"malformed_count": sum(row["status"] == "malformed" for row in rows),
},
)
return csv_path, json_path
```

collectors.py

text

```
"""Raw-observable collectors for modular experiment execution.

Collectors deliberately perform no seed-level inference, bootstrap, or DID
analysis. The T_window compatibility collector preserves the successful
production driver's flat CSV/NPZ contract.
"""

from __future__ import annotations

from typing import Any, Protocol

import numpy as np

from .factors import Condition
from .spec import CollectorSpec
from .validation import ConditionSchemaError, validate_prefix_outputs,
validate_twindow_branch_output



class Collector(Protocol):
collector_id: str
schema_version: str
core_schema_version: str
study_schema_version: str
```

```
def validate_engine_result(
self,
*,
engine: Any,
condition: Condition,
model: Any,
protocol: Any,
prefix: dict[str, Any],
branch_results: dict[str, dict[str, Any]],
) -> None: ...

def scalar_rows(
self,
*,
condition: Condition,
branch_results: dict[str, dict[str, Any]],
) -> list[dict[str, Any]]: ...

def array_payload(
self,
*,
condition: Condition,
model: Any,
prefix: dict[str, Any],
branch_results: dict[str, dict[str, Any]],
) -> dict[str, Any]: ...



FROZEN_STUDY_SCALAR_KEYS = (
"D_B_final",
"P_mean_full",
"P_mean_immediate",
"P_mean_recovery",
"P_mean_late",
"Weighted_similarity_final",
"Weighted_similarity_late",
"Jaccard_final",
"Retention_selected_late",
"Emergence_unselected_late",
"Retention_contrast_late",
"Corr_Rpast_Pfull",
```

```
"Corr_Rpast_Plate",
"Total_edge_weight",
"Mean_row_sum",
"Max_row_sum",
"Nonzero_edge_fraction",
"Effective_edges",
"Weight_gini",
"Weight_entropy",
"r_mean_future",
"r_mean_late",
"Minimum_similarity",
"G_min_future",
"G_min_pre_future",
"Pre_gap_violation_fraction",
"Projection_active_fraction",
"Projection_active_given_violation",
"Projection_norm_mean_active",
"Projection_norm_max",
"Projection_relative_state_mean_active",
"Projection_relative_state_max",
"Projection_relative_update_mean_active",
"Projection_relative_update_max",
)

EXTENDED_STUDY_SCALAR_KEYS = (
"P_mean_late_fixed_physical",
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
"Corr_Rpast_Plate_fixed_physical",
"r_mean_late_fixed_physical",
"P_mean_late_normalized_EMA",
"Weighted_similarity_late_normalized_EMA",
"Retention_contrast_late_normalized_EMA",
"Corr_Rpast_Plate_normalized_EMA",
"r_mean_late_normalized_EMA",
)

REQUIRED_STUDY_SCALAR_KEYS = FROZEN_STUDY_SCALAR_KEYS +
EXTENDED_STUDY_SCALAR_KEYS
PRIMARY_OUTCOMES = (
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
```

```
"P_mean_late_fixed_physical",
)
SECONDARY_OUTCOMES = (
"Weighted_similarity_late_normalized_EMA",
"Retention_contrast_late_normalized_EMA",
"P_mean_late_normalized_EMA",
)



class TWindowLegacyCollectorV1:
collector_id = "aief.twindow.legacy-flat"
schema_version = "v1"
core_schema_version = "aief-engine-v1-raw-v1"
study_schema_version = "aief-1.4-s01-twindow-v1"


@staticmethod
def _rho(condition: Condition) -> float:
return float(condition.value("rho"))


def validate_engine_result(
self,
*,
engine: Any,
condition: Condition,
model: Any,
protocol: Any,
prefix: dict[str, Any],
branch_results: dict[str, dict[str, Any]],
) -> None:
validate_prefix_outputs(prefix, model)
if tuple(branch_results) != condition.expected_branches:
raise ConditionSchemaError(
f"Condition {condition.condition_id} branch set/order {tuple(branch_results)!r} "
f"!= {condition.expected_branches!r}"
)
rho = self._rho(condition)
# Direct simulate_future() does not yet contain the eight historical
# cross-branch fields added by finalize_historical_branch_metrics().
# It must, however, contain every other frozen scalar and every extended
# scalar exposed by engine v1, including fields not selected for the CSV.
extended_engine_keys = tuple(getattr(engine, "EXTENDED_BRANCH_SCALAR_KEYS", ()))
```

```
direct_engine_scalar_keys = FROZEN_STUDY_SCALAR_KEYS + extended_engine_keys
required_engine_scalars = tuple(dict.fromkeys(direct_engine_scalar_keys +
REQUIRED_STUDY_SCALAR_KEYS))
for branch, result in branch_results.items():
validate_twindow_branch_output(
engine=engine,
branch=branch,
result=result,
model=model,
protocol=protocol,
rho=rho,
required_scalar_keys=required_engine_scalars,
primary_outcomes=PRIMARY_OUTCOMES,
secondary_outcomes=SECONDARY_OUTCOMES,
)


def scalar_rows(
self,
*,
condition: Condition,
branch_results: dict[str, dict[str, Any]],
) -> list[dict[str, Any]]:
rho = self._rho(condition)
rows: list[dict[str, Any]] = []
for branch, result in branch_results.items():
row: dict[str, Any] = {
"seed_index": condition.seed_index,
"seed": condition.seed,
"rho": rho,
"T_window": result["T_window"],
"branch": branch,
}
for key in REQUIRED_STUDY_SCALAR_KEYS:
if key not in result:
raise ConditionSchemaError(f"{branch} missing scalar {key}")
row[key] = result[key]
rows.append(row)
return rows


def array_payload(
self,
*,
```

```
condition: Condition,
model: Any,
prefix: dict[str, Any],
branch_results: dict[str, dict[str, Any]],
) -> dict[str, Any]:
"""Preserve the successful driver's complete flat NPZ logical contract.

Although this is called an array payload, the oracle driver also placed
zero-dimensional scalar metadata in the NPZ. Those entries are retained
for strict workflow compatibility in v1.
"""
seed = condition.seed
rho = self._rho(condition)
rho_text = str(rho) # legacy key formatting used by analysis_v3
payload: dict[str, Any] = {
f"seed_{seed}_rho_{rho_text}_config_T_window": model.T_window,
f"seed_{seed}_rho_{rho_text}_config_alpha": model.alpha,
}
for key, value in prefix.items():
payload[f"prefix_seed_{seed}_rho_{rho_text}_{key}"] = value
for branch, result in branch_results.items():
for key, value in result.items():
payload[f"{branch}_seed_{seed}_rho_{rho_text}_{key}"] = value
return payload


def scalar_fieldnames(self) -> tuple[str, ...]:
return (
"seed_index",
"seed",
"rho",
"T_window",
"branch",
*REQUIRED_STUDY_SCALAR_KEYS,
)


def checkpoint_filename(self, condition: Condition) -> str:
rho = self._rho(condition)
return f"condition_seed_{condition.seed}_rho_{rho:.2f}.npz"
```

```
def build_collector(spec: CollectorSpec) -> Collector:
if spec.collector_id != TWindowLegacyCollectorV1.collector_id:
raise ConditionSchemaError(f"Unknown collector ID: {spec.collector_id}")
expected_versions = (
TWindowLegacyCollectorV1.schema_version,
TWindowLegacyCollectorV1.core_schema_version,
TWindowLegacyCollectorV1.study_schema_version,
)
observed_versions = (
spec.schema_version,
spec.core_schema_version,
spec.study_schema_version,
)
if observed_versions != expected_versions:
raise ConditionSchemaError(
f"Collector schema mismatch: spec={observed_versions!r}, implementation={expected_versions!r}"
)
return TWindowLegacyCollectorV1()
```

The next files will be:

factors.py oracle.py runner.py registry.py spec.py validation.py

This is a strong start. The package is following the requested architecture rather than merely wrapping the old driver in another monolith. The artifact hierarchy, explicit RNG namespace, compatibility collector, atomic writes, locking, and rebuildable catalog are all directionally correct.

This assessment is provisional until the remaining files are reviewed, especially runner.py, spec.py, factors.py, oracle.py, and validation.py.
Immediate compile/import checks
1. Verify the package initializer docstring

The pasted initializer begins:

Python

```
""AIEF Modular Experiment Driver Platform v1.
```

rather than:

Python

"""AIEF Modular Experiment Driver Platform v1.

If that is literal, it is a syntax error. If it is paste corruption, ignore it.
2. Linux filename case matters

The heading says:

text

Branches.py

but the package imports:

Python

from .branches import ...

On the Steam Deck’s case-sensitive filesystem:

text

Branches.py != branches.py

The actual filename must be:

text

aief_modular/branches.py

Likewise, the initializer must be:

text

aief_modular/__init__.py

These are P0 import requirements.
artifacts.py
Positive assessment

The following are good:

temp files are created in the destination directory;
file contents are flushed and fsynced;
replacement is atomic;
parent directories are fsynced;
NPZ loading uses allow_pickle=False;
execution directories are create-only;
baseline and production executions have separate paths;
execution IDs include mode, timestamp, and random suffix;
existing execution directories are never overwritten;
locks are acquired with O_EXCL;
stale locks are not silently stolen;
baseline, condition, analysis, failure, and log paths are structurally separated.

P0: json_safe() is too permissive for authority manifests

This fallback is dangerous:

Python

return str(value)

It silently converts unsupported objects to strings. Examples include:

sets;
dataclasses not explicitly serialized;
Path;
complex values;
arbitrary plugin objects;
enum instances;

functions.

This can hide a malformed manifest and make canonical scientific content depend on arbitrary repr/string behavior.

There is also a possible key collision here:

Python

```
return {
str(key): json_safe(item)
for key, item in value.items()
}
```

For example:

Python

```
{1: "a", "1": "b"}
```

collapses to one JSON key.
Required design

Separate strict production JSON from diagnostic-safe JSON.

Python

```
def strict_json_value(value: Any) -> Any:
if isinstance(value, Mapping):
result = {}
for key, item in value.items():
if not isinstance(key, str):
raise ArtifactIntegrityError(
f"JSON mapping key must be str, got "
f"{type(key).__name__}"
)
if key in result:
raise ArtifactIntegrityError(
```

```
f"Duplicate JSON key after normalization: {key}"
)
result[key] = strict_json_value(item)
return result


if isinstance(value, (tuple, list)):
return [strict_json_value(item) for item in value]


if isinstance(value, np.ndarray):
return strict_json_value(value.tolist())


if isinstance(value, (np.integer, np.floating, np.bool_)):
return strict_json_value(value.item())


if isinstance(value, float):
if not math.isfinite(value):
raise ArtifactIntegrityError(
"Non-finite float in strict production JSON"
)
return value


if isinstance(value, (str, int, bool)) or value is None:
return value


raise ArtifactIntegrityError(
f"Unsupported JSON value type: {type(value).__name__}"
)
```

Then provide a separately named function for failure diagnostics:

Python

```
diagnostic_json_safe(...)
```

Only failure reports should convert nonfinite values to None or unsupported values to explicit diagnostic strings.

Production manifests, specifications, identity payloads, catalogs, and completion records should use strict serialization.
P1: CSV writing permits missing fields

This catches extra fields:

Python

extrasaction="raise"

but missing fields are silently written as blanks.

For authoritative tables, validate every row:

Python

expected = set(fieldnames)

for index, row in enumerate(rows):
observed = set(row)

missing = expected - observed
extra = observed - expected

if missing or extra:
raise ArtifactIntegrityError(
f"CSV row {index} schema mismatch: "
f"missing={sorted(missing)}, "
f"extra={sorted(extra)}"
)

Also require unique field names:

Python

if len(fieldnames) != len(set(fieldnames)):
raise ArtifactIntegrityError(

"CSV fieldnames contain duplicates"
)

If condition-status rows legitimately have optional failure columns, define an explicit nullable schema and populate successful rows with empty/None values. Do not rely on accidental missing dictionaries.
P1: generic output paths contain a hardcoded study name

This line is not generic:

Python

condition_status=leaf / "twindow_condition_status.csv"

Every future study would inherit a T-window-specific filename.

Move this into ArtifactPolicySpec, for example:

Python

condition_status_filename: str
baseline_certificate_filename: str
baseline_by_seed_filename: str
manifest_filename: str

For S01 compatibility:

text

condition_status_filename = twindow_condition_status.csv

Other studies can use their own compatible names or a generic:

text

condition_status.csv

The analysis script currently expects twindow_condition_status.csv, so S01’s legacy artifact policy must preserve it.
P1: validate all path components

Before using:

Python

study_id
study_slug
study_version
run_id
mode
execution_id
filename prefixes

require safe path components. Reject:

text

/
..
leading/trailing whitespace
empty strings
path separators
control characters

Recommended pattern:

Python

^[A-Za-z0-9][A-Za-z0-9._-]*$

run_id should be restricted to lowercase hexadecimal of the expected length.

This validation likely belongs in spec.py and validation.py, but execution_paths() should not assume unsafe values are impossible without an assertion.
P1: execution-directory creation is not itself transactional

If one subdirectory creation fails after leaf.mkdir(), a partial execution directory remains.

That is acceptable if the runner:

catches the failure;
writes structured failure state where possible;
never reuses the directory;
catalogues it as malformed/incomplete.

Do not attempt silent cleanup if evidence may already have been written.
Lock scope

The lock protects one execution leaf, which is appropriate. It does not prevent two separate execution IDs for the same scientific run from running simultaneously. That appears intentional.

Record this explicitly:

text

lock scope = execution_id
not run_id

branches.py
Positive assessment

This module correctly distinguishes:

text

full frozen RNG namespace
executed study branch subset

It also correctly preserves:

text

future_shuffle child index = 6

rather than assigning it index 3.

Good features:

exact seven-branch namespace;
explicit supported mappings;
no aliased legacy branch names;
branch role validation;
intervention/forcing descriptor validation;
engine delegation for RNG generation;
RNG dictionary order check.

P0/P1: requires_rng is not validated

BranchSpec apparently contains:

Python

requires_rng

but SUPPORTED_EXECUTED_BRANCHES does not specify or validate it.

Add:

Python

"continue": {
...
"requires_rng": False,
},
"reset_ema": {
...
"requires_rng": False,
},

"reset_B": {
...
"requires_rng": False,
},
"future_shuffle": {
...
"requires_rng": True,
},

Then include requires_rng in the validation loop.

Clarify the semantics:

every branch may receive an RNG object because the engine API requires one;
requires_rng=True means the branch is expected to consume randomness under the declared protocol.

A future audit can verify that non-random branches do not advance their RNG streams if that is part of the contract.
P1: validate executed branch order explicitly

validate_branch_compatibility() validates each branch but should also require the S01 execution order:

Python

```
expected_executed = (
"continue",
"reset_ema",
"reset_B",
"future_shuffle",
)

observed = tuple(branch.engine_branch for branch in spec.branches)

if observed != expected_executed:
raise SpecValidationError(...)
```

This may already occur in spec.py; verify it once the remaining files arrive.
catalog.py

Positive assessment

The catalog is correctly treated as:

text

derived
rebuildable
non-authoritative

Other strengths:

deterministic manifest scanning;
deterministic row sorting;
malformed manifest rows preserved and marked;
duplicate execution identities rejected;
atomic CSV and JSON writes;
explicit replacement flag;
logical source hashes and completion data included.

P1: validate catalog field types

This line can be wrong for malformed but parseable manifests:

Python

"freezeable": bool(manifest.get("freezeable", False))

For example:

JSON

{"freezeable": "false"}

becomes Python True.

Require:

Python

```
freezeable = manifest.get("freezeable", False)
if not isinstance(freezeable, bool):
raise CatalogError(
"Manifest freezeable field must be boolean"
)
```

Apply similar validation to:

study identifiers;
execution mode;
seed/condition counts;
source hashes;
run ID;
spec hash.

Malformed but parseable manifests should become status="malformed".
P1: verify path agrees with manifest identity

The catalog should check that the directory hierarchy agrees with:

text

```
study_id + slug
study_version
run_id
execution mode
execution_id
```

Otherwise, a copied manifest in the wrong directory can catalogue misleadingly.

For example, derive expected relative path:

Python

```
expected = (
Path(f"{study_id}_{study_slug}")
/ study_version
/ run_id
/ execution_mode
/ execution_id
)
```

Compare with manifest_path.parent.relative_to(root).
Catalog pair transactionality

CSV and JSON are individually atomic but not promoted as a pair. Because the catalog is non-authoritative and rebuildable, this is acceptable. Record a generation ID in both outputs if you want to detect mismatched catalog pairs after interruption.
collectors.py
Positive assessment

This is well aligned with the intended separation:

no bootstrap or DID logic;
condition-level data only;
legacy T-window scalar schema preserved;
complete flat NPZ contract preserved;
legacy rho key formatting preserved;
branch order is checked;
prefix and branch outputs are delegated to validators;
all direct engine extended scalars are checked;
current CSV field order is preserved;
checkpoint filename matches the old driver.

The distinction between:

text

condition-local frozen scalars
historical cross-branch scalars

is correctly understood. The modular condition runner should not add the eight historical baseline-only metrics to ordinary S01 condition results.

P1: rename FROZEN_STUDY_SCALAR_KEYS

This tuple contains the 34 condition-local historical fields, not all 42 frozen baseline fields.

A clearer name would be:

Python

CONDITION_LOCAL_LEGACY_SCALAR_KEYS

That prevents future developers from assuming it is the complete frozen baseline schema.
P1: pin the expected engine extended schema

This currently discovers:

Python

engine.EXTENDED_BRANCH_SCALAR_KEYS

dynamically.

Because the collector is a compatibility collector, it should have an explicit expected engine schema and verify equality:

Python

```
EXPECTED_ENGINE_EXTENDED_SCALAR_KEYS = (
...
)

observed = tuple(
getattr(engine, "EXTENDED_BRANCH_SCALAR_KEYS", ())
)

if observed != EXPECTED_ENGINE_EXTENDED_SCALAR_KEYS:
raise ConditionSchemaError(
```

"Engine extended scalar schema differs from "
"collector compatibility contract"
)

Otherwise, adding an unrelated extended field to a future engine module silently changes what this collector requires.

The engine hash also protects this, but explicit schema validation gives a clearer failure.
P1: collector-construction error type

An unknown collector ID or collector-version mismatch is a specification/compatibility error, not a condition-result error.

Prefer:

Python

UnsupportedPolicyError

or:

Python

SpecValidationError

instead of ConditionSchemaError.
P1: validate scalar value shape/type

Before adding a field to a CSV row, require it to be scalar:

Python

value = result[key]
array = np.asarray(value)

if array.ndim != 0:

raise ConditionSchemaError(
f"{branch} scalar {key} has shape {array.shape}"
)

Permit expected NaNs such as projection diagnostics where legacy behavior allows them. Do not require every legacy scalar to be finite.
Logical oracle compatibility

The legacy NPZ behavior is correctly preserved:

Python

for key, value in result.items():
payload[...] = value

That includes duplicated scalars, but this is appropriate for TWindowLegacyCollectorV1.

A compact collector should be introduced separately rather than changing this one.
Cross-module concerns to check in the remaining files

The remaining modules must demonstrate the following.
spec.py

immutable dataclasses;
strict type/value validation;
safe path components;
canonical serialization without arbitrary string fallback;
no absolute paths/timestamps/environment in scientific identity;
exact S01 branch order;
exact level IDs and ordering;
collector and artifact-policy versions;
reference rho level;
full regeneration requirement.

factors.py

exact seed generation;
exact condition order;

exact rho -> T_window mapping;
immutable condition objects;
unique condition IDs;
no split-state reuse;
full-precision numeric metadata.

oracle.py

independently pinned authority lock support;
frozen baseline execution;
logical oracle comparison;
exact NaN-aware arrays and scalars;
no compressed-byte comparison for regenerated NPZ;
exact condition/branch coverage;
source/spec/collector hash linkage.

runner.py

imports engine, never copies its step map;
correct baseline-only behavior;
condition transaction ordering;
lock acquired before mutable writes;
durable status writes;
typed failures;
failure manifest;
no message-string fatality heuristic;
no unsafe resume;
full output count validation;
final hashes written before freezeable status;
nonzero exit on failure.

registry.py

explicit built-in registration;
no duplicate IDs/slugs;
deterministic list/describe;
no untrusted dynamic imports;
idempotent load_builtin_studies().

validation.py

typed exceptions;
strict manifest/spec validation;
complete S01 shape/range/time checks;
NaN-aware scalar/array equality;
finite similarity range with permitted NaN;
density in [0,1];
horizon availability;
exact expected counts;
no silent coercions.

Provisional verdict
Architecture

Strong and aligned with the design brief.
Shown implementation

Promising, but not yet approved for execution.
Immediate blockers to check/fix

Ensure __init__.py has a valid triple-quoted docstring.
Ensure the actual filename is lowercase branches.py.
Replace permissive manifest json_safe() behavior for production authority JSON.
Move the T-window-specific condition-status filename into the artifact policy.
Validate requires_rng.
Validate CSV rows for missing as well as extra fields.

Once the remaining modules are available, the most important question will be whether runner.py preserves numerical/oracle behavior while correctly committing conditions and final artifacts.

factors.py

text

"""Typed model-factor resolution and deterministic condition expansion."""

from __future__ import annotations

```
import dataclasses
import hashlib
import importlib
import itertools
import math
from dataclasses import dataclass
from typing import Any

from .spec import ExperimentSpec, FactorLevel, ModelFactorSpec, canonical_json, sha256_canonical
from .validation import SpecValidationError, UnsupportedPolicyError


SUPPORTED_RESOLVERS = frozenset({"engine_v1.twindow_scale"})
SUPPORTED_REGENERATION_POLICY = "full_from_initialization"


@dataclass(frozen=True)
class Condition:
condition_id: str
seed: int
seed_index: int
factor_levels: tuple[tuple[str, FactorLevel], ...]
factor_values: tuple[tuple[str, float], ...]
model_config_hash: str
protocol_hash: str
expected_branches: tuple[str, ...]

def level(self, factor_name: str) -> FactorLevel:
for name, level in self.factor_levels:
if name == factor_name:
return level
raise KeyError(factor_name)

def value(self, factor_name: str) -> float:
return self.level(factor_name).value
```

```
def validate_factor_compatibility(axis: ModelFactorSpec) -> None:
if axis.regeneration_policy != SUPPORTED_REGENERATION_POLICY:
raise UnsupportedPolicyError(
f"Factor {axis.name!r} requests unsupported regeneration policy "
f"{axis.regeneration_policy!r}"
)
if axis.resolver_id not in SUPPORTED_RESOLVERS:
raise UnsupportedPolicyError(
f"Factor resolver {axis.resolver_id!r} is not implemented by modular platform v1"
)



def resolve_model_config(base_model: Any, selections: tuple[tuple[ModelFactorSpec,
FactorLevel], ...]) -> Any:
"""Create a fresh immutable engine ModelConfig for selected factor levels."""
model = base_model
for axis, level in selections:
validate_factor_compatibility(axis)
if axis.resolver_id == "engine_v1.twindow_scale":
rho = float(level.value)
if not math.isfinite(rho) or rho <= 0.0:
raise SpecValidationError(f"rho must be finite and positive, got {rho!r}")
try:
t_star = float(model.T_star)
except Exception as exc:
raise SpecValidationError("Base model does not expose finite T_star") from exc
t_window = rho * t_star
model = dataclasses.replace(model, T_window=t_window)
else: # defensive: validate_factor_compatibility should already reject it
raise UnsupportedPolicyError(axis.resolver_id)


if model is base_model and selections:
raise RuntimeError("Factor resolver failed to create a fresh model configuration")
validate = getattr(model, "validate", None)
if not callable(validate):
raise SpecValidationError("Resolved model config has no validate() method")
validate()
return model
```

```
def reference_selections(spec: ExperimentSpec) -> tuple[tuple[ModelFactorSpec, FactorLevel], ...]:
selected: list[tuple[ModelFactorSpec, FactorLevel]] = []
for axis in spec.model_axes:
selected.append(
(
axis,
next(level for level in axis.levels if level.level_id == axis.reference_level_id),
)
)
return tuple(selected)



def reference_model_config(spec: ExperimentSpec) -> Any:
return resolve_model_config(spec.base_model, reference_selections(spec))



def model_config_hash(model: Any) -> str:
return sha256_canonical(model)



def protocol_hash(spec: ExperimentSpec) -> str:
return sha256_canonical(spec.protocol)



def _condition_id(seed: int, selections: tuple[tuple[ModelFactorSpec, FactorLevel], ...]) -> str:
suffix = "__".join(level.level_id for _axis, level in selections)
return f"seed_{seed}__{suffix}" if suffix else f"seed_{seed}"



def expand_conditions(spec: ExperimentSpec) -> tuple[Condition, ...]:
"""Expand in exact seed-major, declared-factor-level order."""
for axis in spec.model_axes:
validate_factor_compatibility(axis)

level_products = tuple(itertools.product(*(axis.levels for axis in spec.model_axes)))
conditions: list[Condition] = []
seen_ids: set[str] = set()
p_hash = protocol_hash(spec)
```

```
expected_branches = tuple(branch.engine_branch for branch in spec.branches)


for seed_index, seed in enumerate(spec.seed_design.paired_seeds):
for level_tuple in level_products:
selections = tuple(zip(spec.model_axes, level_tuple, strict=True))
model = resolve_model_config(spec.base_model, selections)
condition_id = _condition_id(seed, selections)
if condition_id in seen_ids:
raise SpecValidationError(f"Duplicate condition ID: {condition_id}")
seen_ids.add(condition_id)
conditions.append(
Condition(
condition_id=condition_id,
seed=int(seed),
seed_index=seed_index,
factor_levels=tuple((axis.name, level) for axis, level in selections),
factor_values=tuple((axis.name, float(level.value)) for axis, level in selections),
model_config_hash=model_config_hash(model),
protocol_hash=p_hash,
expected_branches=expected_branches,
)
)

if len(conditions) != spec.artifact_policy.expected_condition_count:
raise SpecValidationError(
f"Expanded {len(conditions)} conditions, expected "
f"{spec.artifact_policy.expected_condition_count}"
)
return tuple(conditions)



def selections_for_condition(
spec: ExperimentSpec,
condition: Condition,
) -> tuple[tuple[ModelFactorSpec, FactorLevel], ...]:
by_name = dict(condition.factor_levels)
return tuple((axis, by_name[axis.name]) for axis in spec.model_axes)



def model_for_condition(spec: ExperimentSpec, condition: Condition) -> Any:
```

model = resolve_model_config(spec.base_model, selections_for_condition(spec, condition))
observed = model_config_hash(model)
if observed != condition.model_config_hash:
raise SpecValidationError(
f"Condition {condition.condition_id} model hash changed: "
f"expanded={condition.model_config_hash}, resolved={observed}"
)
return model

factors.py is well structured and correctly narrow for modular platform v1. It supports only the certified T_window factor, resolves fresh immutable model configurations, and expands conditions in the required seed-major order.
Provisional verdict

S01 compatibility: strong
Deterministic expansion: correct
Full-regeneration declaration: correct
Actual full-regeneration enforcement: must be verified in runner.py
No immediate P0 blocker for S01
General multifactor extensibility: needs additional validation before claiming broader support

Positive assessment
Exact condition order

This correctly produces:

text

seed 0, rho 0.5
seed 0, rho 0.75
seed 0, rho 1.0
seed 0, rho 1.5
seed 0, rho 2.0
seed 1, rho 0.5
...

because:

Python

for seed_index, seed in enumerate(...):
for level_tuple in level_products:

and itertools.product() preserves declared axis and level order.
Correct TwindowTwindow resolution

The resolver uses:

Python

t_window = rho * model.T_star
model = dataclasses.replace(
model,
T_window=t_window,
)

This matches the successful production driver.
Fresh configuration object

dataclasses.replace() creates a new ModelConfig, preventing mutation of the base configuration.
Explicit compatibility limits

These are good:

Python

SUPPORTED_RESOLVERS = {
"engine_v1.twindow_scale"
}
SUPPORTED_REGENERATION_POLICY = (
"full_from_initialization"
)

Unsupported future factors fail instead of silently changing engine behavior.
Deterministic hashes and IDs

Each condition records:

text

model config hash
protocol hash
expected branch order
factor level metadata
seed and seed index

That is sufficient to detect many forms of condition drift.
Important runner dependency
Full regeneration is declared here, not enforced here

This file validates:

Python

axis.regeneration_policy
== "full_from_initialization"

but it cannot ensure fresh trajectory initialization.

runner.py must prove that, for every condition, it performs:

Python

split_state, prefix = engine.simulate_prefix(
condition.seed,
model_for_condition(spec, condition),
protocol,
)

It must not cache or reuse:

text

split_state
prefix
coherence_acc
history
B
theta

across factor levels.

This is the most important cross-file check remaining.
P1 corrections
1. Expected condition count should not live in ArtifactPolicySpec

Current code checks:

Python

spec.artifact_policy.expected_condition_count

Condition count is a design property, not an artifact-naming property.

Prefer one of:

Python

spec.design.expected_condition_count

or derive it directly:

Python

derived_count = (
len(spec.seed_design.paired_seeds)
* math.prod(
len(axis.levels)
for axis in spec.model_axes
)

)

If the specification contains a prespecified count, compare it against the derived count during spec validation.

For v1 compatibility this coupling does not alter results, but it weakens separation of concerns.
2. Restrict modular v1 to exactly one supported axis

Each individual axis is validated, but two axes could both request:

Python

resolver_id="engine_v1.twindow_scale"

They would be applied sequentially, with the latter effectively replacing the earlier T_window. That would create misleading multifactor conditions.

For platform v1 require:

Python

if len(spec.model_axes) != 1:
raise UnsupportedPolicyError(
"Modular platform v1 supports exactly one "
"model axis"
)

axis = spec.model_axes[0]

if axis.name != "rho":
raise UnsupportedPolicyError(...)

if axis.resolver_id != "engine_v1.twindow_scale":
raise UnsupportedPolicyError(...)

This may belong in the S01 study validator or spec.py.

Do not claim generic crossed-factor support merely because itertools.product() is present.
3. Validate T∗T∗ explicitly before multiplication

Current code converts:

Python

t_star = float(model.T_star)

but does not explicitly check it before using it.

model.validate() should eventually reject nonfinite T_window, but produce a clearer error:

Python

if not math.isfinite(t_star) or t_star <= 0.0:
raise SpecValidationError(
f"Base model T_star must be finite and "
f"positive, got {t_star!r}"
)

Likewise validate:

Python

t_window = rho * t_star

if not math.isfinite(t_window) or t_window <= 0.0:
raise SpecValidationError(...)

4. Convert dataclasses.replace() errors to typed spec errors

If the base model is not a compatible dataclass, this currently raises a raw TypeError.

Use:

Python

```
try:
model = dataclasses.replace(
model,
T_window=t_window,
)
except TypeError as exc:
raise SpecValidationError(
"Base model is not a compatible frozen "
"ModelConfig dataclass"
) from exc
```

This keeps failure classification consistent.
5. Harden reference-level resolution

This can raise an uninformative StopIteration:

Python

```
next(
level
for level in axis.levels
if level.level_id == axis.reference_level_id
)
```

Use:

Python

```
matches = [
level
for level in axis.levels
if level.level_id == axis.reference_level_id
]

if len(matches) != 1:
raise SpecValidationError(
```

f"Factor {axis.name!r} requires exactly one "
f"reference level {axis.reference_level_id!r}; "
f"found {len(matches)}"
)

spec.py should already reject this, but local defensive validation is useful.
6. Validate reconstructed condition levels against the spec

This function:

Python

by_name = dict(condition.factor_levels)

can silently collapse duplicate factor names. It also permits a FactorLevel object that has the correct factor name but is not one of the factor’s declared levels.

Use explicit checks:

Python

if len(condition.factor_levels) != len(
spec.model_axes
):
raise SpecValidationError(...)

names = [
name
for name, _level in condition.factor_levels
]

if len(names) != len(set(names)):
raise SpecValidationError(
f"Condition {condition.condition_id} has "
f"duplicate factor names"
)

Then for each axis:

Python

```
declared = {
level.level_id: level
for level in axis.levels
}

condition_level = by_name.get(axis.name)

if condition_level is None:
raise SpecValidationError(...)

expected = declared.get(condition_level.level_id)

if expected is None or condition_level != expected:
raise SpecValidationError(
f"Condition {condition.condition_id} uses "
f"undeclared or modified level "
f"{condition_level!r}"
)
```

This matters if conditions are ever deserialized from checkpoints or manifests.
7. Include factor names in condition IDs

Current IDs concatenate only level_id:

Python

```
suffix = "__".join(
level.level_id
for _axis, level in selections
)
```

For S01, rho_0p50 is probably already descriptive. For future safety, make the encoding explicit:

Python

suffix = "__".join(
f"{axis.name}_{level.level_id}"
for axis, level in selections
)

If level IDs already include the axis name, this can become redundant. Define one canonical rule in spec.py and test it. Do not allow arbitrary level IDs to contain path separators.

Condition IDs should remain identifiers, not authorities for recovering numeric values. The code correctly stores exact factor values separately.
8. Decouple Condition from mutable nested metadata

Condition is frozen, but that does not guarantee that FactorLevel.metadata or other nested objects are immutable.

spec.py should canonicalize metadata into immutable tuples or a read-only mapping. Otherwise a caller can mutate metadata after expansion while retaining the same Condition object.

The condition hash should derive from canonical serialized values rather than Python object identity.
Cleanup

These imports appear unused:

Python

hashlib
importlib
canonical_json

Remove them unless the unfinished code will use them.

Line-length cleanup is optional.
Required tests for factors.py
S01 expansion order

Python

conditions = expand_conditions(spec)

assert len(conditions) == 200

assert [
conditions[i].value("rho")
for i in range(5)
] == [0.5, 0.75, 1.0, 1.5, 2.0]

assert conditions[0].seed == first_seed
assert conditions[5].seed == second_seed

Condition IDs

Python

assert conditions[0].condition_id == (
f"seed_{first_seed}__rho_0p50"
)

Adjust to the chosen canonical naming rule.
Model hashes

All seeds at the same rho should share the same model hash:

Python

for rho in rho_values:
hashes = {
condition.model_config_hash
for condition in conditions
if condition.value("rho") == rho
}
assert len(hashes) == 1

Different rho levels should have distinct model hashes:

Python

assert len({
condition.model_config_hash
for condition in conditions
}) == 5

Reference configuration

Python

reference = reference_model_config(spec)

assert reference.T_window == spec.base_model.T_star

Prefer an exact comparison because rho is exactly 1.0.
Reconstruction

Python

for condition in conditions:
reconstructed = model_for_condition(
spec,
condition,
)
assert (
model_config_hash(reconstructed)
== condition.model_config_hash
)

Unsupported factors

Test rejection of:

unknown resolver;
non-full regeneration;

rho zero;
negative rho;
NaN rho;
infinity rho;
multiple T_window axes.

Determinism

Call expand_conditions(spec) twice and require complete object equality and identical canonical serialization.
Provisional conclusion

factors.py is suitable for S01 after the surrounding spec enforces:

text

exactly one rho axis
five declared levels
rho=1 reference
full regeneration
40 exact paired seeds
200 expected conditions

The implementation correctly resolves the current T_window factor. The main remaining scientific requirement is in runner.py: every condition must regenerate its prefix from initialization, with no cross-level state reuse.

runner.py

text

"""Generic modular experiment runner for certified engine-v1 studies."""

from __future__ import annotations

import importlib
import json
import os

```
import platform
import sys
import time
import traceback
from pathlib import Path
from typing import Any, Mapping


import numpy as np


from .artifacts import (
ExecutionLock,
ExecutionPaths,
atomic_write_csv,
atomic_write_json,
atomic_write_npz,
atomic_write_text,
create_execution_directory,
execution_paths,
file_sha256,
make_execution_id,
utc_now_text,
)
from .branches import branch_rng_child_index, make_branch_rngs, validate_branch_compatibility
from .collectors import TWindowLegacyCollectorV1, build_collector
from .factors import (
Condition,
expand_conditions,
model_for_condition,
reference_model_config,
validate_factor_compatibility,
)
from .registry import StudyRegistration
from .spec import (
ExperimentSpec,
canonical_data,
canonical_json,
scientific_identity_payload,
scientific_run_sha256,
spec_sha256,
validate_experiment_spec,
)
from .validation import (
```

```
ArtifactIntegrityError,
BaselineGateError,
ConditionSchemaError,
SpecValidationError,
UnsupportedPolicyError,
)



RUNNER_VERSION = "v1.0"
RUN_ID_LENGTH = 16



def _module_source_hash(module: Any) -> str:
path_value = getattr(module, "__file__", None)
if not path_value:
raise ArtifactIntegrityError(f"Module {module.__name__!r} has no source file")
path = Path(path_value)
if path.suffix == ".pyc" and path.with_suffix(".py").exists():
path = path.with_suffix(".py")
return file_sha256(path)



def compute_source_hashes(
spec: ExperimentSpec,
registration: StudyRegistration,
engine: Any,
) -> dict[str, str]:
"""Hash all execution and plugin sources under stable logical names."""
modules = {
"engine": engine,
"runner": importlib.import_module("aief_modular.runner"),
"spec": importlib.import_module("aief_modular.spec"),
"registry": importlib.import_module("aief_modular.registry"),
"factors": importlib.import_module("aief_modular.factors"),
"branches": importlib.import_module("aief_modular.branches"),
"collectors": importlib.import_module("aief_modular.collectors"),
"validation": importlib.import_module("aief_modular.validation"),
"artifacts": importlib.import_module("aief_modular.artifacts"),
"study": importlib.import_module(registration.source_module),
}
```

```
cli_path = Path(__file__).resolve().parent.parent / "AIEF_modular_driver_v1.py"
hashes = {name: _module_source_hash(module) for name, module in modules.items()}
if not cli_path.is_file():
raise ArtifactIntegrityError(f"CLI source not found: {cli_path.name}")
hashes["cli"] = file_sha256(cli_path)
return dict(sorted(hashes.items()))



def build_engine_protocol(engine: Any, spec: ExperimentSpec, branches: tuple[str, ...]) -> Any:
protocol = engine.ProtocolConfig(
T_sim=spec.protocol.T_sim,
burn_end_time=spec.protocol.burn_end_time,
split_time=spec.protocol.split_time,
future_duration=spec.protocol.future_duration,
diagnostic_stride=spec.protocol.diagnostic_stride,
branches=tuple(branches),
)
return protocol



def validate_engine_compatibility(engine: Any, spec: ExperimentSpec) -> None:
validate_experiment_spec(spec)
validate_branch_compatibility(spec)
for axis in spec.model_axes:
validate_factor_compatibility(axis)
required_api = (
"ModelConfig",
"ProtocolConfig",
"simulate_prefix",
"simulate_future",
"make_branch_rngs",
"run_baseline_equivalence",
"REQUIRED_BRANCH_MATRIX_KEYS",
)
missing = [name for name in required_api if not hasattr(engine, name)]
if missing:
raise UnsupportedPolicyError(f"Engine module missing required API: {missing}")

reference = reference_model_config(spec)
reference.validate()
```

```
protocol = build_engine_protocol(
engine,
spec,
tuple(branch.engine_branch for branch in spec.branches),
)
protocol.validate(reference)



def dry_run(registration: StudyRegistration) -> dict[str, Any]:
"""Validate and compute identities without creating an output directory."""
spec = registration.build()
engine = importlib.import_module(spec.engine_module)
validate_engine_compatibility(engine, spec)
source_hashes = compute_source_hashes(spec, registration, engine)
conditions = expand_conditions(spec)
identity_sha = scientific_run_sha256(spec, source_hashes)
return {
"status": "validated",
"study_id": spec.study_id,
"study_slug": spec.study_slug,
"study_version": spec.study_version,
"spec_sha256": spec_sha256(spec),
"scientific_identity_sha256": identity_sha,
"run_id": identity_sha[:RUN_ID_LENGTH],
"seed_count": spec.seed_design.count,
"condition_count": len(conditions),
"scalar_row_count": spec.artifact_policy.expected_scalar_row_count,
"branch_rng_namespace": list(spec.branch_rng_namespace),
"executed_branches": [branch.engine_branch for branch in spec.branches],
"future_shuffle_rng_child_index": branch_rng_child_index(spec, "future_shuffle"),
"collector_schemas": [
{
"collector_id": item.collector_id,
"schema_version": item.schema_version,
"core_schema_version": item.core_schema_version,
"study_schema_version": item.study_schema_version,
}
for item in spec.collectors
],
"source_hashes": source_hashes,
}
```

```
def _model_manifest(spec: ExperimentSpec) -> dict[str, Any]:
model = spec.base_model
return {
"N": int(model.N),
"dt": float(model.dt),
"tau": float(model.tau),
"omega": [float(value) for value in model.omega],
"a": float(model.a),
"q": float(model.q),
"delta": float(model.delta),
"eta": float(model.eta),
"mu": float(model.mu),
"gamma": float(model.gamma),
"B_c": float(model.B_c),
"B_init_scale": float(model.B_init_scale),
"T_star": float(model.T_star),
"scale_epsilon": float(model.scale_epsilon),
}



def _initial_manifest(
*,
spec: ExperimentSpec,
spec_hash: str,
scientific_hash: str,
run_id: str,
execution_id: str,
execution_mode: str,
source_hashes: Mapping[str, str],
paths: ExecutionPaths,
) -> dict[str, Any]:
rho_axis = next(axis for axis in spec.model_axes if axis.name == "rho")
rho_values = [float(level.value) for level in rho_axis.levels]
twindow_values = [float(level.metadata["T_window"]) for level in rho_axis.levels]
collector = spec.collectors[0]
return {
"manifest_schema": "aief-modular-execution-manifest-v1",
"runner_version": RUNNER_VERSION,
"study_id": spec.study_id,
"study_slug": spec.study_slug,
```

```
"study_version": spec.study_version,
"study_name": "AIEF_1_4_Twindow_sensitivity",
"title": spec.title,
"purpose": spec.purpose,
"spec_sha256": spec_hash,
"scientific_identity_sha256": scientific_hash,
"run_id": run_id,
"execution_id": execution_id,
"execution_mode": execution_mode,
"created_utc": utc_now_text(),
"execution_provenance": {
"pid": os.getpid(),
"host": platform.node(),
"started_utc": utc_now_text(),
},
"status": "initialized",
"freezeable": False,
"source_hashes": dict(source_hashes),
"canonical_spec": canonical_data(spec),
"scientific_identity_payload": scientific_identity_payload(spec, source_hashes),
"model": _model_manifest(spec),
"protocol": {
"T_sim": spec.protocol.T_sim,
"burn_end_time": spec.protocol.burn_end_time,
"split_time": spec.protocol.split_time,
"future_duration": spec.protocol.future_duration,
"diagnostic_stride": spec.protocol.diagnostic_stride,
"branches": [branch.engine_branch for branch in spec.branches],
},
"study": {
"rho_values": rho_values,
"T_window_values": twindow_values,
"n_seeds": spec.seed_design.count,
"statistical_unit": "paired seed/graph",
"expected_condition_count": spec.artifact_policy.expected_condition_count,
"expected_scalar_row_count": spec.artifact_policy.expected_scalar_row_count,
},
"randomization": {
"master_seed": spec.seed_design.master_seed,
"paired_seeds": list(spec.seed_design.paired_seeds),
"seed_generation": spec.seed_design.generation_rule,
"branch_rng_namespace": list(spec.branch_rng_namespace),
```

```
"executed_branches": [branch.engine_branch for branch in spec.branches],
"branch_rng_child_indices": {
branch.engine_branch: branch_rng_child_index(spec, branch.engine_branch)
for branch in spec.branches
},
"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,
},
"schema": {
"core_schema_version": collector.core_schema_version,
"study_schema_version": collector.study_schema_version,
"collectors": [
{
"collector_id": item.collector_id,
"schema_version": item.schema_version,
"core_schema_version": item.core_schema_version,
"study_schema_version": item.study_schema_version,
}
for item in spec.collectors
],
"artifact_naming_policy_id": spec.artifact_policy.policy_id,
},
"artifact_authority": {
"manifest": "execution_authority",
"consolidated_csv": "authoritative_study_output",
"consolidated_npz": "authoritative_study_output",
"condition_checkpoints": spec.artifact_policy.checkpoint_role,
"catalog": "derived_non_authoritative",
"analysis_directory": "analysis_layer_only",
},
"environment": {
"python": platform.python_version(),
"numpy": np.__version__,
"platform": platform.platform(),
},
"artifacts": {
"scalar_csv": paths.scalar_csv.name,
"branch_npz": paths.branch_npz.name,
"condition_status": paths.condition_status.name,
"conditions_directory": paths.conditions.name,
},
}
```

```
def _validate_authority_paths(
frozen_manifest: str | Path,
frozen_npz: str | Path,
frozen_scalar: str | Path,
) -> dict[str, Path]:
paths = {
"manifest": Path(frozen_manifest),
"npz": Path(frozen_npz),
"scalar": Path(frozen_scalar),
}
for label, path in paths.items():
if not path.is_file():
raise ArtifactIntegrityError(f"Frozen {label} authority not found: {path}")
return paths



def _run_baseline_gate(
*,
engine: Any,
spec: ExperimentSpec,
authority: Mapping[str, Path],
) -> dict[str, Any]:
reference_model = reference_model_config(spec)
baseline_protocol = build_engine_protocol(engine, spec, tuple(spec.branch_rng_namespace))
reference_model.validate()
baseline_protocol.validate(reference_model)
return engine.run_baseline_equivalence(
seeds=list(spec.seed_design.paired_seeds),
config=reference_model,
protocol=baseline_protocol,
frozen_manifest_path=str(authority["manifest"]),
frozen_npz_path=str(authority["npz"]),
frozen_csv_path=str(authority["scalar"]),
branch_order=tuple(spec.branch_rng_namespace),
)



def _validate_baseline_result(result: Mapping[str, Any], spec: ExperimentSpec) -> None:
if result.get("status") != "passed":
```

```
raise BaselineGateError(
f"Mandatory frozen baseline gate failed: {result.get('error', 'per-seed mismatch')}"
)
if int(result.get("n_seeds", -1)) != spec.baseline_gate.expected_seed_count:
raise BaselineGateError("Baseline result seed count is incorrect")
seed_results = result.get("seed_results", {})
if set(seed_results) != {str(seed) for seed in spec.seed_design.paired_seeds}:
raise BaselineGateError("Baseline certificate seed cohort is incomplete")
if any(item.get("status") != "passed" for item in seed_results.values()):
raise BaselineGateError("At least one baseline seed is not certified")



def _write_condition_status(path: Path, rows: list[dict[str, Any]]) -> None:
if not rows:
return
failed = any(row.get("status") == "failed" for row in rows)
fields = ("seed", "rho", "status", "elapsed_seconds", "error") if failed else (
"seed",
"rho",
"status",
"elapsed_seconds",
)
normalized = []
for row in rows:
normalized.append({field: row.get(field, "") for field in fields})
atomic_write_csv(path, normalized, fields)



def _simulate_condition(
*,
engine: Any,
spec: ExperimentSpec,
condition: Condition,
collector: TWindowLegacyCollectorV1,
) -> tuple[list[dict[str, Any]], dict[str, Any]]:
# A new ModelConfig and a new prefix are created for every condition. There
# is intentionally no split-state cache in modular platform v1.
model = model_for_condition(spec, condition)
protocol = build_engine_protocol(
engine,
spec,
```

```
tuple(branch.engine_branch for branch in spec.branches),
)
model.validate()
protocol.validate(model)


split_state, prefix = engine.simulate_prefix(condition.seed, model, protocol)
rngs = make_branch_rngs(engine, condition.seed, tuple(spec.branch_rng_namespace))
branch_results: dict[str, dict[str, Any]] = {}
rho = float(condition.value("rho"))
for branch_spec in spec.branches:
branch = branch_spec.engine_branch
result = engine.simulate_future(
split_state.clone(),
prefix,
branch,
model,
protocol,
rngs[branch],
)
result["T_window"] = model.T_window
result["rho"] = rho
branch_results[branch] = result


collector.validate_engine_result(
engine=engine,
condition=condition,
model=model,
protocol=protocol,
prefix=prefix,
branch_results=branch_results,
)
rows = collector.scalar_rows(condition=condition, branch_results=branch_results)
payload = collector.array_payload(
condition=condition,
model=model,
prefix=prefix,
branch_results=branch_results,
)
return rows, payload
```

```
def execute(
registration: StudyRegistration,
*,
mode: str,
frozen_manifest: str | Path,
frozen_npz: str | Path,
frozen_scalar: str | Path,
output_root: str | Path,
resume: bool = False,
) -> ExecutionPaths:
"""Run baseline-only or complete production execution."""
if resume:
raise UnsupportedPolicyError(
"Resume is not implemented in modular platform v1; checkpoints are crash diagnostics only"
)
if mode not in {"baseline", "production"}:
raise ValueError(f"Unsupported execution mode: {mode}")


spec = registration.build()
engine = importlib.import_module(spec.engine_module)
validate_engine_compatibility(engine, spec)
source_hashes = compute_source_hashes(spec, registration, engine)
full_spec_hash = spec_sha256(spec)
scientific_hash = scientific_run_sha256(spec, source_hashes)
run_id = scientific_hash[:RUN_ID_LENGTH]
execution_id = make_execution_id(mode)
paths = execution_paths(output_root, spec, run_id, mode, execution_id)
create_execution_directory(paths)


manifest = _initial_manifest(
spec=spec,
spec_hash=full_spec_hash,
scientific_hash=scientific_hash,
run_id=run_id,
execution_id=execution_id,
execution_mode=mode,
source_hashes=source_hashes,
paths=paths,
)
atomic_write_json(paths.manifest, manifest)
```

```
current_condition: Condition | None = None
status_rows: list[dict[str, Any]] = []
started = time.time()


try:
with ExecutionLock(paths.lock_file):
authority = _validate_authority_paths(frozen_manifest, frozen_npz, frozen_scalar)
manifest["frozen_authority"] = {
"manifest_file_sha256": file_sha256(authority["manifest"]),
"npz_file_sha256": file_sha256(authority["npz"]),
"scalar_csv_file_sha256": file_sha256(authority["scalar"]),
}
try:
with open(authority["manifest"], "r", encoding="utf-8") as handle:
frozen_data = json.load(handle)
manifest["frozen_authority"]["production_source_sha256"] = frozen_data.get(
"source_sha256", "NOT_RECORDED"
)
except Exception as exc:
raise ArtifactIntegrityError(f"Cannot read frozen manifest: {exc}") from exc
atomic_write_json(paths.manifest, manifest)


baseline = _run_baseline_gate(engine=engine, spec=spec, authority=authority)
# Persist both passing and failing baseline diagnostics before the
# gate is interpreted. A failed certificate is never reusable.
atomic_write_json(paths.baseline_certificate, baseline)
atomic_write_json(paths.baseline_by_seed, baseline.get("seed_results", {}))
_validate_baseline_result(baseline, spec)
manifest["baseline_equivalence"] = {
"status": "passed",
"n_seeds_tested": baseline["n_seeds"],
"n_seeds_passed": baseline["n_seeds"],
"branches_validated": list(spec.branch_rng_namespace),
"certificate_path": paths.baseline_certificate.name,
"seed_results_path": paths.baseline_by_seed.name,
"certificate_sha256": file_sha256(paths.baseline_certificate),
"seed_results_sha256": file_sha256(paths.baseline_by_seed),
"engine_source_sha256": source_hashes["engine"],
"runner_source_sha256": source_hashes["runner"],
"study_source_sha256": source_hashes["study"],
}
atomic_write_json(paths.manifest, manifest)
```

```
if mode == "baseline":
manifest.update(
{
"status": "baseline_only_completed",
"freezeable": False,
"completion": {
"status": "baseline_only_completed",
"study_grid_started": False,
"baseline_passed": True,
"elapsed_seconds": time.time() - started,
},
"output_hashes": {
"baseline_certificate": file_sha256(paths.baseline_certificate),
"baseline_seed_results": file_sha256(paths.baseline_by_seed),
},
}
)
atomic_write_json(paths.manifest, manifest)
return paths


conditions = expand_conditions(spec)
manifest["conditions"] = [
{
"condition_id": condition.condition_id,
"seed": condition.seed,
"seed_index": condition.seed_index,
"factor_levels": {
factor_name: {
"level_id": level.level_id,
"label": level.label,
"value": level.value,
"metadata": dict(level.metadata),
}
for factor_name, level in condition.factor_levels
},
"factor_values": dict(condition.factor_values),
"model_config_hash": condition.model_config_hash,
"protocol_hash": condition.protocol_hash,
"expected_branches": list(condition.expected_branches),
}
for condition in conditions
```

```
]
atomic_write_json(paths.manifest, manifest)


collector_spec = spec.collectors[0]
collector = build_collector(collector_spec)
if not isinstance(collector, TWindowLegacyCollectorV1):
raise UnsupportedPolicyError("Only TWindowLegacyCollectorV1 is supported in v1")


all_rows: list[dict[str, Any]] = []
consolidated_payload: dict[str, Any] = {}
successful = 0


for current_condition in conditions:
condition_started = time.time()
try:
condition_rows, condition_payload = _simulate_condition(
engine=engine,
spec=spec,
condition=current_condition,
collector=collector,
)
overlap = set(consolidated_payload) & set(condition_payload)
if overlap:
raise ConditionSchemaError(
f"Consolidated NPZ key collision: {sorted(overlap)[:10]}"
)
checkpoint = paths.conditions / collector.checkpoint_filename(current_condition)
atomic_write_npz(checkpoint, condition_payload)


# Commit in-memory consolidation only after checkpoint success.
all_rows.extend(condition_rows)
consolidated_payload.update(condition_payload)
status_rows.append(
{
"seed": current_condition.seed,
"rho": current_condition.value("rho"),
"status": "success",
"elapsed_seconds": time.time() - condition_started,
}
)
_write_condition_status(paths.condition_status, status_rows)
```

```
successful += 1
except Exception as exc:
status_rows.append(
{
"seed": current_condition.seed,
"rho": current_condition.value("rho"),
"status": "failed",
"elapsed_seconds": time.time() - condition_started,
"error": f"{type(exc).__name__}: {exc}",
}
)
_write_condition_status(paths.condition_status, status_rows)
raise


expected_conditions = spec.artifact_policy.expected_condition_count
expected_rows = spec.artifact_policy.expected_scalar_row_count
if successful != expected_conditions:
raise ArtifactIntegrityError(
f"Successful condition count {successful} != {expected_conditions}"
)
if len(all_rows) != expected_rows:
raise ArtifactIntegrityError(f"Scalar row count {len(all_rows)} != {expected_rows}")
checkpoint_count = len(list(paths.conditions.glob("condition_seed_*_rho_*.npz")))
if checkpoint_count != expected_conditions:
raise ArtifactIntegrityError(
f"Checkpoint count {checkpoint_count} != {expected_conditions}"
)


all_rows.sort(key=lambda row: (int(row["seed_index"]), float(row["rho"]), str(row["branch"])))
atomic_write_csv(paths.scalar_csv, all_rows, collector.scalar_fieldnames())
atomic_write_npz(paths.branch_npz, consolidated_payload)


output_hashes = {
"scalar_csv": file_sha256(paths.scalar_csv),
"branch_npz": file_sha256(paths.branch_npz),
"condition_status_csv": file_sha256(paths.condition_status),
"baseline_certificate": file_sha256(paths.baseline_certificate),
"baseline_seed_results": file_sha256(paths.baseline_by_seed),
}
manifest.update(
{
```

```
"status": "completed",
"freezeable": True,
"condition_status": {
"path": paths.condition_status.name,
"sha256": output_hashes["condition_status_csv"],
"expected_rows": expected_conditions,
"successful_rows": successful,
"failed_rows": 0,
},
"checkpoints": {
"role": spec.artifact_policy.checkpoint_role,
"count": checkpoint_count,
"authoritative_consolidation": paths.branch_npz.name,
},
"completion": {
"status": "completed",
"n_conditions_requested": expected_conditions,
"n_conditions_successful": successful,
"n_conditions_failed": 0,
"n_scalar_rows": len(all_rows),
"n_matrix_entries": len(consolidated_payload),
"elapsed_seconds": time.time() - started,
"freezeable": True,
},
"output_hashes": output_hashes,
}
)
atomic_write_json(paths.manifest, manifest)
return paths


except Exception as exc:
trace_path = paths.failure / "traceback.txt"
atomic_write_text(trace_path, traceback.format_exc())
failure_artifacts: dict[str, Any] = {}
if paths.baseline_certificate.exists():
failure_artifacts["baseline_certificate"] = {
"path": paths.baseline_certificate.name,
"sha256": file_sha256(paths.baseline_certificate),
}
if paths.condition_status.exists():
failure_artifacts["condition_status"] = {
"path": paths.condition_status.name,
```

```
"sha256": file_sha256(paths.condition_status),
}
failure = {
"status": "failed",
"freezeable": False,
"failure": {
"exception_type": type(exc).__name__,
"message": str(exc),
"condition_id": current_condition.condition_id if current_condition else None,
"traceback_path": str(trace_path.relative_to(paths.leaf)),
"elapsed_seconds": time.time() - started,
"committed_condition_rows": len(status_rows),
"diagnostic_artifacts": failure_artifacts,
},
}
manifest.update(failure)
atomic_write_json(paths.manifest, manifest)
atomic_write_json(paths.failed_manifest, manifest)
raise
```

Try again.

With runner.py visible, the central S01 execution path is much clearer. The most important numerical requirement—fresh initialization for every (seed,ρ)(seed,ρ)—is implemented correctly.
Provisional verdict

S01 numerical orchestration: strong
Full-regeneration enforcement: correct
Branch RNG handling: correct
Baseline gate: correctly mandatory
Condition stopping/failure behavior: strong
Artifact completion logic: good
General modularity: still partially T-window-specific
Certification/provenance: needs several final hardening changes
Final approval: pending spec.py, validation.py, oracle.py, and registry.py

There is no obvious numerical drift from the monolithic S01 driver in the shown runner.
The most important success: full regeneration is real

This code establishes the required condition-level regeneration:

Python

```
for current_condition in conditions:
...
_simulate_condition(...)
```

and inside:

Python

```
model = model_for_condition(
spec,
condition,
)

split_state, prefix = engine.simulate_prefix(
condition.seed,
model,
protocol,
)
```

There is:

```
no split-state cache;
no prefix cache;
no reuse across rho;
a newly resolved ModelConfig for every condition;
a newly simulated prefix for every condition.
```

This correctly enforces:

text

```
changed T_window -> regenerate from initialization
```

The comment and implementation agree.

S01 compatibility flow

The following matches the successful driver:

Resolve Twindow=ρT∗Twindow=ρT∗.
Simulate a fresh prefix.
Create RNGs using all seven frozen branch names.
Execute the four declared study branches.
Preserve future_shuffle child index 6.
Add rho and T_window metadata.
Validate through the legacy collector.
Write per-condition checkpoint.
Append branch rows and flat payload only after checkpoint success.
Consolidate after 200 successful conditions.
Sort scalar rows by seed index, rho, and branch.
Write legacy-compatible filenames and schemas.

The deliberate double clone:

Python

split_state.clone()

followed by the engine’s internal clone matches the monolithic compatibility path. Do not optimize it before oracle testing.
Strong runner features
Mandatory baseline

There is no production bypass:

Python

baseline = _run_baseline_gate(...)
_validate_baseline_result(...)

Both baseline-only and production modes execute the gate.
Baseline diagnostics survive failure

The certificate and seed results are written before interpretation:

Python

```
atomic_write_json(
paths.baseline_certificate,
baseline,
)
```

That is good failure evidence.
Typed failure flow